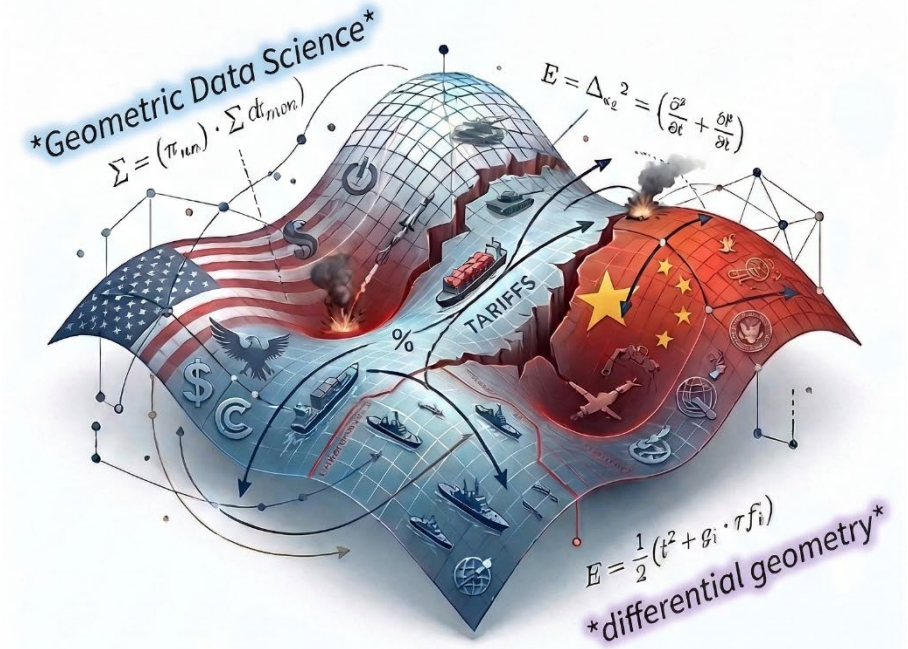
Lumière de l'Intelligence de la Géométrie Différentielle

『ジオメトリック・インテリジェンス』

(GEOMETRIC INTELLIGENCE)



- ✔ 最先端の<幾何学的AI>を使って立体的な<経営環境・政策環境>地形図(可微分多様体)を構築。
- ✔ 立体地形図上で、アインシュタイン方程式で使われるる共変微分・リー微分を計算。
- ✔ 最小リスク・最大効果の経営・政策アクションを導き出す。

ジオメトリック・インテリジェンス

自然界の「かたち」は、曲がった高次元空間でできている

物理学者や化学者は、**100年以上前から**、自然界の「かたち」が—私たちの目に見える3次元の空間だけでなく—**微分ができる高次元の曲がった空間**であることに気づいていました。数学では、この空間を「**多様体（manifold）**」と呼びます。

1854年、ドイツの数学者ベルンハルト・リーマンが、ゲッティンゲン大学での就任講演「幾何学の基礎をなす仮説について」において、「曲がった高次元空間の幾何学」—後にリーマン幾何学と呼ばれる数学体系—を創始しました。リーマンの革新は、「空間の曲がり具合」を各点ごとに定義される「計量テンソル」という数学的対象で記述したことにあります。平面や球面のような2次元の曲面だけでなく、3次元、4次元、あるいは任意の n 次元の「曲がった空間」の幾何学を、統一的な数学的言語で扱えるようにしたのです。

1915年、アルベルト・アインシュタインがこのリーマン幾何学を使って一般相対性理論を完成させました。アインシュタインの理論は、ニュートンの万有引力の法則を置き換えるものです。ニュートンは「質量の間に引力が働く」と考えましたが、アインシュタインは「質量が空間そのものを曲げ、その曲がった空間の幾何学が物体の運動を決定する」と考えました。宇宙の重力は、「4次元の曲がった空間（時空多様体）」の幾何学で完全に記述されるのです。

以来110年間、物理学者はこの数学を使い続けています。GPSの衛星軌道の計算には一般相対性理論の補正が不可欠であり、もしこの補正を行わなければ位置の誤差は1日あたり約10キロメートルに達します。ブラックホールの写真撮影（2019年、Event Horizon Telescope）も、曲がった時空の中を光がどう進むかをリーマン幾何学で計算することで初めて可能になりました。

化学者は、分子の形の変化が「高次元の曲がった面（コンフォメーション空間）」の上の運動であることを知っています。タンパク質の折りたたみ—アミノ酸の鎖が立体的な構造を形成する過程—は、数千次元のコンフォメーション空間上の経路探索問題であり、分子動力学シミュレーションはこの多様体上の数値計算として実行されます。

ロボット工学者は、ロボットの動きが「関節角度が作る曲がった空間」の上の経路であることを利用して制御しています。たとえば、6つの関節を持つロボットアームの構成空間は、各関節角度が $[0,2\pi)$ の範囲を動くため、6次元のトーラス T^6（ドーナツの6次元版）になります。ロボットの最適な動きは、この6次元トーラス上の測地線（最短経路）として計算されます。

そして、これらの分野ではすべて、「曲がった空間の上で微分計算をする」ことで、予測・制御・最適化を行っています。 微分計算とは、「空間上の量がどの方向にどれだけ速く変化するか」を計算することです。物理学者は時空の曲率を微分計算で求め、化学者は分子のエネルギー面の勾配を微分計算で求め、ロボット工学者は関節トルクの最適値を微分計算で求めています。

なぜ、これが経営者と政策立案者にとって決定的に重要なのか

本書は、この同じ数学的手法を、企業の経営環境と国家の政策環境に応用します。 そしてそれは、経営者と政策立案者にとって、これまでの意思決定を根本から変えうるインパクトを持っています。

現在の意思決定は「平面の地図」で行われている

現在、企業の経営判断や政府の政策決定を支えるデータ分析の大半は、**直線的な手法**で行われています。回帰分析は「$y = ax + b$ のような直線で関係を近似する」手法です。相関分析は「2つの変数が直線的にどれだけ連動するか」を測定します。主成分分析（PCA）は「データの分散が最大になる直線的な方向」を見つけます。線形最適化は「制約条件が直線（超平面）で表されるとき、目的関数を最大化する点」を求めます。

これらはすべて、**「世界は直線でできている」という暗黙の仮定**の上に成り立っています。

しかし、現実の経営環境や国際情勢は直線的ではありません。いくつかの具体例を挙げましょう。

為替レートの非線形性。 為替が1%動いたときの企業業績への影響は、為替が90円（円高局面）のときと150円（円安局面）のときでまったく異なります。円高局面では1%の追加円高が輸出企業に壊滅的な影響を与えますが、円安局面での1%の追加円安の影響は限定的かもしれません。つまり、「同じ1%の変動」でも、「どの位置にいるか」によって影響がまったく違うのです。これは、直線的な回帰分析（業績 $= a \times$ 為替 $+ b$）では捉えられない非線形構造です。

金利政策の非線形性。 金利を0.25%上げたときの市場の反応は、金利が0%（ゼロ金利環境）のときと5%（高金利環境）のときでまったく異なります。ゼロ金利からの利上げは「レジームチェンジ」（体制転換）を引き起こしうるほど市場に大きな衝撃を与えますが、5%から5.25%への利上げの影響は相対的に小さいでしょう。

同盟政策の非線形性。 同盟関係を強化したときの安全保障環境への影響は、国際環境が安定しているときと緊張しているときでまったく異なります。安定期の同

盟強化は「保険」として機能しますが、緊張期の同盟強化は「挑発」と受け取られ、むしろ緊張を高める可能性があります。

つまり、「同じアクション」でも、「どこで打つか」によって結果がまったく異なるのです。 これこそが「空間が曲がっている」ことの本質的な意味です。

これは、地球の表面を平面地図で見ているのと同じ問題です。メルカトル図法（多くの世界地図で使われている図法）では、グリーンランドがアフリカ大陸より大きく見えます。しかし実際には、アフリカ大陸の面積はグリーンランドの約14倍です。平面の地図で最短距離だと思って引いた直線は、実際の地球上では最短経路ではありません。東京からニューヨークへの最短飛行ルートは、太平洋を横断する直線（地図上では短く見える）ではなく、北極圏を通る大圏航路（地図上では大回りに見える）です。**平面地図の直線を信じて飛行すれば、燃料を無駄にし、時間を浪費し、最悪の場合は目的地に到達できません。**

経営判断や政策決定でも、まったく同じことが起きています。 直線的な分析手法（回帰分析、PCA、線形最適化）は、経営環境の「メルカトル図法」です。見た目はわかりやすいですが、現実の「曲がった」経営環境を平面に押し込めているため、距離も方向も歪んでいます。

「曲がった地形図」で意思決定すると何が変わるか

本書の手法は、経営環境や政策環境を**「曲がった立体地形図」**として構築します。この地形図は、単なる比喩ではなく、数学的に厳密に定義されたリーマン多様体であり、その上であらゆる微分計算が可能です。この地形図の上では、以下のことが可能になります。

1.「どこが危険か」が事前にわかる。

平面地図では「A地点とB地点は近い」としか言えません。立体地形図では「A地点とB地点の間に標高3,000メートルの山脈がある」ことがわかります。

本書の数学的な言葉でいえば、「2つの経営状態の間にスカラー曲率が負の領域（不安定構造）が存在する」ことが、多様体の曲率計算で判明します。経営の言葉に翻訳すると、「新興国市場への進出経路上に、構造的な不安定領域—経済危機や政治的混乱に対して脆弱な状態—がある」ことを、**実際に進出する前に**知ることができるのです。

「スカラー曲率が負」とは何を意味するのか。直感的には、球面の上にボールを置くと転がり落ちますが（正の曲率：安定）、馬の鞍（サドル）の上にボールを置くと、ある方向には安定ですが別の方向には不安定です（負の曲率：不安

定）。経営環境の多様体上で曲率が負の領域は、「ちょっとした外乱で経営状態が大きく変動しうる、構造的に不安定な領域」です。

2.「アクションが環境をどう変えるか」をシミュレーションできる。

従来のデータ分析は、基本的に「打ったアクションの結果を後から測定する」ものでした。A/Bテスト、差分の差分析（DID）、回帰不連続デザイン—これらはすべて、「何かをやった後に、結果を観察する」手法です。

本書の手法は根本的に異なります。「アクションを打つ前に、そのアクションが経営環境や国際情勢の**構造そのもの**をどう変形させるかをシミュレーションする」ことを可能にします。

数学的には、政策や経営施策を「ベクトル場」（多様体の各点に方向と大きさを割り当てる関数）としてモデル化し、そのベクトル場のLie微分 $\mathcal{L}_V g$ を計算します。Lie微分がゼロならば（Killing方程式 $\mathcal{L}_V g = 0$ が成立）、その施策は「経営環境の構造を保存したまま、状態だけを移動させる」穏やかな施策です。Lie微分がゼロでなければ、その施策は「経営環境の構造そのもの—どの状態が近くてどの状態が遠いか、どこが安定でどこが不安定か—を変えてしまう」構造変革型の施策です。

数十億円の投資判断や、外交関係を根本から変える政策決定を、**「打ってみないとわからない」から「打つ前に構造変化をシミュレーションできる」に変えるのです。**

3. 複数のアクションが「互いに邪魔し合うか」がわかる。

企業がコスト削減と市場拡大を同時に進めるとき、2つの施策が互いに打ち消し合う場合があります。コスト削減のために研究開発費を減らすと、新市場での競争力が落ちるかもしれません。逆に、市場拡大のために大量の投資を行うと、コスト削減の効果が相殺されるかもしれません。

政府が経済制裁と同盟強化を同時に行うとき、一方が他方の効果を損なう場合があります。制裁対象国との貿易を止めると、その国と取引のある同盟国の経済にも影響が及び、同盟関係が弱体化する可能性があります。

従来の統計分析では、「交互作用項を回帰式に入れて、統計的に有意かどうかを検定する」ことしかできません。これでは「干渉がある/ない」の2値的な判断しかできず、「どの方向にどれだけ干渉するか」はわかりません。

本書の手法では、2つのベクトル場（2つの施策）のLie括弧 $[V_A, V_B]$ を計算します。Lie括弧は「2つの施策を順番に実行したとき、実行順序によって結果がどれ

だけずれるか」を定量的に測定します。Lie括弧がゼロなら2つの施策は干渉しない（順序に依存しない）。ゼロでなければ、**干渉の方向と大きさ**がベクトルとして得られます。

4.「最もリスクの低い経路」が計算できる。

企業が現在の経営状態から目標とする経営状態に移行するとき、「最短距離」で移行するのが最良とは限りません。最短距離のルートが「不安定領域」（負の曲率の領域）を通過するなら、その途上で経営環境の急変に遭遇するリスクがあります。

平面地図の直線は最短距離ですが、立体地形図上では、山脈を越える直線ルートより、山脈を迂回する遠回りの方がはるかに安全です。同様に、経営環境の多様体上の**測地線**（幾何学的にエネルギーが最小の経路）は、直線的な計画とはまったく異なる「最適ルート」を示すことがあります。

さらに、ポントリャーギンの最大原理（最適制御理論の中心定理）を用いると、「利用可能な施策の制約の下で、コストを最小化する移行経路」を計算できます。これは、測地線（自由落下の経路）に対して「制御力」を加えた最適経路です。

要約すると、本書の手法は、意思決定を「平面地図で飛行計画を立てること」から「地球儀と3D地形データで飛行計画を立てること」に進化させます。 必要な計算は増えますが、得られる判断の質は根本的に向上します。目的地への到達確率が上がり、途中の危険を事前に回避でき、燃料（コスト）を最適化できます。

【図：図0-1：直線的分析vs幾何学的インテリジェンス】 *図0-1：直線的分析（左：平面地図、危険領域を直線で通過）と幾何学的インテリジェンス（右：曲がった多様体、測地線が危険領域を回避）の比較*

最先端の〈幾何学的AI〉を使って立体的な〈経営環境・政策環境〉地形図（可微分多様体）を構築。立体地形図上で、アインシュタイン方程式で使われる共変微分・リー微分を計算し、最小リスク・最大効果の経営・政策アクションを導き出す。

本書の3つの読み方

本書は、読者の知識背景に応じて3つの入口を用意しています。各章で以下のマークが付いたセクションを辿ってください。すべてのセクションを通読する必要はなく、自分の知識背景に応じて必要な部分だけを読めるように設計されています。

▶ **数学の基礎から理解したい方へ** — 微分幾何学の定義・定理・証明を、高校数学からの橋渡し付きで記述しています。微分幾何学を学習済みの方は、橋渡し部分（「【高校数学からの橋渡し】」と表示されたボックス）を飛ばし、定義・定理・証明を直接お読みください。微分幾何学が初めての方は、橋渡し部分から読んでください。なお、微分幾何学に精通している数学者であっても、VAE（変分オートエンコーダ）やNeural ODEなどのAIモデルには馴染みがない場合が多いと思います。その場合は、「▶ AIモデルの基礎から理解したい方へ」のセクションを併せてお読みください。

▶ **AIモデルの基礎から理解したい方へ** — VAE（変分オートエンコーダ）、Vector Diffusion Maps、Neural ODE等の幾何学的AIモデルを、「なぜこのモデルを使うのか」「内部で何が起きているのか」「Pythonコードの各行は何をしているのか」のレベルで解説します。機械学習やディープラーニングに日常的に触れているエンジニアの方は、基礎的な解説を飛ばして、本書固有のポイント（デコーダの滑らかさ条件、ヤコビ行列のフルランク検証など）に集中してください。逆に、AIモデルに馴染みのない数学者や経営者の方は、このセクションを重点的にお読みください。

▶ **ビジネス・政策の意思決定に活用したい方へ** — 数式とコードは最小限にし、2つのケーススタディを通じて「何が計算され、何がわかり、どう意思決定に使えるか」を記述します。ケーススタディA（製造業X社の経営環境多様体）とケーススタディB（国家安全保障会議の国際情勢多様体）が各章で進行します。

> **知識マトリクスのどこにいても読める設計です。** 微分幾何学に強いがVAEを知らない数学者は「▶ AIモデル」を重点的に。AIに強いが微分幾何学を知らないエンジニアは「▶ 数学」を重点的に。両方とも基礎レベルの経営者・政策立案者は「▶ ビジネス」を辿ってください。すべてに精通した方は全セクションを通読してください。

序章（続き）：なぜ今、社会データに幾何学的AIが必要なのか

1. 物理学と工学は、すでに「多様体の上」で計算している

▶ 数学の基礎から理解したい方へ

アインシュタインの一般相対性理論は、重力を4次元擬リーマン多様体 (M, g) 上の曲率として記述する。アインシュタイン方程式

$$R_{\mu\nu} - \frac{1}{2} g_{\mu\nu} R + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu}$$

の左辺に現れるリッチテンソル $R_{\mu\nu}$、スカラー曲率 $R = g^{\mu\nu} R_{\mu\nu}$、計量テンソル $g_{\mu\nu}$ は、すべてレヴィ・チヴィタ接続 ∇ から導かれるリーマン曲率テンソル ${R^{\rho}}_{\sigma\mu\nu}$ の縮約として定義される。

この方程式が意味していることを、微分幾何学の言葉で正確に述べよう。左辺はリーマン多様体 (M, g) の幾何学的性質—時空がどれだけ曲がっているか—を記述しており、右辺はエネルギー・運動量テンソル $T_{\mu\nu}$ —時空中の物質やエネルギーの分布—を記述している。この方程式は「物質が時空を曲げ、時空の曲がりが物質の運動を決定する」という双方向の関係を定式化している。

リーマン曲率テンソルの定義を明示すると：

$${R^{\rho}}_{\sigma\mu\nu} = \partial_\mu \Gamma^{\rho}_{\nu\sigma} - \partial_\nu \Gamma^{\rho}_{\mu\sigma} + \Gamma^{\rho}_{\mu\lambda} \Gamma^{\lambda}_{\nu\sigma} - \Gamma^{\rho}_{\nu\lambda} \Gamma^{\lambda}_{\mu\sigma}$$

ここで $\Gamma^{\rho}_{\mu\nu}$ はレヴィ・チヴィタ接続のクリストッフェル記号であり、計量テンソルから

$$\Gamma^{\rho}_{\mu\nu} = \frac{1}{2} g^{\rho\lambda} \left(\partial_\mu g_{\nu\lambda} + \partial_\nu g_{\mu\lambda} - \partial_\lambda g_{\mu\nu} \right)$$

で計算される。リッチテンソルは曲率テンソルの縮約 $R_{\mu\nu} = {R^{\lambda}}_{\mu\lambda\nu}$ であり、スカラー曲率はさらに計量で縮約 $R = g^{\mu\nu} R_{\mu\nu}$ して得られる。

物理学・工学において多様体上のテンソル計算が用いられる分野は広範にわたる。

量子力学・場の量子論。 ゲージ理論は主ファイバー束 $P(M, G)$（M：基底多様体、G：構造群）上の接続として定式化される。電磁気学は $U(1)$ ゲージ理論、弱い力は $SU(2)$ ゲージ理論、強い力は $SU(3)$ ゲージ理論である。共変微分 $D_\mu = \partial_\mu + igA_\mu$（$A_\mu$ はゲージ場）がゲージ変換の下で整合的に変換されることが理論の無矛盾性を保証する。

弦理論と量子重力理論。 高次元多様体（10次元や11次元）の余剰次元がカラビ・ヤウ多様体としてコンパクト化され、その幾何学的性質（ホッジ数など）が4次元時空の素粒子の種類や性質を決定する。

ロボティクスと制御工学。 n 個の回転関節を持つロボットの構成空間は n 次元トーラス $T^n = \left(S^1\right)^n$ である。ロボットの先端を目標位置に最短時間で移動させる問題は、T^n 上の測地線問題またはリーマン多様体上の最適制御問題として定式化される。構成空間上のリーマン計量は、各関節のイナーシャ（慣性モーメント）に対応する。

流体力学。 曲面上の流れは、曲面の接束 TM 上のベクトル場として記述され、ナビエ・ストークス方程式は共変微分を用いて

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + \nabla_{\mathbf{u}}\mathbf{u}\right) = -\mathrm{grad}\, p + \mu\, \Delta_g \mathbf{u}$$

と書かれる。ここで $\nabla_{\mathbf{u}}\mathbf{u}$ はレヴィ・チヴィタ接続による共変微分、Δ_g はラプラス・ベルトラミ作用素である。大気の流れや海流のモデリングにおいて、地球の曲率を無視できない場合にこの定式化が用いられる。

構造力学。 応力テンソル σ_{ij} は物体内部の力の状態を記述する対称 $(0,2)$-テンソル場であり、曲面シェル構造（ドーム、冷却塔、航空機の胴体など）の設計には、曲面上のテンソル解析が不可欠である。

これらの分野で多様体上の計算が「コストに見合う」理由は明白である。 多様体構造を無視して $\mathbb{R}^n$ 上の線形近似で計算すると、GPSの軌道計算に1日あたり10キロメートルの誤差が生じ、ロボットの制御が発散し（関節角度の周期性を無視するため）、流体シミュレーションが質量保存則を破る非物理的な解を出す。**線形近似では現実を記述できないから**こそ、物理学者と工学者はコストを払って多様体上で計算するのである。

本書は、社会データについても同じ論理が成り立つことを主張する。

【高校数学からの橋渡し】 上の数式群は難しく見えますが、本質は「空間の曲がり具合を数値で計算する公式」です。

- $g_{\mu\nu}$（計量テンソル）= 「各点で距離をどう測るか」の情報。第2章で詳しく学びます。
- $\Gamma^{\rho}_{\mu\nu}$（クリストッフェル記号）= 「座標系がどれだけ曲がっているか」の補正値。第3章で学びます。

- $R^{\rho}{}_{\sigma\mu\nu}$（リーマン曲率テンソル）= 「空間がどれだけ曲がっているか」の数値。第3章で学びます。

本書では、これらすべてをPyTorchの自動微分で計算するため、数式を手で解く必要はありません。

▶ AIモデルの基礎から理解したい方へ

物理学で使われている数学的道具（リーマン計量、共変微分、曲率テンソル）を、AIモデルと組み合わせることで社会データに適用可能にしたのが、**Geometric Data Science（幾何学的データサイエンス）**と呼ばれるアプローチです。

この「翻訳」がどのように行われるかを、具体的に対応させて説明します。

物理学での使い方 → 本書でのAI＋数学による応用：

物理学	AI＋数学による社会データへの応用	使うAIモデル
時空の計量 $g_{\mu\nu}$ → 重力の効果	VAEデコーダのヤコビ行列から引き戻し計量 $g_{ij} = J^{\top}J$ → データ空間の距離構造	VAE（第5章）
曲率テンソル $R^{\rho}{}_{\sigma\mu\nu}$ → 空間の曲がり	自動微分で計量の2階微分を計算 → 経営環境の安定性/不安定性	PyTorch autograd（第7章）
測地線 → 自由落下の経路	Neural ODEで微分方程式を数値解 → 最小コストの状態遷移経路	Neural ODE（第7章）
場の理論のゲージ場 → 外力の効果	政策・施策をベクトル場として学習 → 政策効果のシミュレーション	Neural ODE（第7章）
接続ラプラシアン → 空間の接続構造	VDMで離散データから接続構造を抽出	VDM（第6章）

「物理学のために開発された数学」を「AIモデルの力を借りて」社会データに適用する ―これが本書のアプローチの核心です。

重要な点は、VAEやNeural ODEは単に「便利なツール」として使われるのではなく、**数学的に厳密な多様体構造を保証するための条件を満たすように設計される**ことです。たとえば、通常の機械学習ではReLU活性化関数が広く使われますが、本書ではReLUは使えません。なぜなら、ReLUは $x = 0$ で微分不可能であり、デコーダの像が「折れ線の集合」になってしまい、曲率テンソルの計算に必要な2階微分が定義できないからです。代わりに tanh、GELU、Softplus などの C^{∞}（何回でも微分可能な）活性化関数を使います。

このように、本書では**AIモデルの設計に数学的制約を課す**ことで、「AIモデルの出力が数学的に厳密な多様体になる」ことを保証します。これは通常の機械学習の実務とは異なるアプローチであり、第5章で詳しく解説します。

▶ ビジネス・政策の意思決定に活用したい方へ

GPS衛星の軌道計算、半導体チップの設計、ロボットの制御、大気の流れの予測—これらはすべて、「曲がった空間の上での計算」の成果です。もしGPSの軌道を「地球は平ら」と仮定して計算すれば、あなたのスマートフォンの地図アプリは使い物になりません。もしロボットの関節角度を「直線的な空間」と仮定して制御すれば、ロボットは暴走します。

「直線で近似すると致命的に間違える」からこそ、物理学者はコストを払って多様体上で計算するのです。

企業の経営環境も「曲がった空間」です。売上と広告費の関係は、広告費が少ないときと多いときでまったく異なります（収穫逓減）。為替変動と原材料価格の影響は、国内生産比率と海外生産比率によってまったく異なります。市場シェアの変化と価格競争の激しさは、業界の集中度や参入障壁の高さによって非線形に連動します。

これらの「曲がり」を無視した直線的な分析は、GPSの軌道を直線で近似するのと同じ種類の誤差を経営判断にもたらします。 その誤差が許容範囲内であれば直線的分析で十分ですが、数十億円の投資判断や国家の安全保障政策を左右する場面では、その誤差は致命的になりえます。

---

2. 社会科学・ビジネスの世界では、多様体はまだ「見る」だけの道具にとどまっている

▶ 数学の基礎から理解したい方へ

ここ10年ほどで、高次元データの低次元可視化手法が社会科学やビジネスの現場に広く普及した。代表的な手法を列挙し、その数学的限界を明確にする。

UMAP（Uniform Manifold Approximation and Projection, McInnes et al., 2018）。 データの近傍グラフを構成し、低次元空間での配置を確率的に最適化する。理論的基盤としてリーマン幾何学と代数的位相幾何学を援用するが、得られるのは離散的な点配置であり、C^k 級多様体構造は構成されない。

t-SNE（t-distributed Stochastic Neighbor Embedding, van der Maaten & Hinton, 2008）。 高次元でのガウスカーネル近傍確率と、低次元でのスチューデント

t分布近傍確率のKLダイバージェンスを最小化する。UMAPと同様、得られるのは離散点配置である。

Isomap（Tenenbaum et al., 2000）。 近傍グラフ上の最短経路距離で測地距離を近似し、多次元尺度法（MDS）で低次元に埋め込む。測地距離の近似という意味ではリーマン幾何学的な発想を含むが、やはり得られるのは離散点配置であり、連続的な計量やテンソル場は定義されない。

これらの手法に共通する本質的限界は以下である。

(i) **リーマン計量が定義されない。** 低次元空間上の2点間のユークリッド距離が、元の高次元空間での距離を定量的に反映しているかは保証されない。UMAPの論文自体が、低次元空間上のユークリッド距離の定量的解釈に注意を促している。

(ii) **微分が定義されない。** 点配置は離散的であり、C^k 級の滑らかさを持たないため、勾配、ヘッシアン（2階微分）、曲率テンソルといった微分的な量は定義できない。

(iii) **テンソル場が定義されない。** ベクトル場やテンソル場を定義するには、各点に接空間が存在し、接空間の間の関係（接続）が定義されている必要がある。離散点配置にはこれらが欠けている。

(iv) **制御系の定式化が不可能。** 多様体上のアフィン制御系 $\dot{z}^k = F^k(z) + u_\alpha B_\alpha^k(z)$ を定式化するには、滑らかなベクトル場 F, B_α が定義できる C^k 級多様体が必要である。離散点配置ではこれは不可能である。

したがって、UMAP等の可視化手法は「データの大まかな構造を視覚的に把握する」という目的には有効であるが、**共変微分、Lie微分、測地線の計算、最適制御シミュレーション**は数学的に不可能である。**これが、UMAP等に追加のコストを払って可微分多様体を構成する数学的な動機である。**

▶ AIモデルの基礎から理解したい方へ

UMAPとVAEの違いを理解することが、本書のアプローチの核心を理解する鍵です。

UMAPとVAEはどちらも「高次元データを低次元空間に写す」AIモデルですが、その出力の性質が根本的に異なります。

UMAPの内部動作（簡略版）： 1. 高次元データの各点について、k個の最近傍点を見つける 2. 近傍関係を「重み付きグラフ」（点と線の集合）として表現 3. 低次元空間（通常2次元か3次元）に点を配置し、高次元での近傍関係が低次元でもできるだけ保存されるように、点の配置を最適化する

結果として得られるのは**「離散的な点の配置」**です。点と点の間には「何もない」。ある2点の間にカーソルを置いても、そこには定義されたデータ点が存在しません。

VAEの内部動作（簡略版）： 1. **エンコーダ**（ニューラルネット）が高次元データを低次元の確率分布（平均と分散）に変換 2. その確率分布からサンプリングして潜在表現（低次元の点）を得る 3. **デコーダ**（ニューラルネット）が潜在表現から元の高次元データを復元

VAEのデコーダ $f_\theta: \mathbb{R}^d \to \mathbb{R}^n$ は**連続的な関数**です。潜在空間の**任意の点**（訓練データに対応しない点も含めて）に対して、データ空間の点を生成できます。つまり、「点と点の間」が**滑らかに埋まっている**のです。

この「滑らかさ」こそが決定的な違いです。デコーダが連続的で滑らかな関数であるため、潜在空間上で微分が定義でき、計量テンソルが定義でき、曲率が計算でき、ベクトル場が定義でき、制御シミュレーションが可能になります。

UMAPは「離散的な点の配置図」を作る。VAEは「滑らかな多様体」を作る。 この違いが、本書のアプローチを可能にする基盤です。

ただし、VAEの出力が「数学的に厳密な多様体」になるためには、デコーダに追加の条件を課す必要があります（第1章の命題1.1で述べた3条件：C^k 級、フルランク、単射）。通常のVAEでは、これらの条件は必ずしも保証されません。本書のアプローチは、これらの条件を**意図的に設計し、訓練後に検証する**ことで、VAEの出力が数学的に厳密な多様体になることを保証します。

▶ ビジネス・政策の意思決定に活用したい方へ

多くの企業が、データ分析チームに「顧客データを可視化してほしい」「市場データの全体像を見せてほしい」と依頼し、UMAPやt-SNEの散布図を受け取っています。これらの散布図は、データの大まかな構造—クラスタの数、クラスタ間の距離感、外れ値の位置—を視覚的に把握するのに便利です。

しかし、これは**「地図を見る」**行為にすぎません。地図を見て「ここに山がありそうだ」「ここに川がありそうだ」とわかることは有用ですが、地図の上で以下のことはできません。

- 「A地点からB地点への正確な距離は何キロメートルか？」—UMAPの散布図上の距離は定量的に信頼できません
- 「A地点からB地点への最短ルートはどこを通るか？」—離散的な点の間に「ルート」は定義されません

- 「この山を越えるのにどれだけのコスト（エネルギー）が必要か？」—計量が定義されていないため計算できません
- 「この施策を打ったら、現在地からどの方向にどれだけ移動するか？」—ベクトル場が定義されていないためシミュレーションできません

本書は、「地図を見る」だけでなく、**「地図の上で計算する」**ことを可能にします。それは、散布図を「リーマン多様体」（計算可能な立体地形図）に置き換えることで実現されます。

追加コストは、データサイエンスチームがVAEの設計・訓練と計量計算の手法を学ぶ時間です。リターンは、「見るだけ」から「計算して意思決定に使える」への質的な飛躍です。

---

3. なぜ、この状況が続いてきたのか

▶ 数学の基礎から理解したい方へ

社会データに微分幾何学が適用されてこなかった理由は、3つの構造的要因に帰着する。

(i) 多様体仮説の妥当性の不確実性。 物理学では、系の自由度が物理法則から決定されるため、多様体構造は理論的に保証されることが多い。たとえば、N 個の粒子の古典力学系の状態空間は $6N$ 次元多様体 $T^*\left(\mathbb{R}^{3N}\right)$（余接束）であり、これは力学の原理から導かれる。しかし、社会データの場合、データの生成メカニズムは人間の意思決定、制度設計、文化的慣習、偶発的事象の組み合わせであり、低次元多様体構造が存在するかどうかは自明でない。

さらに、社会データには物理データと比較して桁違いに深刻な問題がある。

- **欠損値の多さ。** GDP統計は四半期ごと（年4回）、企業決算は年に2〜4回、選挙データは数年に1度しか得られない。日次データがある金融市場でさえ、非営業日のギャップがある。物理学の加速器実験が毎秒数十億回のデータを記録するのとは、桁が違う。

- **観測誤差とノイズの大きさ。** アンケート調査の回答バイアス、SNSのボットによるデータ汚染、会計データの計上基準の違い（IFRSとUS GAAPの差異など）、政府統計の速報値と改定値の乖離。CERNの検出器が 10^{-21} メートルの精度で測定できるのに対し、社会データの精度は数%〜数十%の範囲でばらつく。

- **潜在変数の多さ。** 消費者の購買行動は、価格や品質だけでなく、気分、文化的背景、SNSのトレンド、将来の景気に対する期待、社会的規範、心理的バイアスなど、無数の要因に左右される。これらの多くは直接観測できない。物理系では「制御可能」か「無視可能」であることが多い変数が、社会系では本質的な役割を果たす。

(ii) 人材と教育の構造的ギャップ。 物理学や工学の大学院教育では、微分幾何学・テンソル解析・多様体論は標準的なカリキュラムに含まれる。たとえば、Misner-Thorne-Wheeler の *Gravitation* や do Carmo の *Riemannian Geometry* は物理学・数学の大学院生の標準テキストである。一方、経済学・経営学・政治学・社会学の教育課程では、微分幾何学はまず教えられない。数理経済学や金融工学でさえ、確率微分方程式（伊藤の公式）や偏微分方程式（ブラック・ショールズ方程式）は扱っても、リーマン幾何学やテンソル解析に踏み込むことは極めて稀である。

(iii) 概念的フレームワークの不在。 (i) と (ii) が重なった結果、「社会データから可微分多様体を構成し、その上でテンソル計算を行う」という発想自体が、選択肢として認知されていなかった。物理学者にとって多様体は「空間そのもの」であるから計算の道具として使うのは自然であるが、企業の経営企画部門や政府のシンクタンクにとって、この発想は概念的な跳躍を要する。

▶ AIモデルの基礎から理解したい方へ

理由を3つ、AIの観点から説明します。

(i) データが「汚い」。 物理学のデータと社会データでは、データの質が根本的に異なります。

CERNの粒子加速器LHC（Large Hadron Collider）は、毎秒6億回の陽子-陽子衝突を起こし、そこから毎秒約100万個のイベントを選別して記録します。各イベントのデータは、数千のセンサーによって高精度で測定されます。重力波検出器LIGO（Laser Interferometer Gravitational-Wave Observatory）は、10^{-21} メートル（原子核の直径の100万分の1以下）の変位を検出できます。

一方、GDP統計は四半期ごとの80データ点（20年分）しかありません。企業決算は年に数回です。選挙データは数年に1度です。金融市場の日次データ（年間約250営業日）が「高頻度」と見なされますが、物理実験の毎秒数億回とは7桁以上の差があります。

さらに、社会データには系統的なバイアスが含まれます。アンケートの回答には社会的望ましさバイアス（本音と異なる回答をする傾向）があり、SNSデータには

ボットアカウントによる汚染があり、会計データには企業ごとの計上基準の違いがあります。これらのバイアスは、物理実験のように「制御された環境」で除去することが困難です。

VAEのようなニューラルネットワークモデルは、データの量と質に大きく依存します。「ゴミを入れればゴミが出る（Garbage In, Garbage Out）」の原則は、幾何学的AIでも例外ではありません。むしろ、曲率テンソルのように計量の2階微分を含む量は、データのノイズに対して極めて敏感であり、データ品質の問題が増幅されます。

(ii) 幾何学的AIの人材がいない。 機械学習のコミュニティでも、微分幾何学を本格的に学んだ研究者は少数派です。PyTorchやTensorFlowを使ってCNN（畳み込みニューラルネット）やTransformerを実装できるエンジニアは多くいますが、リーマン計量やクリストッフェル記号をPyTorchで計算する方法を知っているエンジニアはごくわずかです。

逆に、微分幾何学の専門家である数学者は、VAEの訓練方法やPyTorchの使い方を知らないことが多いです。「数学の知識」と「AIの実装力」の両方を持つ人材が極めて希少であることが、この分野の発展を阻んできました。

(iii) Geometric Data Science自体が新しい分野。 Geometric Deep Learningの統一的な枠組みを提示したBronstein et al.の論文は2021年であり、わずか数年前です。VAEの潜在空間にリーマン計量を導入したArvanitidis et al.の論文（Latent Space Oddity）は2018年です。社会データからの多様体構成と制御シミュレーションを統合したアプローチは、本書が提示する試みの中核をなすものであり、まだ学術研究としても発展途上です。

▶ 経営者・政策立案者の方へ

なぜこの技術が今まで使われなかったのか。理由は3つあり、いずれも理解しておく価値があります。

第一に、企業や政府のデータは、物理学のデータと比べて「汚い」のです。 CERNの加速器が毎秒6億回のデータを記録するのに対し、GDP統計は四半期に1回です。企業の月次売上データは年間12点しかありません。さらに、データには人間の偏り（バイアス）が含まれます。アンケートの回答は本音と異なる場合があり、会計データは計上基準によって変わります。このような「汚い」データから、物理学者が使うような精密な数学的構造を構築できるのか—これが長年の疑問でした。

本書は、この疑問に対する回答を提供します。答えは「条件付きでYes」です。データが一定の条件（第7節の5つのチェック）を満たせば、十分な品質の多様体を

構築できます。条件を満たさなければ、構築できません—そして**その判断自体が有用な情報**です。

第二に、この数学を知る人が、ビジネスや政策の現場にいなかったのです。 企業のデータサイエンスチームは、統計学・機械学習・データエンジニアリングに強い人材で構成されていますが、微分幾何学を専門的に学んだ人材が含まれていることはほとんどありません。政府の政策分析部門も同様です。本書は、この知識ギャップを埋めるために書かれています。

第三に、そして最も重要なこととして、「そもそもこういうことが可能だ」ということ自体が知られていなかったのです。 企業の経営企画部門や政府のシンクタンクにとって、「顧客行動データから可微分多様体を構築し、その上で政策ベクトル場のLie微分を計算する」というアイデアは、これまで選択肢として浮かんだことがなかったはずです。本書は、その可能性を示します。

---

4. 今、状況が変わりつつある

▶ 数学の基礎から理解したい方へ

以下の技術的突破口により、社会データからの多様体構成が数学的に正当化可能になった。各突破口の数学的内容を正確に記述する。

(1) VAEデコーダの正則性理論。

変分オートエンコーダ（VAE, Kingma & Welling, 2014）のデコーダ $f_\theta : \mathbb{R}^d \to \mathbb{R}^n$ に対して、以下の条件を課す。

(a) f_θ が C^k 級（$k \geq 3$）であること。ニューラルネットワークの各層が線形変換と活性化関数の合成であるから、ネットワーク全体の滑らかさは活性化関数の滑らかさで決定される。ReLU $= \max(0, x)$ は $x = 0$ で微分不連続（$C^0 \setminus C^1$）であるため不可。tanh、GELU $= x \cdot \Phi(x)$（Φ：標準正規CDF）、Softplus $= \log(1 + e^x)$ はいずれも C^∞ であり、これらを全層で使用すれば $f_\theta \in C^\infty$ が保証される。

(b) ヤコビ行列 $J_{f_\theta}(z) = \left(\partial f_\theta^a / \partial z^i\right) \in \mathbb{R}^{n \times d}$ が、関心領域 $U \subset \mathbb{R}^d$ のすべての点で $\operatorname{rank} J_{f_\theta}(z) = d$（フルランク）であること。

この2条件の下で、逆関数定理（Lee [1], Theorem 4.14）により $f_\theta|_U$ は C^k 級はめ込みとなる。さらに U がコンパクトかつ $f_\theta|_U$ が単射であれば、コンパクト

空間からハウスドルフ空間への連続単射は同相であるから（Lee [1], Theorem 4.25）、$f_\theta|_U$ は C^k 級埋め込みとなり、$f_\theta(U)$ は $\mathbb{R}^n$ 内の d 次元 C^k 級埋め込み部分多様体となる。

ここで「フルランク」の意味を詳しく説明する。 ヤコビ行列 $J \in \mathbb{R}^{n\times d}$（$n > d$）がフルランク d であるとは、J の d 個の列ベクトルが線形独立であること、すなわち $\ker J = \{0\}$ であることを意味する。同値な条件として、J の特異値分解 $J = U\Sigma V^\top$ において、d 個の特異値 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_d$ がすべて正（$\sigma_d > 0$）であることがある。

フルランク条件が意味する幾何学的直感は、「デコーダが潜在空間の d 個の独立な方向を、データ空間でも d 個の独立な方向に写している」ということである。もし $\sigma_d = 0$（ランクが d 未満）であれば、潜在空間のある方向がデコーダによって「つぶされ」、その方向に動いてもデータ空間では何も変わらない。この場合、引き戻し計量 $g_{ij} = \left(J^\top J\right)_{ij}$ は退化（正定値でなくなる）し、逆行列 g^{ij} が存在しなくなり、クリストッフェル記号の計算が不可能になる。

(2) Vector Diffusion Maps (Singer & Wu, 2012)。

離散データ点 $\{z_i\}_{i=1}^N$ からグラフ接続ラプラシアンを構成する。各データ点の k 近傍を見つけ、ガウスカーネル $w_{ij} = \exp\left(-\| z_i - z_j \|^2/\varepsilon\right)$ で重み付けし、各辺 (i,j) に局所接空間間の最適回転行列 $O_{ij} \in O(d)$ を割り当てる。グラフ接続ラプラシアン S_ε は

$$(S_\varepsilon v)_i = \frac{1}{d_i}\sum_j w_{ij}\left(v_i - O_{ij}v_j\right), \quad d_i = \sum_j w_{ij}$$

で定義される。

連続極限定理（Singer & Wu, 2012）。 仮定A1の下で、$N \to \infty$, $\varepsilon \to 0$ のとき、レート条件 $N\varepsilon^{d/2+1} \to \infty$ を満たせば、S_ε は連続多様体 M_0 の接束上の接続ラプラシアンにスペクトルの意味で収束する。

この定理は、離散データから連続的な幾何構造を抽出することの数学的正当性を保証する。ただしこれは漸近結果であり、有限 N での近似誤差の定量的評価は一般には困難である。

(3) Neural ODE (Chen et al., 2018)。

微分方程式の右辺をニューラルネットワーク f_θ でパラメトリゼーション：

$$\frac{dz}{dt} = f_\theta(z, t), \quad z(0) = z_0$$

ODE solver（ルンゲ・クッタ法等）で $z(0) \to z(T)$ を計算し、損失関数でパラメータ θ を学習する。逆伝播は**随伴法**（adjoint method）を用い、メモリ効率よく勾配を計算する。

随伴法の数学的内容を述べる。随伴状態 $a(t) = \partial L / \partial z(t)$（$L$ は損失関数）は、随伴方程式

$$\frac{da}{dt} = -a(t)^\top \frac{\partial f_\theta}{\partial z}(z(t), t)$$

を**時間逆向き**に解くことで得られる。これは、ポントリャーギンの最大原理（第13章で使用）における随伴方程式 $\dot{p}_k = -\partial H / \partial z^k$ の離散アナログである。

Neural ODEの多様体上への拡張は、単にユークリッド空間 $\mathbb{R}^d$ 上のODEをリーマン多様体上に書き直すことに相当する。具体的には、VAEの潜在空間 $U \subset \mathbb{R}^d$ が単一のチャートで覆われる場合（VAEの潜在空間では通常そうである）、座標表示 $\dot{z}^k = f_\theta^k(z, t)$ がそのまま有効であるが、計量構造の保存（幾何学的積分器の使用など）に注意が必要である。

（4）PyTorch/JAXの自動微分によるテンソル計算。

PyTorchの autograd モジュールおよびJAXの grad 関数は、計算グラフの動的構築と連鎖律の自動適用により、任意の計算可能な関数の勾配を正確に計算する。これにより、リーマン計量 $g_{ij}(z) = \left(J_{f_\theta}^\top J_{f_\theta}\right)_{ij}$ からクリストッフェル記号 Γ_{ij}^k、さらにリーマン曲率テンソル $R^l{}_{ijk}$ までの**フルチェーン計算**が、手計算なしにプログラム的に実行可能になった。

特に create_graph=True オプションは、1階微分の結果をさらに微分する（高階微分を計算する）ことを可能にし、曲率テンソルの計算に不可欠である。

▶ AIモデルの基礎から理解したい方へ

ここ数年で、4つの技術的ブレイクスルーが揃いました。それぞれを、AIの知識だけで理解できるように説明します。

（1）VAE（変分オートエンコーダ）—データを「滑らかな低次元空間」に圧縮する。

VAEは2014年にKingmaとWellingによって提案されたニューラルネットワークモデルです。通常のオートエンコーダ（データを圧縮して復元するモデル）との最大の違いは、**エンコーダが確率分布を出力する**ことです。

通常のオートエンコーダでは、入力データ x に対してエンコーダは1つの潜在表現 z を出力します。VAEでは、エンコーダは正規分布の**平均** μ と**分散** σ^2 を出力し、潜在表現は $z = \mu + \sigma \cdot \varepsilon$（$\varepsilon$ は標準正規分布からのランダムサンプル）として得られます。

この「確率的な」設計の利点は、潜在空間が「穴」のない滑らかな空間になりやすいことです。通常のオートエンコーダでは、訓練データに対応する点の周辺にしか意味のある潜在表現が存在しませんが、VAEでは確率分布の「広がり」によって、訓練データの間も滑らかに埋まります。

本書でVAEを使う理由は、この「滑らかさ」が多様体構造の前提条件だからです。さらに、デコーダに tanh 等の滑らかな活性化関数を使い、ヤコビ行列がフルランクであることを検証することで、VAEの出力が数学的に厳密な多様体になることを保証します。

(2) Vector Diffusion Maps（VDM）—離散データから「幾何構造」を抽出する。

VDMは2012年にSingerとWuによって提案された手法です。VAEが「滑らかな潜在空間」を作るのに対し、VDMは「離散データ点の集合から、多様体の幾何構造（接空間の向き、平行移動の情報）を推定する」手法です。

内部動作は以下の通りです。(1) データ点間の距離を計算し、ガウスカーネルで重みを付ける。(2) 各点の近傍データ点から、局所的な接空間（その点の近くでの「方向」の空間）を推定する。(3) 隣接する点の接空間の間の「向きのずれ」を回転行列で表す。(4) これらの情報をまとめて「接続ラプラシアン」という行列を構成する。

接続ラプラシアンの固有ベクトルは、多様体の大域的な幾何構造を表す「座標」として使えます。SingerとWuの定理は、データ点の数を無限に増やすと、この離散的な計算結果が連続多様体上の正確な幾何構造に収束することを保証しています。

(3) Neural ODE—微分方程式の「法則」をデータから学習する。

Neural ODEは2018年にChenらによって提案されたモデルです。通常のニューラルネットワーク（ResNetなど）が「離散的な層の積み重ね」であるのに対し、Neural ODEは「連続的な変換」として定式化されます。

ResNetの残差ブロック $h_{t+1} = h_t + f_\theta(h_t, t)$ を、ステップ幅を0に近づける極限として見ると、連続版の微分方程式 $dz/dt = f_\theta(z, t)$ になります。Neural ODEはこの微分方程式をODEソルバー（数値積分法）で解くことで、「入力 $z(0)$ から出力 $z(T)$ への連続的な変換」を計算します。

本書でNeural ODEを使う理由は、(1) 経営環境や国際情勢の「変化の法則」をデータから学習できること、(2) 制御入力（政策）を微分方程式に追加することで、制御シミュレーションが可能になること、(3) 随伴法による効率的な勾配計算が、ポントリャーギンの最大原理（最適制御理論）と数学的に対応すること、の3点です。

(4) 自動微分—テンソル計算を「手計算なし」で実行する。

PyTorchの autograd やJAXの grad は、コンピュータが連鎖律（chain rule：合成関数の微分法則）を自動的に適用して、任意の関数の微分を正確に計算する技術です。

高校で学んだ合成関数の微分 $\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$ を思い出してください。自動微分は、この連鎖律を何百段階にもわたって自動的に適用します。ニューラルネットワークは本質的に合成関数の連鎖ですから、自動微分によって出力のパラメータに対する微分が自動的に計算されます。

本書では、自動微分を以下の「フルチェーン計算」に使います。

1. VAEデコーダ f_θ のヤコビ行列 J_{f_θ} を自動微分で計算 → 引き戻し計量 $g_{ij} = \left(J^\top J\right)_{ij}$
2. 計量 g_{ij} の偏微分 $\partial g_{jl} / \partial z^i$ を自動微分で計算 → クリストッフェル記号 Γ^k_{ij}
3. クリストッフェル記号の偏微分 $\partial \Gamma^l_{ik} / \partial z^j$ を自動微分で計算 → リーマン曲率テンソル $R^l{}_{ijk}$
4. ベクトル場の偏微分を自動微分で計算 → Lie括弧 $[V, W]$ とLie微分 $\mathcal{L}_V g$

ステップ3では、すでに1階微分の結果（クリストッフェル記号）をさらに微分する必要があります。PyTorchでこれを行うには create_graph=True オプションが必要です。これが、ReLU活性化関数では不可能でtanh等の滑らかな活性化関数が必要な理由です—ReLUの「折れ曲がり」の点で2階微分が定義できないため、ステップ3が破綻します。

▶ 経営者・政策立案者の方へ

ここ数年で、3つの技術的ブレイクスルーが揃い、ようやく「社会データに物理学者と同じ数学を適用する」ことが可能になりました。

ブレイクスルー1：データを「滑らかな地形図」に変換する技術（VAE）。 企業のビジネスデータを入力すると、「経営環境の地形図」が出力されます。この地形図は、通常のUMAPやt-SNEの散布図とは根本的に異なります。散布図が「点の配置」にすぎないのに対し、この地形図は**距離・傾斜・曲率がすべて厳密に定義された、計算可能な地図**です。カーナビの2D地図と、Google Earthの3D地形データの違いを想像してください。

ブレイクスルー2：地形図の上で「ナビゲーション」する技術（Neural ODE＋制御理論）。 「この政策を打つと、経営環境の地形図の上をどの方向に移動するか」「目標地点への最短ルートはどこか」「そのルートの途中に危険な地帯（不安定領域）はないか」を計算できます。

ブレイクスルー3：すべてのテンソル計算を自動的に実行する技術（自動微分）。 計量、クリストッフェル記号、曲率テンソル、Lie微分—これらの計算は、30年前なら数学者が何週間もかけて手計算する必要がありました。現在は、PyTorchの自動微分により、コンピュータが秒単位で正確に計算します。数学の専門家がいなくても、コンピュータが計算を実行してくれます（ただし、結果の解釈にはドメインの専門知識が不可欠です）。

これらの道具を組み合わせることで、社会データから可微分多様体を構築し、その上で物理学者と同じレベルの微分計算・テンソル計算・制御シミュレーションを行うことが、初めて数学的に正当な形で可能になりました。

---

5. 本書を読むと何ができるようになるのか

▶ 数学の基礎から理解したい方へ

本書により、以下の数学的操作が社会データに対して実行可能になる。

(A) スカラー曲率 $\mathrm{Scal}(z)$ によるデータの非線形構造の定量化。

リーマン多様体 (M, g) のスカラー曲率 $\mathrm{Scal}(z) = g^{ij}\mathrm{Ric}_{ij}$（$\mathrm{Ric}_{ij} = R^{l}{}_{ilj}$）は、各点における「総合的な曲がり具合」を1つのスカラー値で表す。$\mathrm{Scal}(z) > 0$ の点では測地球の体積がユークリッド空間より小さく（測地線が収束）、$\mathrm{Scal}(z) < 0$ の点では体積が大きい（測地線が発散）。データの文脈では、$\mathrm{Scal}(z) > 0$ は「近

傍の状態が集約する安定構造」、$\mathrm{Scal}(z) < 0$ は「近傍の状態が散逸する不安定構造」に対応する。

これは主成分分析（PCA）では原理的に得られない情報である。PCAは線形部分空間への射影であり、曲率—本質的に非線形な量—はPCAの枠組みでは定義すらできない。

(B) Lie微分 $\mathcal{L}_V g$ による政策の構造変形の定量評価。

政策介入や経営施策をベクトル場 $V \in \mathfrak{X}(M)$ としてモデル化する。計量テンソル g のLie微分

$$(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$$

は、ベクトル場 V のフロー ϕ_t に沿って計量がどう変化するかを記述する。具体的には、$\phi_t^* g = g + t(\mathcal{L}_V g) + O\left(t^2\right)$ である。

Killing方程式 $\mathcal{L}_V g = 0$ は、V が等長変換（isometry）を生成すること—すなわち V のフローが計量を保存すること—を意味する。$\mathcal{L}_V g \neq 0$ の場合、$\left(\mathcal{L}_V g\right)_{ij}$ の各成分が「計量のどの方向がどれだけ変形するか」を定量的に示す。

従来の政策評価手法（DID、回帰不連続デザインなど）は「結果の差分」を測定するものであり、「データ空間の幾何構造の変形」を測定するものではない。Lie微分はこれと質的に異なる情報を提供する。

(C) 測地線方程式とポントリャーギンの最大原理による最適制御経路の算出。

測地線方程式

$$\ddot{\gamma}^k + \Gamma_{ij}^k \dot{\gamma}^i \dot{\gamma}^j = 0$$

はエネルギー汎関数 $E[\gamma] = \frac{1}{2}\int g_{ij}\dot{\gamma}^i\dot{\gamma}^j\, dt$ の臨界点（固定端点変分のオイラー・ラグランジュ方程式）であり、多様体上の「最も自然な」（エネルギー最小の）経路を与える。

制御入力 $u(t) \in U_{\mathrm{ad}} \subset \mathbb{R}^m$ が存在する場合、アフィン制御系

$$\dot{z}^k = F^k(z) + \sum_{\alpha=1}^{m} u_\alpha(t) B_\alpha^k(z)$$

に対するポントリャーギンの最大原理により、ハミルトニアン

$$H(z,p,u) = p_k\Big(F^k(z) + u_\alpha B^k_\alpha(z)\Big) - \ell(z,u)$$

と正準方程式 $\dot{z}^k = \partial H/\partial p_k$, $\dot{p}_k = -\partial H/\partial z^k$ から最適制御 $u^*(t)$ と最適経路 $z^*(t)$ が求まる。

(D) Lie括弧 $[V_A, V_B]$ による複数政策の干渉判定。

2つのベクトル場（2つの政策）V_A, V_B のLie括弧 $[V_A, V_B]$ は、2つのフローの非可換性を測定する。$[V_A, V_B] = 0$ ならば2つのフローは可換（実行順序に依存しない）であり、$[V_A, V_B] \neq 0$ ならば非可換（実行順序によって結果が異なる）。

$[V_A, V_B]$ の大きさ $\| [V_A, V_B] \|$ は「2つの政策の干渉の強さ」を、$[V_A, V_B]$ の方向は「干渉によって生じるずれの方向」を定量的に示す。

▶ AIモデルの基礎から理解したい方へ

本書で得られる4つの能力を、AIの実装の観点から説明します。

(A) 曲率マップの生成。 VAEの潜在空間上の格子点でスカラー曲率を計算し、ヒートマップとして可視化。正の曲率（赤）＝安定領域、負の曲率（青）＝不安定領域。これはUMAPの散布図にはない「構造の質」の情報です。PyTorchのautogradで計量→クリストッフェル記号→曲率テンソル→スカラー曲率のフルチェーン計算を実行。

(B) 政策影響のシミュレーション。 政策ベクトル場 V をニューラルネットで学習（時系列データの差分から推定）。Lie微分 $\mathcal{L}_V g$ を計算し、政策が計量構造をどう変えるかを評価。$\| \mathcal{L}_V g \| \approx 0$ なら構造保存型、大きければ構造変形型。

(C) 最適経路の計算。 測地線をNeural ODEで数値解として計算。制御付き最適経路はポントリャーギンの最大原理に基づき、随伴法（Neural ODEの逆伝播と同じ数学）で計算。

(D) 政策干渉の判定。 2つのベクトル場のヤコビ行列をautogradで計算し、Lie括弧 $[V_A, V_B] = JW \cdot V - JV \cdot W$ を評価。ゼロに近ければ干渉なし、大きければ干渉あり。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：製造業X社—経営環境多様体

X社は売上高1,000億円規模の中堅製造業です。主力製品の国内市場が成熟し、新興国市場への進出を検討しています。経営企画部門は以下の問いに答える必要があります。

問い1：新興国市場の経営環境は、現在の国内市場と「どれだけ違うのか」？ これは「距離の問題」です。リーマン計量（第2章）を使って、「国内市場の経営状態」と「新興国市場の経営状態」の間のリーマン距離を計算します。ユークリッド距離（直線距離）とリーマン距離（地形を考慮した距離）は大きく異なる可能性があり、リーマン距離の方が「実質的な移行の困難さ」を正確に反映します。

問い2：新興国市場への移行は「安定した経路」を通れるか？ これは「曲率の問題」です。移行経路上のスカラー曲率（第3章）を計算し、「不安定領域」（負の曲率の領域）を通過するかどうかを確認します。不安定領域を通過するルートは、経営環境の急変に遭遇するリスクが高い。

問い3：複数の施策のうち、どれが「最もリスクの低い経路」を提供するか？ これは「最適経路の問題」です。測地線（第13章）を計算して「最も自然な遷移経路」を求め、ポントリャーギンの最大原理で「利用可能な施策の制約下での最適経路」を計算します。

問い4：複数の施策は「互いに干渉する」か、「独立に機能する」か？ これは「Lie括弧の問題」です。2つの施策のベクトル場のLie括弧（第4章）を計算し、干渉の有無と方向・大きさを定量化します。

X社が集めるべきデータセット： - 自社データ：50店舗×月次（売上、客単価、来店数、在庫回転率、従業員数、顧客満足度など15指標）×10年分 - マクロ経済データ：GDP成長率、インフレ率、失業率、消費者信頼感指数、為替レート、金利、原材料価格指数 - 業界データ：市場規模、競合シェア、原材料価格、サプライチェーン指標、特許出願数 - 新興国データ：対象国のGDP、人口動態、中間層比率、政治安定性指数、規制環境指数、インフラ整備状況

合計約30指標、約6,000データ点（月次×10年×50拠点）。

ケーススタディB：国家安全保障会議—国際情勢多様体

ある国の国家安全保障会議は、急速に変化する国際環境の中で、同盟政策と経済安全保障政策の最適な組み合わせを決定する必要があります。

問い1：現在の国際情勢は「安定」か「不安定」か？ 曲率分析（第3章）で回答。

問い2：同盟強化政策は「国際秩序の構造を保存する」か「構造を変える」か？ Lie微分（第4章）で回答。

問い3：経済制裁と同盟強化を同時に実行した場合、「干渉」するか？ Lie括弧（第4章）で回答。

問い4：目標とする国際秩序への「最小リスク経路」は？ 最適制御（第13章）で回答。

収集するインテリジェンスデータセット： – 軍事指標：各国の軍事費/GDP比、核戦力（核弾頭数、運搬手段数）、通常戦力（現役兵力、主要装備数）、サイバーセキュリティ能力指数、宇宙能力指数 – 外交指標：同盟関係ネットワーク（二国間同盟の有無と強度）、国連総会投票パターン（投票の一致度行列）、二国間条約数、外交官交換数、首脳会談頻度 – 経済安全保障指標：貿易依存度行列（二国間貿易額/GDP比）、エネルギー輸入依存度（化石燃料輸入先の集中度）、レアアース・半導体など戦略物資の調達先集中度、金融制裁に対する脆弱性指標（ドル建て取引依存度、SWIFT依存度） – 社会・政治指標：民主主義指数（V-Dem, Freedom House）、報道の自由度（国境なき記者団）、政権安定性指数、国内世論の対外政策支持率

193カ国×40指標×30年分 = 約230,000データ点（年次データ）。月次の貿易・金融データを追加すれば70万点以上に。

6. この手法が正当に機能するための条件

▶ 数学の基礎から理解したい方へ

本手順の数学的正当性は4つの仮定に依存する。

仮定A1（多様体仮説）． $X = \{x_i\} \subset \mathbb{R}^n$ がある d 次元コンパクト連結 C^∞ 部分多様体 M_0 の近傍からノイズ付き標本。

仮定A2（デコーダ正則性）． (a) $f_\theta \in C^k$ （$k \geq 3$）。(b) $\mathrm{rank}\, J_{f_\theta}(z) = d$ （$\forall z \in U$）。

仮定A3（十分なデータ量）． $N\varepsilon^{d/2+1} \to \infty$。

仮定A4（数値安定性）． $\kappa\big(g(z)\big)$ 有界。

仮定が破れた場合： A1→多様体不存在。A2(a)→曲率テンソル定義不能。A2(b)→計量退化。A3→推定不正確。A4→数値不安定。

【高校数学からの橋渡し】 仮定A1は「データが低次元の曲面の近くに集中している」こと。A2は「VAEのデコーダが十分に良い関数である」こと。A3は「データの数が十分に多い」こと。A4は「コンピュータの計算が壊れない」こと。

▶ AIモデルの基礎から理解したい方へ

4つの条件を、AIモデルの言葉で説明します。

条件1（多様体仮説 → データの構造）： 高次元データが実際には低次元の「面」の上に乗っていること。VAEがうまく学習できる前提条件。PCAのスクリープロットで確認。

条件2（デコーダ正則性 → モデルの品質）： VAEのデコーダが「良い関数」であること。(a) 活性化関数の選択：ReLUは $x = 0$ で折れ曲がっており微分不可能（C^0）。tanh は何回でも微分できる（C^∞）。本書ではReLUは使えません。(b) ヤコビ行列のフルランク：デコーダが「つぶれていない」こと。潜在空間の異なる方向が、データ空間でも異なる方向に写されること。torch.linalg.svdvals で検証。

条件3（データ量 → 学習の質）： VDMの連続極限定理が機能するための十分なデータ量。経験則として $N > 100 \times 2^d$。

条件4（数値安定性 → 計算の信頼性）： 計量テンソルの条件数（最大固有値/最小固有値）が大きすぎないこと。$\kappa > 10^4$ の領域では計算が不安定。

最重要警告： PyTorchは条件が満たされていなくてもエラーを出しません。曲率もLie微分も計算されます。しかし、その数値に意味があるかは条件次第です。

▶ ビジネス・政策の意思決定に活用したい方へ

本書の手法は「どんなデータからでも多様体を作れる」ものではありません。

コンピュータは言われた通りに計算を実行します。しかし、不適切なデータから構築された「経営環境多様体」に基づいて数十億円の投資判断を行えば、砂上の楼閣です。

データの品質確認は、投資判断そのものと同じくらい重要です。

---

7. データ品質の5つのチェック

チェック1：低次元構造（仮定A1）。 PCAスクリープロットで分散80%以上が少数主成分で説明されるか。

ケーススタディA： X社30指標。6主成分で87%。→ 合格。

ケーススタディB： 193カ国×40指標。8主成分で82%。→ 合格。

チェック2：データ密度（仮定A3）。 $N > 100 \times 2^d$ が目安。

ケーススタディA： $d = 6$, 目安6,400点。月次10年6,000点 → ぎりぎり。週次化を検討。

ケーススタディB： $d = 8$, 目安25,600点。年次5,790点では不足 → 月次データ追加で70,000点に。

チェック3：デコーダ忠実度（仮定A2）。 再構成誤差5〜10%以下。ヤコビ行列最小特異値 $> 10^{-4}$。

チェック4：結果の安定性（仮定A4）。 ランダムシード変更、ε 変更、ブートストラップで安定確認。

チェック5：他手法との整合性。 曲率パターンが過去の既知イベントと対応するか。

---

8. 実務上の検証方法

▶ 数学の基礎から理解したい方へ

A2(b)は $\sigma_{\min}\left(J_{f_\theta}(z)\right) > \delta$ のSVD検証。A3はスペクトルギャップ $\lambda_2 - \lambda_1$ の ε 依存性。A4は $\kappa\left(g(z)\right)$ モニタリング。A1は固有次元推定（Facco et al. [6], Levina-Bickel [7]）による間接検証のみ。

▶ AIモデルの基礎から理解したい方へ

仮定	検証方法	Pythonツール
A1	固有次元推定＋PCA	scikit-dimension, sklearn
A2(a)	tanh/GELU/Softplus使用	活性化関数の選択
A2(b)	ヤコビ行列SVD	torch.linalg.svdvals
A3	経験則＋ブートストラップ	sklearn.utils.resample
A4	条件数モニタリング	torch.linalg.cond

▶ ビジネス・政策の意思決定に活用したい方へ

チームに確認させてください。「再構成誤差は？」「ヤコビ行列の最小特異値は？」「シードを変えて結果は一致するか？」明確な回答がなければ、分析結果は信頼できません。

---

9. 実務的な推奨

第一に、 仮定を事前・事後に検証。**第二に、** 結果は「探索的な洞察」。**第三に、** 従来手法と併用。**第四に、** ロバスト性を常にチェック。**第五に、** 本書の幾何構造は「非線形相関構造」であり因果関係ではない。

本書の構成

第I部（第1〜4章）：数学の基礎。 位相空間→可微分多様体→接空間→リーマン計量→接続→曲率→Lie微分。

第II部（第5〜8章）：幾何学的AIの道具。 VAE→VDM→計量学習→Neural ODE→自動微分テンソル計算→全体像。

第III部（第9〜14章）：実践。 データ収集→多様体構成→幾何構造抽出→曲率分析→Lie微分→制御シミュレーション→可視化・意思決定。

各章に3層構成（数学/AI/ビジネス）、2つのケーススタディ、品質チェックポイント、コスト対リターンの明記があります。

それでは、多様体の世界への旅を始めましょう。

第1章：位相空間から可微分多様体へ

この章で得られるもの—コストに見合うリターン

第1章は、本書全体の「数学的基礎工事」です。ビルの基礎工事と同様に、目に見える華やかな部分ではありませんが、この基礎が崩れれば、その上に建つすべて—曲率分析、Lie微分、最適制御シミュレーション—が崩壊します。この章では、「多様体とは何か」を正確に定義し、VAEのデコーダの出力が多様体になるための条件を証明します。

▶ 数学の基礎から理解したい方へ：なぜこの基礎に立ち返る価値があるか

本章で確立する命題1.1（VAEデコーダの像の多様体構造）は、本書全体の数学的正当性の出発点である。この命題が保証するのは以下である。

(i) 引き戻し計量 $g_{ij}(z) = \left(J_{f_\theta}^{\top} J_{f_\theta}\right)_{ij}$ が正定値リーマン計量となること（第2章の基盤）。

(ii) クリストッフェル記号 Γ_{ij}^{k} が well-defined であること（第3章の基盤）。
(iii) リーマン曲率テンソル $R^{l}{}_{ijk}$ が well-defined であること（第3章の基盤）。
(iv) Lie微分 $\mathcal{L}_V g$ の計算が正当であること（第4章の基盤）。
(v) 測地線方程式とポントリャーギンの最大原理が適用可能であること（第13章の基盤）。

(i)〜(v) のすべてが、デコーダ f_θ が C^k 級はめ込みであること—すなわち、$f_\theta \in C^k$ $(k \geq 3)$ かつヤコビ行列 $J_{f_\theta}(z)$ がフルランクであること—に依存している。そして、はめ込みから埋め込みへの昇格には、定義域のコンパクト性と値域のハウスドルフ性が必要である。これらの位相的条件を正確に理解することは、「なぜこの条件が必要なのか」「条件が破れると何が壊れるのか」を判断するために不可欠である。

コスト： 位相空間の定義から始める、やや抽象的な準備。特に、開集合、コンパクト性、ハウスドルフ性といった概念は、日常的にテンソル計算を行う微分幾何学者にとっては「当然の前提」として意識されることが少ないかもしれない。しかし、これらの前提が成り立たない場合（たとえば、VAEの潜在空間の関心領域がコンパクトでない場合）に何が起こるかを理解するには、本章の内容が必要である。

リターン： 後続13章分の計算の数学的正当性。上記（i)〜(v) のすべてが本章の命題1.1に依存しており、命題1.1の証明を省略すると、後続の全計算は「結果は出るが数学的に正当化されない」という状況に陥る。

▶ AIモデルの基礎から理解したい方へ：なぜ位相空間の定義から始めるのか

「VAEで圧縮すれば多様体になるのでは？」—この直感は半分正しいですが、半分間違いです。

VAEのデコーダは、潜在空間（低次元の座標空間）からデータ空間（高次元の空間）への関数です。この関数の「像」（出力の全体）が、数学的に厳密な意味での「多様体」になるかどうかは、デコーダの性質に依存します。

たとえば、以下のような問題が起こりえます。

問題1：デコーダが「つぶれている」場合。 デコーダのヤコビ行列のランクが落ちている（特異値が0に近い）場合、潜在空間のある方向に動いても、データ空間では何も変化しません。これは「潜在空間の1次元が無駄になっている」状態であり、デコーダの像は d 次元多様体ではなく、それより低い次元の構造になってい

ます。この場合、引き戻し計量 $g = J^{\top}J$ が正定値でなくなり（ゼロ固有値を持つ）、逆行列が存在せず、クリストッフェル記号が計算できません。

問題2：デコーダが「折れ曲がっている」場合。 ReLU活性化関数を使ったデコーダは、入力空間を多面体的な領域に分割し、各領域内ではアフィン（直線的）な写像を行います。領域の境界（ReLUが0になる超平面）では微分が不連続になり、曲率テンソルの計算に必要な2階微分が定義できません。

問題3：デコーダが「自己交差している」場合。 潜在空間の異なる2点 $z_1 \neq z_2$ がデコーダによって同じデータ点 $f_\theta(z_1) = f_\theta(z_2)$ に写される場合、デコーダの像は「自分自身と交差する」構造になります。8の字のような形です。この場合、交差点でデコーダの像の位相構造が壊れ、多様体にはなりません。

本章では、これらの問題がすべて起こらないための条件を数学的に厳密に定義し、その条件のもとでデコーダの像が多様体になることを証明します。そのために、「多様体とは何か」を正確に定義する必要があり、その前提として位相空間の概念が不可欠なのです。

コスト： 位相空間の抽象的な概念を理解する時間。特に「開集合」「連続写像」「同相写像」「コンパクト性」「ハウスドルフ性」は、通常のAI/機械学習の文脈では登場しない概念です。

リターン： 「VAEの設計で何を間違えると、以降の全計算が壊れるか」が正確にわかるようになる。具体的には： - ReLU → tanh への切り替えが「なぜ」必要なのか（C^k 級条件） - ヤコビ行列のSVD検証が「なぜ」重要なのか（フルランク条件） - 潜在空間の「関心領域」をコンパクトに取ることが「なぜ」重要なのか（はめ込み→埋め込みの昇格条件）

これらはすべて、トラブルシューティングの際に不可欠な知識です。VAEの出力に基づく幾何学的分析が「おかしな結果」を出したとき、原因を特定し対処するためには、本章の概念的理解が必要です。

▶ ビジネス・政策の意思決定に活用したい方へ：なぜ基礎工事が必要か

本章はビルの「基礎工事」に相当します。経営者や政策立案者が本章の数学的内容を直接理解する必要はありません。しかし、データサイエンスチームがこの基礎を理解しているかどうかが、分析結果の信頼性を決定的に左右します。

たとえ話で説明します。 あなたが新しいオフィスビルを建設するとき、基礎工事の詳細（杭の深さ、鉄筋の配置、コンクリートの配合）を自分で理解する必要はありません。しかし、施工業者が「杭を打たなくても大丈夫ですよ」と言ったと

き、それが問題であることは知っておくべきです。同様に、データサイエンスチームが「ReLUでいいですよ」「ヤコビ行列の検証は省略しました」と言ったとき、それが本書のアプローチでは**重大な問題**であることを知っておくべきです。

経営者として確認すべきこと： チームに「VAEのデコーダが多様体を生成するための3つの条件（滑らかさ、フルランク、単射）を説明してください」と質問してください。明確に回答できないチームには、この基礎の学習が必要です。明確に回答でき、かつ検証結果を示せるチームであれば、分析結果を信頼する根拠があります。

1.1 位相空間と連続写像

▶ 数学の基礎から理解したい方へ

位相空間の定義

定義1.1（位相空間）． 集合 X とその部分集合の族 $\mathcal{O} \subset 2^X$ の組 $(X, \mathcal{O})$ が**位相空間**（topological space）であるとは、$\mathcal{O}$ が以下の3条件を満たすことをいう：

（T1）$\emptyset \in \mathcal{O}$ かつ $X \in \mathcal{O}$。

（T2）$\{U_\lambda\}_{\lambda \in \Lambda} \subset \mathcal{O}$（$\Lambda$ は任意の添字集合）ならば $\bigcup_{\lambda \in \Lambda} U_\lambda \in \mathcal{O}$。

（T3）$U_1, U_2, \ldots, U_n \in \mathcal{O}$（有限個）ならば $\bigcap_{k=1}^{n} U_k \in \mathcal{O}$。

$\mathcal{O}$ を X 上の**位相**（topology）と呼び、$\mathcal{O}$ の元を**開集合**（open set）と呼ぶ。

注意1.1. （T2）では合併の添字集合 Λ は任意（非可算でもよい）であるが、（T3）では共通部分は**有限個**に限定される。この非対称性は本質的であり、無限個の開集合の共通部分は一般に開集合にならない。例：$\mathbb{R}$ 上で $U_n = \left(-1/n, 1/n\right)$ は各 n で開集合であるが、$\bigcap_{n=1}^{\infty} U_n = \{0\}$ は（$\mathbb{R}$ の標準位相で）開集合ではない。

例1.1（$\mathbb{R}^n$ の標準位相）． $\mathbb{R}^n$ 上の開球 $B(x, r) = \{y \in \mathbb{R}^n \mid \| x - y \| < r\}$ の任意個の合併で表される集合全体の族を $\mathcal{O}$ とすると、$\left(\mathbb{R}^n, \mathcal{O}\right)$ は位相空間となる。これが $\mathbb{R}^n$ の**標準位相**（Euclidean topology）であり、本書で $\mathbb{R}^n$ と書くときは常にこの位相を想定する。

例1.2（離散位相と密着位相）． 任意の集合 X に対し、$\mathcal{O} = 2^X$（X のすべての部分集合を開集合とする）は**離散位相**、$\mathcal{O} = \{\varnothing, X\}$ は**密着位相**と呼ばれる。離散位

相は「あらゆる部分集合が開」であり、密着位相は「ほとんど何も開でない」。どちらも数学的に有効な位相であるが、実用上は $\mathbb{R}^n$ の標準位相が圧倒的に重要である。

【高校数学からの橋渡し】 位相空間の定義は抽象的ですが、要は「どの部分集合を『開いている』と呼ぶかのルール」を定めるものです。

高校で学んだ実数直線 $\mathbb{R}$ で考えましょう。開区間 $(0,1) = \{x \in \mathbb{R} \mid 0 < x < 1\}$ は「開集合」です。なぜなら、この区間の中のどの点を取っても、その点の周りに十分小さな開区間（たとえば $(0.4,0.6)$）を取れば、それも $(0,1)$ の中に収まるからです。つまり、開集合の中のどの点にも「少し動いてもまだ集合の中にいられる余裕」があります。

一方、閉区間 $[0,1] = \{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$ は開集合ではありません。端点 0 の周りにどんなに小さな開区間を取っても、負の数を含んでしまい、$[0,1]$ からはみ出します。

この「余裕がある」という直感を、3つの公理（T1)-(T3）として抽象的に定式化したのが位相空間の定義です。$\mathbb{R}^n$ のような具体的な空間だけでなく、VAEの潜在空間のような一般的な空間でも「近さ」や「連続性」を定義できるようになります。

連続写像と同相写像

定義1.2（連続写像）． 位相空間の間の写像 $f:(X,\mathcal{O}_X) \to (Y,\mathcal{O}_Y)$ が**連続**（continuous）であるとは、Y の任意の開集合 $V \in \mathcal{O}_Y$ に対して、その逆像 $f^{-1}(V) = \{x \in X \mid f(x) \in V\}$ が X の開集合であること、すなわち $f^{-1}(V) \in \mathcal{O}_X$ が成り立つことをいう。

注意1.2. この定義は、ε-δ 論法による連続性の定義の一般化である。$\mathbb{R}^n$ の標準位相の場合、定義1.2は通常の ε-δ 連続性と同値である。位相空間の連続性の定義の利点は、距離関数（ノルム）を必要とせず、開集合の構造のみで連続性を定義できることにある。これにより、距離が自然に定義されない空間（たとえば、関数空間の一部や、商位相空間など）でも連続性を扱える。

【高校数学からの橋渡し】 高校で学んだ連続関数の定義―「x が a に近づくとき $f(x)$ が $f(a)$ に近づく」―を一般化したものです。位相空間の言葉では、「Y 側で『近い』と見なされる点の集合（開集合 V）を f で引き戻すと、X 側でも『近い』と見なされる点の集合（開集合 $f^{-1}(V)$）になる」と言い換えています。

直感的には「近い点を近い点に写す」ということです。もし f が連続でなければ、Y の中で近い2点の逆像が X の中でバラバラの場所にあるかもしれません。連続性は、この「近さの保存」を保証するものです。

定義1.3（同相写像）． 位相空間の間の写像 $f: X \to Y$ が**同相写像**（homeomorphism）であるとは、以下の3条件をすべて満たすことをいう：

(H1) f は全射（surjective）：すべての $y \in Y$ に対し $f(x) = y$ となる $x \in X$ が存在する。 (H2) f は単射（injective）：$f\left(x_1\right) = f\left(x_2\right)$ ならば $x_1 = x_2$。
(H3) f と f^{-1} がともに連続。

同相写像が存在するとき、X と Y は**同相**（homeomorphic）であるといい、$X \cong Y$ と書く。

注意1.3. (H3) の「f^{-1} も連続」という条件は、「f が連続な全単射」というだけでは十分でないことに注意。連続な全単射であっても逆写像が連続でない例は存在する（ただし、定義域がコンパクトで値域がハウスドルフの場合は、連続な全単射の逆写像は自動的に連続になる—これが定理1.1の本質である）。

【高校数学からの橋渡し】 同相写像は「ゴムのように変形して互いに移り合える」関係に対応します。

たとえば、正方形と円は同相です。正方形のゴムシートを引き延ばして丸い形にできます。この変形は連続的であり（途中で切ったり貼ったりしない）、逆方向にも連続的に戻せます。

一方、球面（ボールの表面）とトーラス（ドーナツの表面）は同相ではありません。ドーナツには「穴」がありますが、球面にはありません。球面をゴムのように連続的に変形しても、穴を作ることはできません。穴を作るには「切る」操作が必要であり、切る操作は連続ではないからです。

この「穴の数」のような、連続変形で変わらない性質を研究するのがトポロジー（位相幾何学）です。本書では、トポロジーの深い理論には立ち入りませんが、多様体を定義するための最小限の位相的概念をこのセクションで導入します。

コンパクト性

定義1.4（開被覆と有限部分被覆）． 位相空間 X の部分集合 A に対し、開集合の族 $\{U_\lambda\}_{\lambda \in \Lambda}$ が A の**開被覆**（open cover）であるとは、$A \subset \bigcup_{\lambda \in \Lambda} U_\lambda$ が成り立つことをいう。$\{U_\lambda\}$ の有限個の部分族 $\{U_{\lambda_1}, \ldots, U_{\lambda_n}\}$ が $A \subset U_{\lambda_1} \cup \cdots \cup U_{\lambda_n}$ を満たすとき、これを**有限部分被覆**（finite subcover）と呼ぶ。

定義1.5（コンパクト性）． 位相空間 X が**コンパクト**（compact）であるとは、X の任意の開被覆が有限部分被覆を持つことをいう。

定理1.0（ハイネ・ボレルの定理）． $\mathbb{R}^n$ の部分集合 K がコンパクトであるための必要十分条件は、K が有界かつ閉であることである。

【高校数学からの橋渡し】 コンパクト性の直感は「有限の大きさに収まっていて、境界を含む」です。

$\mathbb{R}^n$ の場合、ハイネ・ボレルの定理により、コンパクト ⇔ 有界かつ閉 です。

- 閉区間 $[0,1]$：有界（0と1の間に収まる）で閉（端点0と1を含む）→ コンパクト。
- 開区間 $(0,1)$：有界だが閉でない（端点を含まない）→ コンパクトでない。
- $\mathbb{R}$ 全体：閉だが有界でない → コンパクトでない。

コンパクト性が重要な理由は、「コンパクト集合上の連続関数は最大値と最小値を取る」（ワイエルシュトラスの最大値定理）のように、コンパクト集合上では良い性質が保証されることが多いためです。本書では、はめ込み→埋め込みの昇格定理（定理1.1）でコンパクト性が本質的に使われます。

なぜVAEの文脈でコンパクト性が重要か。 VAEの潜在空間は $\mathbb{R}^d$ 全体ですが、実際にデータが分布する領域は有限の範囲に収まっています（KL正則化によって、潜在分布は標準正規分布に近づけられるため、$\|z\|$ が大きい領域にはほとんどデータがない）。分析の「関心領域」U をこのデータ分布の範囲に制限すれば、U はコンパクト（有界閉）に取ることができます。これにより、定理1.1の適用条件が満たされます。

ハウスドルフ性

定義1.6（ハウスドルフ性）． 位相空間 X が**ハウスドルフ**（Hausdorff、T_2 空間ともいう）であるとは、任意の相異なる2点 $p,q \in X$ に対し、$p \in U$，$q \in V$，$U \cap V = \varnothing$ なる開集合 $U,V \in \mathcal{O}$ が存在することをいう。

例1.3. $\mathbb{R}^n$ はハウスドルフ空間である。$p \neq q$ に対し、$r = \|p-q\|/2 > 0$ として $U = B(p,r)$，$V = B(q,r)$ を取れば $U \cap V = \varnothing$。

注意1.4. 私たちが日常的に扱う空間（$\mathbb{R}^n$、球面 S^n、トーラス T^n など）はすべてハウスドルフである。ハウスドルフでない空間は数学的には存在するが、本書で扱うことはない。ハウスドルフ性が重要な理由は、「異なる2点を位相的に区別

できる」ことを保証するからである。ハウスドルフ性がなければ、2つの「異なる」点の間に開集合による分離が不可能であり、多様体の構造が破綻する。

【高校数学からの橋渡し】 ハウスドルフ性は「異なる2点を区別できる」という常識的な性質を数学的に厳密に述べたものです。

私たちが普通に考える空間では、2つの異なる点 p と q の周りに「重ならない」領域を取ることは当然可能です。たとえば、$p = 0$ と $q = 1$ の周りに、$(-0.3, 0.3)$ と $(0.7, 1.3)$ という重ならない開区間を取れます。

この「当然の性質」が成り立たない空間は、非常に「病的な」空間であり、物理学やデータサイエンスで登場することはありません。しかし、数学的な厳密性のために、多様体の定義にこの条件が含まれています。

▶ AIモデルの基礎から理解したい方へ

位相空間の概念とAIモデルの関係。 このセクションの数学は直接AIモデルに結びつかないように見えますが、実は間接的に重要です。

連続写像とニューラルネットワーク。 VAEのデコーダ f_θ は連続写像です。なぜなら、デコーダは線形変換（行列の掛け算＋バイアスの加算）と活性化関数（tanh、ReLU等）の合成であり、線形変換は連続、tanh も ReLU も連続であるから、その合成も連続です。

「デコーダが連続」ということは、「潜在空間で近い2つの点 z_1 と z_2 は、データ空間でも近い2つの点 $f_\theta(z_1)$ と $f_\theta(z_2)$ に写される」ことを意味します。これは「潜在空間の滑らかさ」の基礎です。もしデコーダが不連続であれば、潜在空間で隣り合う点がデータ空間でまったく異なる点に写され、「潜在空間の構造」がデータ空間の構造を反映しなくなります。

同相写像とVAEの品質。 理想的なVAEでは、デコーダが「同相写像に近い」写像を学習します。つまり、潜在空間の構造とデータ空間の構造が「位相的に同じ」になることが望ましい。しかし、VAEの訓練が不十分だったり、潜在次元 d の選択が不適切だったりすると、デコーダが「つぶれ」（ランク落ち）や「自己交差」を起こし、同相写像にならない場合があります。

コンパクト性とVAEの潜在空間。 VAEの潜在空間は $\mathbb{R}^d$ 全体ですが、KL正則化により、訓練データの潜在表現は標準正規分布 $\mathcal{N}(0, I_d)$ の周辺に集中します。実用上、$\| z \| < 3\sigma$（$\sigma = 1$）、すなわち $\| z \| < 3$ の領域にデータの99.7%が含まれます。この領域を「関心領域」$U = \{z \in \mathbb{R}^d \mid \| z \| \leq R\}$（$R$ は適切な定数）として設

定すれば、U は有界閉集合であり、ハイネ・ボレルの定理によりコンパクトです。

▶ ビジネス・政策の意思決定に活用したい方へ

このセクションの数学は、「多様体」を正確に定義するための準備です。 経営者・政策立案者が直接理解する必要はありませんが、1つだけ覚えてください。

「多様体」は「局所的には平らに見えるが、全体としては曲がっている、滑らかな空間」です。

地球の表面が最も身近な例です。あなたが立っている場所の半径100メートルの範囲では、地面はほぼ平らに見えます。しかし、地球全体は球面であり、平面ではありません。この「局所的には平らだが、全体としては曲がっている」性質が、多様体の本質です。

企業の経営環境も同様です。ある特定の市場セグメントの中では、売上と価格の関係はほぼ直線的（線形）に見えるかもしれません。しかし、複数の市場セグメント、複数の国、複数の経済環境を横断して見ると、関係は非線形に曲がっています。本書は、この「曲がった空間」を数学的に正確に構築し、その上で計算する方法を提供します。

```
import numpy as np
import matplotlib.pyplot as plt

# 同相写像（連続変形）の視覚化
# 円をパラメータ α で連続的に変形する
t = np.linspace(0, 2*np.pi, 100)
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, alpha in zip(axes, [0, 0.3, 0.6, 1.0]):
    x = np.cos(t) * (1 + alpha)
    y = np.sin(t) * (1 - 0.5*alpha)
    ax.plot(x, y, 'b-', linewidth=2)
    ax.set_aspect('equal'); ax.set_title(f'α = {alpha}')
plt.suptitle('Continuous deformation (homeomorphism)')


> **【図：図1-1：同相写像による連続変形】**
*図1-1：同相写像（ホメオモルフィズム）による連続変形。4つの図形はすべて位相的に同じ（同相）*
plt.show()
```

▶ AIモデルの基礎から理解したい方へ：コードの解説

上のコードは、円（$\alpha = 0$）を連続的に楕円状の形に変形する様子を描画しています。

- $\alpha = 0$：完全な円。$x = \cos t,\ y = \sin t$。
- $\alpha > 0$：x 方向に $(1+\alpha)$ 倍に引き伸ばし、y 方向に $(1-0.5\alpha)$ 倍に押しつぶす。

重要なのは、この変形が**連続的**であること—形が徐々に変わり、途中で切れたり裂けたりしません。4つの図形はすべて**同相**（位相的に同じ形）であり、「穴のない閉曲線」という位相的性質を共有しています。

VAEのデコーダも同様に、潜在空間の「形」をデータ空間の「形」に連続的に変換する写像です。この変換が「切れたり裂けたりしない」ことが、デコーダの像が多様体になるための前提条件です。

1.2 可微分多様体

▶ 数学の基礎から理解したい方へ

チャートとアトラスの定義

定義1.7（チャート）． 位相空間 M の開集合 U と同相写像 $\varphi: U \to \varphi(U) \subset \mathbb{R}^d$ の組 (U, φ) を、M 上の d **次元チャート**（chart，座標近傍）と呼ぶ。φ を**座標写像**、$\varphi(U)$ を**座標近傍の像**、$\varphi(p) = \left(z^1(p), \dots, z^d(p)\right)$ を点 p の**局所座標**と呼ぶ。

チャートの本質は、「多様体 M の一部分 U を、$\mathbb{R}^d$ の一部分 $\varphi(U)$ と『同じ形』（同相）に対応づける」ことです。この対応により、M 上の点を $\mathbb{R}^d$ の座標 $\left(z^1, \dots, z^d\right)$ で表現でき、$\mathbb{R}^d$ 上の通常の微積分が使えるようになります。

【高校数学からの橋渡し】 チャートは「地図」です。地球の表面（球面）は平面ではありませんが、「日本周辺の地図」は地球の一部分を平面（紙の上）に対応させたものです。この対応が「チャート」です。

地図には必ず「歪み」がありますが、十分に小さな範囲であれば歪みは無視できます。数学的にいえば、チャート φ は U と $\varphi(U)$ の間の同相写像であり、U の位相構造（「近さ」の構造）を完全に保存しています。ただし、計量構造（「距離」の定量的な値）は一般には保存されません。計量構造の議論は第2章で行います。

定義1.8（C^k 級アトラス）． M の開被覆 $\{U_\alpha\}_{\alpha\in A}$（すなわち $M=\bigcup_\alpha U_\alpha$）に対するチャートの集合

$$\mathcal{A}=\{(U_\alpha,\varphi_\alpha)\}_{\alpha\in A}$$

が C^k **級アトラス**（C^k atlas）であるとは、すべての $\alpha,\beta\in A$ に対して、$U_\alpha\cap U_\beta\neq\varnothing$ のとき遷移関数

$$\varphi_\beta\circ\varphi_\alpha^{-1}:\varphi_\alpha(U_\alpha\cap U_\beta)\to\varphi_\beta(U_\alpha\cap U_\beta)$$

が C^k 級写像（k 回連続微分可能）であることをいう。

遷移関数の意味を詳しく説明する。 $U_\alpha\cap U_\beta$ は、チャート $(U_\alpha,\varphi_\alpha)$ とチャート (U_β,φ_β) の「重なり」の部分である。この重なり部分にある点 p は、2つの座標系 $\varphi_\alpha(p)=\left(z_\alpha^1,\dots,z_\alpha^d\right)$ と $\varphi_\beta(p)=\left(z_\beta^1,\dots,z_\beta^d\right)$ の両方で表現できる。遷移関数 $\varphi_\beta\circ\varphi_\alpha^{-1}$ は、α の座標から β の座標への変換規則—すなわち**座標変換**—を与える。

「C^k 級」とは、この座標変換が k 回微分可能で、k 回目の微分も連続であることを意味する。座標変換が滑らかであれば、一方の座標系での微分計算の結果を他方の座標系に変換できる。

【高校数学からの橋渡し】 アトラスは「地図帳」です。日本周辺の地図1枚では地球全体をカバーできないので、ヨーロッパの地図、アメリカの地図、アフリカの地図…と複数の地図を集めて地球全体をカバーします。この「地図帳全体」がアトラスです。

隣り合う地図は重なりを持っています。たとえば、日本周辺の地図とロシア周辺の地図は、極東ロシアの部分で重なっています。この重なり部分では、日本地図での座標（緯度・経度の表し方）とロシア地図での座標が異なるため、座標変換が必要です。この座標変換が「遷移関数」です。遷移関数が滑らか（C^k 級）であれば、地図のつなぎ目で微積分の計算結果が整合的につながります。

定義1.9（C^k 級可微分多様体）． 第二可算ハウスドルフ位相空間 M が d 次元 C^k **級可微分多様体**（C^k differentiable manifold）であるとは、M が極大（maximal）C^k 級アトラスを持つことをいう。

注意1.5（第二可算性）． 第二可算（second-countable）とは、M の位相が可算個の開集合で生成されることをいう。$\mathbb{R}^n$ は第二可算である。この条件は、多様体の病的な例（非可算個の連結成分を持つ空間など）を排除するための技術的条件であり、本書で扱うVAEの潜在空間では常に満たされる。

注意1.6（極大アトラス）． C^k 級アトラスは一般に一意でないが、任意の C^k 級アトラスはただ一つの極大 C^k 級アトラス（それと C^k 級両立するすべてのチャートを含む最大のアトラス）に拡張される。したがって、多様体を定義する際には、1つの C^k 級アトラスを指定すれば十分であり、それが生成する極大アトラスにより C^k 級可微分構造が一意に定まる。

注意1.7（本書での滑らかさの仮定：$k \geq 3$）． 本書では、$k \geq 3$ を仮定する。この仮定が必要な理由は、以下の「滑らかさの連鎖」による。

- Step 6（第3章/第11章） でリーマン曲率テンソル ${R^l}_{ijk}$ を計算するには、クリストッフェル記号 Γ_{ij}^k の1階偏微分が必要。
- クリストッフェル記号はリーマン計量 g_{ij} の1階偏微分から構成される。
- したがって、${R^l}_{ijk}$ の計算には g_{ij} の2階偏微分が必要であり、$g_{ij} \in C^2$ が要求される。
- 引き戻し計量（第2章）では $g_{ij}(z) = \left(J_{f_\theta}^{\top} J_{f_\theta}\right)_{ij}$ であり、$f_\theta \in C^k$ のとき $J_{f_\theta} \in C^{k-1}$、したがって $g_{ij} \in C^{k-1}$。
- $g_{ij} \in C^2$ を得るには $k - 1 \geq 2$、すなわち $k \geq 3$ が必要。

C^∞ 活性化関数（tanh, GELU, Softplus）を全層で使用すれば、$f_\theta \in C^\infty$ が保証され、$k \geq 3$ の条件は十分に満たされる。一方、ReLU（$= \max(0, x)$）は $C^0 \setminus C^1$（$x = 0$ で微分不連続）であるため、$k \geq 1$ すら保証されない。

▶ **AIモデルの基礎から理解したい方へ**

VAEのデコーダと可微分多様体の関係。

VAEのデコーダ $f_\theta : \mathbb{R}^d \to \mathbb{R}^n$ は、d 次元の潜在空間から n 次元のデータ空間への写像です。潜在空間 $\mathbb{R}^d$ はそれ自体が d 次元の可微分多様体です（$\mathbb{R}^d$ 全体を覆う1つのチャート (U, id) で C^∞ 級アトラスを構成できる）。

デコーダの**像** $f_\theta(\mathbb{R}^d) \subset \mathbb{R}^n$ が可微分多様体になるかどうかは、f_θ の性質に依存します。次のセクション（1.3）で、その条件を正確に述べ、証明します。

なぜ活性化関数が重要か—より詳しい説明。

ニューラルネットワークは、各層で「線形変換」と「活性化関数」を交互に適用します。L 層のネットワークを数式で書くと：

$$f_\theta(z) = \sigma_L(W_L \cdot \sigma_{L-1}(W_{L-1} \cdots \sigma_1(W_1 z + b_1) \cdots + b_{L-1}) + b_L)$$

ここで W_l は重み行列、b_l はバイアスベクトル、σ_l は活性化関数です。

線形変換 $z \mapsto Wz + b$ は C^∞（何回でも微分可能）です。したがって、ネットワーク全体の滑らかさは**活性化関数の滑らかさ**で決定されます。

ReLU $= \max(0, x)$ **の場合：** $x > 0$ で ReLU$(x) = x$（微分は1）、$x < 0$ で ReLU$(x) = 0$（微分は0）。$x = 0$ では左微分と右微分が異なり（左微分 = 0、右微分 = 1）、微分が不連続です。したがって ReLU は C^0（連続だが微分不連続）。ReLU を使ったネットワーク全体は、入力空間を多面体的な領域に分割し、各領域内ではアフィン写像（$x \mapsto ax + b$）を行います。領域の境界で微分が不連続になるため、2階微分（ヘッシアン）が定義できず、曲率テンソルが計算不能です。

$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ **の場合：** すべての x で何回でも微分可能（C^∞）。$\tanh'(x) = 1 - \tanh^2(x)$。tanh を使ったネットワーク全体は C^∞ 写像となり、何回でも微分でき、曲率テンソルの計算に必要なすべての高階微分が定義されます。

GELU $= x \cdot \Phi(x)$（Φ は標準正規分布の累積分布関数）と **Softplus** $= \log(1 + e^x)$ も同様に C^∞ です。

コスト対リターン。 ReLUから tanh への切り替えのコストは、コードの1行の変更（nn.ReLU() → nn.Tanh()）です。訓練速度がわずかに低下する可能性があります（ReLUの方が勾配消失が起きにくいため、深いネットワークでは訓練が安定しやすい）。しかし、本書のVAEは通常4〜6層程度であり、勾配消失が深刻な問題になることは稀です。一方、リターンは「計量からクリストッフェル記号、曲率テンソル、Lie微分まで計算可能になる」ことであり、本書のアプローチの根幹を支えます。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社の経営環境を「地図帳」で考える

X社のデータサイエンスチームは、30個の経営指標（売上、利益率、市場シェア、為替レート、原材料価格、消費者信頼感指数など）からなるデータセットを用いて、VAEで6次元の潜在空間を学習しました。

この6次元の潜在空間が「経営環境多様体」です。多様体を1枚の地図で表現することはできません—地球の表面を1枚の平面地図で歪みなく表現できないのと同じです。しかし、「日本市場の地図」「東南アジア市場の地図」「欧州市場の地図」のように、複数の地図（チャート）を作ってつなぎ合わせれば、経営環境多様体全体をカバーできます。

重要なのは、隣り合う地図の「つなぎ目」が滑らかであることです。VAEのデコーダに tanh 活性化関数を使うことで、地図のつなぎ目（座標変換）が C^∞ 級（何回でも微分可能）になり、つなぎ目で曲率が計算できます。もしReLUを使うと、つなぎ目に「折れ曲がり」が生じ、その部分で曲率が計算できなくなります。

ケーススタディB：国際情勢の「地図帳」

国際情勢多様体も同様です。「東アジアの安全保障環境」「中東の安全保障環境」「欧州の安全保障環境」は、それぞれ異なるチャート（地図）で記述されます。

しかし、これらの地域の安全保障環境は独立ではありません。エネルギー安全保障（中東の石油供給が東アジアに影響）、同盟ネットワーク（NATOの欧州防衛が米国のアジア戦略に影響）、核戦力のバランス（全球的な核抑止の構造）を通じて密接に結びついています。

多様体の「地図帳」は、これらの地域的な視点（チャート）を、滑らかな座標変換（遷移関数）でつなぎ合わせることで、「局所的な分析を大域的な全体像に統合する」ことを可能にします。

1.3 はめ込みと埋め込み—VAEの出力が多様体になる条件

▶ 数学の基礎から理解したい方へ

このセクションでは、VAEデコーダの像が $\mathbb{R}^n$ 内の埋め込み部分多様体となるための条件を厳密に述べ、証明する。核心となるのは「はめ込み」と「埋め込み」の区別、およびコンパクト空間からの連続単射はめ込みが自動的に埋め込みになるという定理（定理1.1）である。

はめ込みの定義と意味

定義1.10（はめ込み）． C^k 級写像 $f: M \to N$（M, N は可微分多様体）が**はめ込み**（immersion）であるとは、各点 $p \in M$ において微分（接写像）

$$df_p: T_pM \to T_{f(p)}N$$

が**単射**であることをいう。

M が d 次元、N が n 次元（$n \geq d$）のとき、局所座標を用いれば df_p はヤコビ行列 $J_f(p) \in \mathbb{R}^{n \times d}$ で表される。df_p が単射であることは、$J_f(p)$ のランクが d（列フルランク）であることと同値である。

はめ込みの直感的意味。 はめ込みとは、「M を N の中に、つぶさずに写す」操作である。「つぶさない」とは、M 上の異なる方向がすべて N 上でも異なる方向に写されること—すなわち接写像が単射であること—を意味する。

例1.4（はめ込みであるが埋め込みでない例）． $\mathbb{R}$ から $\mathbb{R}^2$ への写像で、像が「8の字」の形になるものを考える。たとえば、$\gamma(t) = (\sin 2t, \sin t)$ とすると、γ は $(-\pi, \pi)$ 上では正則（ヤコビ行列フルランク）であるが、$\gamma(0) = \gamma(\pi) = (0,0)$ となり、自己交差を持つ。接写像は各点で単射（はめ込み条件は満たされる）であるが、γ 自体が単射でないため、像は多様体にならない。

埋め込みの定義と意味

定義1.11（埋め込み）． はめ込み $f: M \to N$ が**埋め込み**（embedding）であるとは、f が単射であり、かつ $f: M \to f(M)$（$f(M)$ には N からの相対位相を入れる）が同相写像であることをいう。

埋め込みの意味。 埋め込みは「M を N の中に、つぶさずかつ自己交差なしに写す」操作である。埋め込みの像 $f(M)$ は、N 内の d 次元**埋め込み部分多様体**（embedded submanifold）となる。

はめ込みと埋め込みの違い。 はめ込みは「各点の近傍で」つぶれがないことを要求するが、大域的な性質（自己交差がないこと、像への写像が同相であること）は要求しない。埋め込みはこれに加えて大域的な条件も要求する。

はめ込みから埋め込みへの昇格定理

定理1.1（Lee [1], Theorem 4.25）． M がコンパクトなハウスドルフ空間、N がハウスドルフ空間であるとき、連続な単射はめ込み $f: M \to N$ は埋め込みである。

証明の概略. f が連続単射であることは仮定。示すべきは $f: M \to f(M)$ が同相であること、すなわち $f^{-1}: f(M) \to M$ が連続であること。M の閉集合 C に対し、M がコンパクトかつ C が閉であるから C はコンパクト。連続写像でコンパクト集合の像はコンパクトであるから、$f(C)$ はコンパクト。N がハウスドルフであるからコンパクト集合は閉。したがって $f(C)$ は N で閉、ゆえに $f(M)$ の相対位相でも閉。これは f^{-1} が連続であることを意味する（閉集合の逆像が閉であることは連続性と同値）。▫

この定理がなぜ重要か。 はめ込み条件（ヤコビ行列フルランク）は「局所的な」条件であり、各点の近傍で検証できる。一方、埋め込み条件は「大域的な」条件であり、像全体の位相構造を調べる必要がある。定理1.1は、「定義域がコンパク

トでハウスドルフ」という位相的条件の下で、局所条件（はめ込み）＋単射から大域条件（埋め込み）が自動的に従うことを示す。これにより、VAEのデコーダの像が多様体であることの検証が、**局所的な条件（ヤコビ行列のフルランク）＋単射性の検証に帰着される。**

VAEデコーダの像の多様体構造

命題1.1（VAEデコーダの像の多様体構造）． VAEデコーダ $f_\theta: U \to \mathbb{R}^n$ が以下の3条件を満たすとする。

(i) $U \subset \mathbb{R}^d$ はコンパクト（有界閉集合）。

(ii) $f_\theta \in C^k$（$k \geq 3$）。これは C^∞ 活性化関数（tanh, GELU, Softplus）の使用により保証される。

(iii) $\mathrm{rank} J_{f_\theta}(z) = d$（$\forall z \in U$）。すなわち、ヤコビ行列が U のすべての点でフルランク。

(iv) $f_\theta|_U$ が単射。

このとき、$f_\theta(U)$ は $\mathbb{R}^n$ 内の d 次元 C^k 級埋め込み部分多様体である。

証明.

Step 1（はめ込みの確認）． $f_\theta \in C^k$ かつ各点 $z \in U$ で $\mathrm{rank} J_{f_\theta}(z) = d$ であるから、定義1.10により $f_\theta|_U$ は C^k 級はめ込みである。

ここで、「ヤコビ行列がフルランク」であることが「微分（接写像）が単射」であることとなぜ同値かを説明する。f_θ の微分 $df_{\theta,z}: T_z U \cong \mathbb{R}^d \to T_{f_\theta(z)}\mathbb{R}^n \cong \mathbb{R}^n$ は、局所座標ではヤコビ行列 $J_{f_\theta}(z) \in \mathbb{R}^{n \times d}$ による線形写像 $v \mapsto J_{f_\theta}(z) \cdot v$ として表される。この線形写像が単射であることは、$\ker J_{f_\theta}(z) = \{0\}$ であること、すなわち $J_{f_\theta}(z)$ のランクが d（列の最大ランク）であることと同値である。

Step 2（埋め込みへの昇格）． U は $\mathbb{R}^d$ のコンパクト部分集合（仮定（i））。$\mathbb{R}^n$ はハウスドルフ空間。$f_\theta|_U$ は連続（$C^k \subset C^0$）な単射（仮定（iv））はめ込み（Step 1）。定理1.1により、$f_\theta|_U$ は C^k 級埋め込みである。

Step 3（埋め込み部分多様体の構造）． $f_\theta|_U$ が C^k 級埋め込みであるから、その像 $f_\theta(U)$ は $\mathbb{R}^n$ 内の d 次元 C^k 級埋め込み部分多様体である（Lee [1], Proposition 5.2）。▫

命題1.1の各条件が破れた場合の帰結。

(i) が破れた場合（U が非コンパクト）：はめ込みから埋め込みへの昇格が保証されない。はめ込みであっても像が多様体にならない例（自己交差、像の位相構造の崩壊）がありうる。

(ii) が破れた場合（f_θ が C^k 級でない、たとえば ReLU 使用時）：はめ込みの定義自体は「C^1 級で接写像が単射」であるから C^1 で足りるが、本書で必要な曲率テンソルの計算には C^3 以上が必要。ReLU は C^1 すら満たさないため、はめ込み条件が点（ReLUの折れ曲がり面上の点）で定義されない。

(iii) が破れた場合（ランクが d 未満のゼロ点が存在）：その点で引き戻し計量 $g_{ij} = \left(J^\top J\right)_{ij}$ がゼロ固有値を持ち、正定値でなくなる。逆行列 g^{ij} が存在せず、クリストッフェル記号の計算が不可能。

(iv) が破れた場合（f_θ が非単射）：像が自己交差し、多様体にならない。交差点での計量や曲率の意味が失われる。

▶ AIモデルの基礎から理解したい方へ

命題1.1を「AIモデルの言葉」で理解する。

命題1.1は、VAEのデコーダの出力が「数学的に正当な多様体になるための4つの条件」を述べています。

条件（i）：関心領域がコンパクト（有界閉集合）。

実装上の意味：VAEの潜在空間 $\mathbb{R}^d$ 全体ではなく、データが分布する有限の範囲（たとえば $\|z\| \leq 3$）を「関心領域」U として設定する。PyTorchでは、潜在表現のノルムを計算して範囲外の点を除外するだけです。

条件（ii）：デコーダが C^k 級（$k \geq 3$）。

実装上の意味：すべての活性化関数を nn.Tanh(), nn.GELU(), nn.Softplus() のいずれかにする。nn.ReLU() は使わない。

```
# ✗ 使用不可（C^0、曲率計算不能）
decoder = nn.Sequential(nn.Linear(d, 64), nn.ReLU(), nn.Linear(64, n))

# ✓ 推奨（C^∞、すべての高階微分が計算可能）
decoder = nn.Sequential(nn.Linear(d, 64), nn.Tanh(), nn.Linear(64, n))
```

条件（iii）：ヤコビ行列がフルランク。

実装上の意味：訓練後に、潜在空間の代表点（少なくとも50〜100点）でヤコビ行列の特異値分解（SVD）を行い、最小特異値が十分に0から離れているか確認する。

```
import torch

def check_jacobian_rank(decoder, z_samples, latent_dim):
    """ヤコビ行列のフルランク条件を検証"""
    min_singular_values = []
    for z in z_samples:
        z = z.requires_grad_(True)
        f = decoder(z)
        n_out = f.shape[-1]

        # ヤコビ行列の計算
        J = torch.zeros(n_out, latent_dim)
        for a in range(n_out):
            grad = torch.autograd.grad(f[a], z, retain_graph=True)[0]
            J[a] = grad

        # 特異値分解
        svs = torch.linalg.svdvals(J)
        min_sv = svs[-1].item()
        min_singular_values.append(min_sv)

    min_overall = min(min_singular_values)
    print(f"最小特異値の最小値: {min_overall:.6f}")
    if min_overall > 1e-4:
        print("✓ フルランク条件を満たしています（仮定A2(b)）")
    else:
        print("✗ 警告: ランクが落ちている点があります")
    return min_singular_values
```

条件（iv）：デコーダが単射。

実装上の意味：異なる潜在点 $z_1 \neq z_2$ に対して $f_\theta(z_1) \neq f_\theta(z_2)$ であること。厳密な検証は難しいが、近似的には以下で確認可能。

```
def check_injectivity(decoder, z_samples, threshold=1e-3):
    """デコーダの近似的単射性を検証"""
    outputs = [decoder(z).detach() for z in z_samples]
    n = len(outputs)
```

```
    collisions = 0
    for i in range(n):
        for j in range(i+1, n):
            if torch.norm(z_samples[i] - z_samples[j]) > threshold:
                if torch.norm(outputs[i] - outputs[j]) < threshold:
                    collisions += 1
                    print(f"  潜在点 {i} と {j} が同じデータ点に写されています")
    if collisions == 0:
        print("✓ 単射性の近似検証に合格")
    else:
        print(f"✗ {collisions} 組の衝突が検出されました")
```

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—「正しい地形図」になっているかの確認

X社のデータサイエンスチームはVAE訓練後に4つの条件を検証しました。

条件1（関心領域のコンパクト性）： 潜在空間上でデータの99.7%が含まれる領域 $\|z\| \leq 3$ を関心領域として設定。→ 自動的にコンパクト。✓

条件2（滑らかさ）： デコーダに nn.Tanh() を使用。→ C^∞ 級を保証。✓

条件3（フルランク）： 潜在空間の100点でヤコビ行列のSVDを実行。最小特異値の最小値は 8.3×10^{-3}（$> 10^{-4}$）。→ フルランク条件を満たす。✓

条件4（単射性）： 1,000組のランダムペアで近似検証。衝突なし。→ 単射性の近似検証に合格。✓

4条件すべて合格。X社の経営環境多様体は数学的に正当な C^∞ 級埋め込み部分多様体です。 この多様体の上で計算される計量、クリストッフェル記号、曲率テンソル、Lie微分はすべて well-defined であり、経営環境の構造を正しく反映していると期待できます。

ケーススタディB：国際情勢—データが疎い領域の問題

安全保障分析チームの検証では、データ公開が限定的な一部の国家（特に権威主義体制の国家のデータ）に関して、ヤコビ行列の最小特異値が低下する傾向が確認されました。具体的には、特定の国のデータ周辺で最小特異値が 3.2×10^{-5} まで低下し、フルランク条件の閾値（10^{-4}）を下回りました。

チームは以下の判断を行いました。

(1) **主要国・主要地域（G20諸国＋主要同盟国）：** データ密度が高く、フルランク条件を満たす。分析を続行。

(2) **データが疎な中小国：** フルランク条件が限界的。分析結果に「信頼度：中」のフラグを付けて提示。

(3) **データが特に疎な一部の国家：** フルランク条件を満たさない。この領域での曲率やLie微分の計算結果は信頼できない。安全保障会議に対し、追加のインテリジェンスデータ収集を勧告。

これは重要な教訓です。 多様体分析は「すべての領域で均一に信頼できる」わけではありません。データの密度や品質が高い領域では信頼性が高く、データが疎な領域では信頼性が低下します。**分析結果には常に「信頼度マップ」を添えるべきです。**

本章の品質チェックポイント—先に進むべきか、戻るべきか？

各章末尾に品質チェックポイントがあります。不合格の場合、先に進んでも結果は信頼できません。データ収集（第9章Step 1）に戻ってやり直すことを検討してください。

▶ 数学の基礎から理解したい方へ

命題1.1の4条件は、第2章以降のすべてが well-defined であるための必要条件。(i) コンパクト性は定理1.1の適用に必須。(ii) C^k $(k \geq 3)$ は曲率テンソルの存在に必須。(iii) フルランクは計量の正定値性に必須。(iv) 単射は像が多様体であることに必須。

いずれが破れても、後続の計算は数学的に正当化されない。

▶ AIモデルの基礎から理解したい方へ

☐ チェックリスト（第1章終了時点）

1. 活性化関数は tanh/GELU/Softplus か？ → Yes: 次へ ／ No: **変更して再訓練**
2. ヤコビ行列の最小特異値 $> 10^{-4}$ か？ → Yes: 次へ ／ 一部 No: **その領域を除外するか d を下げる**
3. 再構成誤差は元データの分散の10%以下か？ → Yes: 次へ ／ No: **アーキテクチャ変更**

4. デコーダの単射性は確認できたか？ → Yes: 次へ ／ 衝突あり: **潜在次元 d を増やす**

不合格の場合： 第2章以降に進んでも、計量・曲率・Lie微分の計算結果は信頼できません。

▶ ビジネス・政策の意思決定に活用したい方へ

チームに確認：「4条件はすべて満たされているか？」

- 4つとも合格 → 次章へ
- 1つでも不合格 → **先に進まないでください。** チームに設計変更を指示してください。

次章予告： 第2章では、多様体上に「方向」（接ベクトル）と「距離」（リーマン計量）を定義します。

ケーススタディA予告： X社の経営環境多様体に初めて「距離」が定義され、「新興国市場と国内市場はどれだけ離れているか？」が数値化されます。

ケーススタディB予告： 「日本とオーストラリアの安全保障環境は、日本と韓国よりも『近い』のか？」がリーマン距離で定量的に答えられるようになります。

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(14, 6))

# 球面 S² - 2次元多様体の最も基本的な例
ax1 = fig.add_subplot(121, projection='3d')
u, v = np.meshgrid(np.linspace(0,2*np.pi,50), np.linspace(0,np.pi,50))
ax1.plot_surface(np.sin(v)*np.cos(u), np.sin(v)*np.sin(u), np.cos(v),
    alpha=0.6, cmap='viridis')
ax1.set_title('Sphere S² - 2D manifold in 3D')

# トーラス T² - 球面とは位相的に異なる2次元多様体
ax2 = fig.add_subplot(122, projection='3d')
R, r = 2.0, 0.8
th, ph = np.meshgrid(np.linspace(0,2*np.pi,50), np.linspace(0,2*np.pi,50))
ax2.plot_surface((R+r*np.cos(ph))*np.cos(th),
    (R+r*np.cos(ph))*np.sin(th), r*np.sin(ph), alpha=0.6, cmap='plasma')
```

```
ax2.set_title('Torus T² - 2D manifold in 3D')


> **【図：図1-2：2次元多様体の例（球面とトーラス）】**
*図1-2：3次元空間に埋め込まれた2次元多様体の例。球面S²（穴なし）とトーラスT²（穴1つ）*

plt.show()
```

第2章：接空間とリーマン計量

この章で得られるもの—コストに見合うリターン

第1章で「多様体」という舞台を作りました。しかし、舞台だけでは何もできません。この章では、多様体の上に2つの基本構造を定義します。第一は**接空間**—多様体の各点で「どの方向に動けるか」を定義するもの。第二は**リーマン計量**—多様体の上で「距離」と「角度」を測れるようにするもの。この2つが揃うことで、多様体は単なる「形」から、定量的な計算が可能な「地形図」に変わります。

▶ 数学の基礎から理解したい方へ

接空間 T_pM は多様体の各点 p における「許される速度ベクトル」の空間であり、d 次元ベクトル空間をなす。接束 $TM = \sqcup_{p\in M} T_pM$ はベクトル場の定義域であり、第4章のLie微分、第12章の制御ベクトル場の定義に不可欠である。

リーマン計量 g は各接空間上の滑らかに変化する正定値対称双線形形式の族であり、計量の導入により以下が可能になる：

(i) 曲線の長さ $L[\gamma] = \int \sqrt{g_{ij}\dot{\gamma}^i\dot{\gamma}^j}\, dt$ と測地線の定義。

(ii) 引き戻し計量 $g_{ij}(z) = \left(J_{f_\theta}^{\top} J_{f_\theta}\right)_{ij}$ によるVAE潜在空間の幾何構造の定量化。

(iii) クリストッフェル記号 Γ_{ij}^k（第3章）の計算の前提条件（$g_{ij} \in C^{k-1}$ が必要）。

(iv) スカラー曲率 $\mathrm{Scal}(z) = g^{ij}\mathrm{Ric}_{ij}$（第3章）の計算。

(v) 計量のLie微分 $\left(\mathcal{L}_V g\right)_{ij} = \nabla_i V_j + \nabla_j V_i$（第4章）の計算。

(vi) エネルギー汎関数 $E[\gamma] = \frac{1}{2}\int g_{ij}\dot{\gamma}^i\dot{\gamma}^j\,dt$ の最小化による測地線の計算（第13章）。

(vii) ポントリャーギンの最大原理におけるハミルトニアン $H(z,p,u)$ の定式化（第13章）。

つまり、計量は本書の第2章以降の**すべての計算の基盤**である。計量なしには、(i)〜(vii) のいずれも定義されない。

コスト： 内積の族という抽象的な概念、および引き戻し計量の計算に必要なヤコビ行列の理解。 **リターン：** Step 4以降のすべての計算の基盤。計量が定義されなければ、本書の手順のStep 4〜10は**一切実行できない**。

▶ AIモデルの基礎から理解したい方へ

UMAPやt-SNEの散布図には「距離」が厳密に定義されていません。UMAPの2次元散布図上で2つの点が3センチ離れていても、それが「元のデータ空間でどれだけ離れているか」を定量的に知ることはできません。UMAPは「近い点は近くに、遠い点は遠くに配置する」傾向がありますが、距離の**定量的な値**は保証しません。

本章で学ぶリーマン計量は、VAEのデコーダのヤコビ行列から計算される**引き戻し計量**として実装されます。これは「潜在空間上の微小な移動が、データ空間でどれだけの変化を引き起こすか」を各点・各方向で正確に定量化する指標です。

引き戻し計量を使うことで、以下が可能になります。

- **2つの経営状態の「本当の距離」の計算。** ユークリッド距離（直線距離）ではなく、データ空間の構造を反映した「リーマン距離」が計算できます。
- **「どの方向に動くと、データ空間で大きな変化が起きるか」の特定。** 計量テンソルの固有値が大きい方向が「敏感な方向」、小さい方向が「鈍感な方向」です。
- **「等距離線」の描画。** ある状態からリーマン距離が等しいすべての状態の集合を描画できます。ユークリッド距離では同心円ですが、リーマン距離では歪んだ楕円になり、経営環境の非等方性（方向によって「遠さ」が異なること）が可視化されます。

引き戻し計量の概念は、2018年のArvanitidis et al.の論文「Latent Space Oddity」で初めてVAEの文脈で実証的に研究されました。この論文は、VAEの潜在空間がユークリッド的に扱われているが実際には「曲がっている」ことを示し、引き戻し計量の導入により潜在空間の真の幾何構造が明らかになることを実証しました。

コスト： VAEデコーダのヤコビ行列の計算（PyTorchのautogradで自動化可能。計算量は1回の推論パスの d 倍程度）。 **リターン：** 「2つの状態はどれだけ離れているか」「どの方向に動くとどれだけの変化が起きるか」が、データ空間の構造を反映した形で定量的に答えられるようになる。

▶ ビジネス・政策の意思決定に活用したい方へ

本章を経ることで、経営環境多様体・国際情勢多様体に初めて**「距離」**が定義されます。

「距離」が定義されると何が変わるのか。具体例で説明します。

Before（距離なし）： 経営企画部門の会議で、「新興国市場は国内市場とかなり違いますね」「欧州市場の方が近い気がします」という定性的な議論が行われます。「かなり違う」「近い気がする」は、担当者の経験と直感に基づいた判断であり、人によって意見が異なります。

After（リーマン距離あり）： 「新興国市場と国内市場のリーマン距離は4.7です。欧州市場との距離は2.1です。新興国市場への移行は、欧州市場への移行の2倍以上の『構造的距離』があります」という定量的な議論が可能になります。さらに、「新興国市場への距離4.7のうち、為替リスク方向の成分が3.2で最も大きく、市場規模方向の成分は1.1で比較的小さい」のように、距離の方向分解も可能です。

コスト： データサイエンスチームがこの章の手法を実装する時間（引き戻し計量の計算コードは本章で提供されます）。 **リターン：** 「似ている」「違う」を数値で語れるようになる。経営会議で「根拠のある距離」に基づいた議論が可能になる。

2.1 接ベクトルと接空間

▶ 数学の基礎から理解したい方へ

接ベクトルの定義

多様体 M の各点 p に「速度ベクトル」を定義する。ユークリッド空間 $\mathbb{R}^n$ では、点 p を通る曲線 $\gamma(t)$（$\gamma(0)=p$）の速度ベクトル $\dot{\gamma}(0)=\left(\dot{\gamma}^1(0),\dots,\dot{\gamma}^n(0)\right)$ は、各成分の微分として自然に定義される。しかし、一般の多様体 M は $\mathbb{R}^n$ の中に「埋め込まれている」とは限らないため、速度ベクトルの定義にはチャート（座標系）を用いる必要がある。

定義2.1（接ベクトル）． d 次元可微分多様体 M の点 $p \in M$ を通る滑らかな曲線 $\gamma: (-\varepsilon, \varepsilon) \to M$（$\gamma(0) = p$, $\varepsilon > 0$）に対し、チャート (U, φ)（$p \in U$）を用いて、γ の p における**接ベクトル**（tangent vector）を

$$v = \dot{\gamma}(0) = \left.\frac{d}{dt}\right|_{t=0} (\varphi \circ \gamma)(t) \in \mathbb{R}^d$$

で定義する。

ここで $\varphi \circ \gamma: (-\varepsilon, \varepsilon) \to \mathbb{R}^d$ は、多様体上の曲線 γ をチャート φ で $\mathbb{R}^d$ に写した曲線であり、$\mathbb{R}^d$ 内の曲線の微分は通常の微積分で定義される。

注意2.1（座標非依存性）． 接ベクトルの定義はチャートの選択に依存するように見えるが、異なるチャート (U', φ') を用いた場合の接ベクトル v' とは、遷移関数のヤコビ行列で関係づけられる：

$$v'^{\,i} = \frac{\partial(\varphi' \circ \varphi^{-1})^i}{\partial z^j}\Big|_{\varphi(p)} v^j$$

これは、v が座標変換に対して共変的に（1階テンソルとして）変換されることを意味し、接ベクトルは座標系の選択によらない幾何学的対象である。

【高校数学からの橋渡し】 接ベクトルは、高校の物理で学んだ「速度ベクトル」の一般化です。

ボールを投げたとき、時刻 t でのボールの位置が $(x(t), y(t))$ ならば、速度ベクトルは $(\dot{x}(t), \dot{y}(t)) = (dx/dt, dy/dt)$ です。これは「ボールがその瞬間にどの方向にどれだけ速く動いているか」を表します。

多様体上の接ベクトルも同じ意味です。ただし、多様体は一般に $\mathbb{R}^n$ ではないため（地球の表面は平面ではないため）、「速度ベクトル」を定義するにはチャート（地図）を使って $\mathbb{R}^d$ に移す必要があります。

たとえば、地球の表面（球面 S^2）上を移動する飛行機の速度ベクトルは、地球の表面に「接する」平面の中にあります。上空に向かったり地中に潜ったりする速度は、「球面上の動き」としては意味がありません。この「接する平面」が接空間、その中のベクトルが接ベクトルです。

接空間の定義

定義2.2（接空間）． 点 $p \in M$ における**接空間**（tangent space）T_pM は、p を通るすべての滑らかな曲線の速度ベクトルの集合であり、d 次元ベクトル空間をなす。

局所座標 $\left(z^1, \ldots, z^d\right)$ に対し、T_pM の**標準基底**は

$$\left\{\left.\frac{\partial}{\partial z^1}\right|_p, \left.\frac{\partial}{\partial z^2}\right|_p, \ldots, \left.\frac{\partial}{\partial z^d}\right|_p\right\}$$

であり、任意の接ベクトル $v \in T_pM$ は

$$v = v^i \left.\frac{\partial}{\partial z^i}\right|_p = \sum_{i=1}^{d} v^i \left.\frac{\partial}{\partial z^i}\right|_p$$

と一意に表される（アインシュタインの縮約規約を使用）。

注意2.2（$\partial/\partial z^i$ の意味）． $\partial/\partial z^i|_p$ は、「p を通り、z^i 座標の方向にのみ動く曲線の速度ベクトル」である。具体的には、曲線 $\gamma_i(t) = \varphi^{-1}(\varphi(p) + t\mathbf{e}_i)$（$\mathbf{e}_i$ は $\mathbb{R}^d$ の第 i 標準基底ベクトル）の $t = 0$ での速度ベクトルが $\partial/\partial z^i|_p$ である。

微分幾何学では、接ベクトルを「関数の方向微分を行う微分作用素」として同定する流儀もある。すなわち、接ベクトル $v \in T_pM$ は、$C^\infty(M)$ から $\mathbb{R}$ への写像 $v: f \mapsto v(f) = v^i\, \partial f/\partial z^i|_p$ として定義される。この同定の下では、$\partial/\partial z^i|_p$ は「座標関数 z^i の方向に沿った偏微分作用素」である。

【高校数学からの橋渡し】 接空間は、「ある点での許される速度ベクトルの全体」です。

> **【図：図2-1：球面上の接空間】** *図2-1：球面S^2上の点pにおける接空間T_pS^2（オレンジの平面）。赤と緑の矢印は接ベクトル（その点での「動ける方向」）*

2次元の球面 S^2 の場合、各点 p での接空間 T_pS^2 は、p における球面の接平面です。これは2次元のベクトル空間であり、S^2 上の曲線の速度ベクトルはすべてこの接平面の中に含まれます。

VAEの潜在空間が d 次元であれば、各点の接空間は d 次元のベクトル空間です。$d = 6$（X社の経営環境多様体の次元）ならば、各経営状態に6つの独立な「動ける方向」があり、それらの方向の組み合わせで任意の方向に動くことができます。

アインシュタインの縮約規約について。 $v = v^i\, \partial/\partial z^i$ は $v = \sum_{i=1}^{d} v^i\, \partial/\partial z^i$ の省略形です。ルールは「同じ文字の添字が上付き（v^i）と下付き（$\partial/\partial z^i$ の i は慣習的に下付きと見なす）に現れたら、その添字について合計する」というものです。テンソル計算では添字が何重にもなるため、$\sum$ 記号を省略するこの規約が非常に便利です。本書では以降、この規約を一貫して使用します。

接束とベクトル場

定義2.3（接束）． 多様体 M の接空間すべてを集めた集合

$$TM = \bigsqcup_{p \in M} T_p M = \{(p, v) \mid p \in M, v \in T_p M\}$$

を**接束**（tangent bundle）と呼ぶ。TM は $2d$ 次元の可微分多様体としての自然な構造を持つ。

【橋渡し】 接束は、「多様体のすべての点の接空間を束ねたもの」です。地球（球面 S^2）の各点に接平面を貼り付けて、それらすべてを1つの空間にまとめたものが接束 TS^2 です。S^2 は2次元ですが、TS^2 は4次元です（点の位置を指定するのに2つのパラメータ、接平面内のベクトルを指定するのに2つのパラメータ）。

定義2.4（ベクトル場）． M 上の**ベクトル場**（vector field）V とは、各点 $p \in M$ に接ベクトル $V(p) \in T_p M$ を滑らかに割り当てる写像 $V: M \to TM$（接束の滑らかな切断 $V \in \Gamma(TM)$）のことである。すべてのベクトル場の集合を $\mathfrak{X}(M)$ と書く。局所座標では $V = V^i(z)\,\partial_i$（$\partial_i = \partial / \partial z^i$ の略記）と表される。ここで $V^i(z)$ は M 上の滑らかな関数（$V^i \in C^\infty(M)$）。

ベクトル場の直感的意味。 ベクトル場は「多様体の各点に矢印を1本ずつ置いたもの」です。天気図の風向き・風速の図（各地点に風の方向と強さを表す矢印が描かれている）は、地球の表面（球面 S^2）上のベクトル場の最も身近な例です。

ベクトル場には「滑らかさ」が要求されます。つまり、隣り合う点の矢印の向きは急激には変わりません。台風の目のような特異点（矢印の大きさがゼロになる点）は存在しうるが、ほとんどの点では矢印が滑らかに変化します。

▶ AIモデルの基礎から理解したい方へ

接ベクトル、接空間、ベクトル場とAIモデルの関係を整理します。

接ベクトルとVAEの関係。 VAEの潜在空間 $\mathbb{R}^d$ 上の点 z から、微小な方向 δz に動くと、デコーダの出力は1次近似で

$$f_\theta(z + \delta z) \approx f_\theta(z) + J_{f_\theta}(z) \cdot \delta z$$

と変化します。ここで $J_{f_\theta}(z) \in \mathbb{R}^{n \times d}$ はヤコビ行列であり、その (a, i) 成分は $\partial f_\theta^a / \partial z^i$ です。

つまり、潜在空間での「方向 δz」が、データ空間での「変化 $J \cdot \delta z$」に対応します。潜在空間での方向 δz は接ベクトルであり、接ベクトルが「データ空間でどのような変化に対応するか」をヤコビ行列が教えてくれるのです。

ヤコビ行列のPython実装。 第1章で紹介した check_jacobian_rank 関数のコアとなるヤコビ行列の計算部分を詳しく解説します。

```
import torch

def compute_jacobian(decoder, z):
    """
    VAEデコーダのヤコビ行列を計算する。

    Args:
        decoder: 訓練済みVAEデコーダ (nn.Module)
        z: 潜在空間上の点 (shape: [d])

    Returns:
        J: ヤコビ行列 (shape: [n, d])
           J[a, i] = ∂f_θ^a / ∂z^i
    """
    z = z.requires_grad_(True)
    f = decoder(z)  # shape: [n] (データ空間の次元)

    n_out = f.shape[-1]  # データ空間の次元 n
    d_in = z.shape[-1]   # 潜在空間の次元 d

    J = torch.zeros(n_out, d_in)
    for a in range(n_out):
        # f の第a成分を z で偏微分
        # retain_graph=True: 他の成分の微分にも計算グラフを再利用
        # create_graph=True: 第3章でΓ計算時にJの微分 (2階微分) が必要
        grad = torch.autograd.grad(
            f[a], z,
            retain_graph=True,
            create_graph=True
        )[0]
        J[a] = grad

    return J

# 使用例
```

```
# decoder = trained_vae.decoder
# z = torch.tensor([0.5, -0.3, 1.2, 0.0, -0.7, 0.4])  # 6次元の潜在点
# J = compute_jacobian(decoder, z)
# print(f"Jacobian shape: {J.shape}")  # [n, 6]
```

コードの各行の意味：

- z.requires_grad_(True) → PyTorchの自動微分システムに「z に関する微分を追跡する」と伝えます。これがないと autograd.grad がエラーを出します。
- f = decoder(z) → デコーダにzを入力してデータ空間の出力fを得ます。f の各成分 f^a が $z^1, z^2, \ldots, z^d$ の関数です。
- torch.autograd.grad(f[a], z, ...) → f の第 a 成分（スカラー値）を z（d 次元ベクトル）で偏微分します。結果は d 次元ベクトル $\left(\partial f^a / \partial z^1, \ldots, \partial f^a / \partial z^d\right)$ で、ヤコビ行列の第 a 行になります。
- retain_graph=True → $a = 0$ で微分した後も計算グラフを保持し、$a = 1,2,\ldots$ でも再利用可能にします。
- create_graph=True → 微分の結果自体も微分可能にします。第3章でクリストッフェル記号を計算する際に g_{ij} の微分（ヤコビ行列の微分 = 2階微分）が必要になるため、このオプションが不可欠です。

ベクトル場とNeural ODE。 Neural ODE $dz/dt = f_\theta(z,t)$ の右辺 $f_\theta(z,t)$ は、潜在空間の各点 z に速度ベクトル $f_\theta(z,t) \in T_z\mathbb{R}^d \cong \mathbb{R}^d$ を割り当てるもの—つまりベクトル場です。

第12章（Step 7）では、政策をベクトル場としてモデル化します。たとえば「コスト削減施策」は、各経営状態 z に対して「この施策を打つと経営状態が $V_{\text{cost}}(z)$ の方向に動く」というベクトルを割り当てます。これがベクトル場 $V_{\text{cost}} \in \mathfrak{X}(M)$ です。

ベクトル場はニューラルネットワークで学習されます。具体的には、時系列データ $\{z(t_k)\}$ の差分 $V(z) \approx \left(z\left(t_{k+1}\right) - z(t_k)\right) / \left(t_{k+1} - t_k\right)$ を教師データとして、ニューラルネットワーク $V_\xi : \mathbb{R}^d \to \mathbb{R}^d$ を訓練します。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—経営環境の「方向」を理解する

X社の経営環境多様体は6次元です。この多様体上の各点（各経営状態）には、6つの独立な「方向」があります。

これらの方向は、具体的には何に対応するのでしょうか？VAEが学習した潜在空間の各次元は、必ずしも「売上方向」「利益率方向」のように直感的に解釈できるとは限りません。しかし、ヤコビ行列 J を分析することで、各潜在次元がデータ空間のどの変数に最も強く影響するかを特定できます。

たとえば、X社のチームの分析では、6つの潜在次元はおおむね以下に対応していることが判明しました。

- 第1次元：国内需要の強さ（売上、来店数、消費者信頼感と強く連動）
- 第2次元：コスト構造（原材料価格、為替レート、人件費と連動）
- 第3次元：競争環境（市場シェア、競合数、価格競争度と連動）
- 第4次元：新興国市場の成熟度（対象国GDP、中間層比率と連動）
- 第5次元：規制環境（政治安定性指数、規制環境指数と連動）
- 第6次元：マクロ経済サイクル（GDP成長率、金利、インフレ率と連動）

第12章では、「新興国市場進出施策」を6次元のベクトル場として表現し、「この施策は主に第4次元（新興国市場の成熟度）方向に経営状態を動かすが、第2次元（コスト構造）にも影響がある」のように、施策の影響を方向ごとに分解して分析します。

ケーススタディB：国際情勢の「動きの方向」

国際情勢多様体（8次元）の各次元は、分析チームの解釈では以下に対応していました。

- 第1次元：大国間パワーバランス（米中GDP比、軍事費比率と連動）
- 第2次元：経済的相互依存度（貿易依存度行列の主成分と連動）
- 第3次元：同盟ネットワークの構造（二国間同盟の密度と連動）
- 第4〜8次元：地域的な安全保障構造の変動因子

同盟強化政策は主に第3次元方向のベクトル場であり、経済制裁政策は主に第2次元方向のベクトル場です。Lie括弧 $\left[V_{\text{alliance}}, V_{\text{sanction}}\right]$ が非ゼロであれば、この2つの政策は干渉し、実行順序によって結果が異なることを意味します。

2.2 リーマン計量—多様体上の「距離」の定義

▶ 数学の基礎から理解したい方へ

内積の復習と一般化

$\mathbb{R}^n$ 上の標準内積は $\langle u, v\rangle = \sum_{i=1}^{n} u^i v^i$ である。この内積は（i）双線形（各変数について線形）、（ii）対称（$\langle u, v\rangle = \langle v, u\rangle$）、（iii）正定値（$\langle u, u\rangle > 0,\ u \neq 0$）を満たす。

この内積を使えば、ベクトルの長さ $\| u \| = \sqrt{\langle u, u\rangle}$ と2つのベクトルの角度 $\cos\theta = \langle u, v\rangle / (\| u \| \| v \|)$ が定義される。

$\mathbb{R}^n$ の標準内積は**すべての点で同じ**内積を使っている。リーマン計量は、この内積を**各点で異なるもの**にすることで、「空間の各場所で距離の測り方が異なる」状況を記述する。

リーマン計量の定義

定義2.5（リーマン計量）． d 次元可微分多様体 M 上の**リーマン計量**（Riemannian metric）g とは、各点 $p \in M$ の接空間 T_pM 上に定義された正定値対称双線形形式

$$g_p : T_pM \times T_pM \to \mathbb{R}$$

の族であって、g_p が p に関して滑らかに（C^∞ 級に）変化するものをいう。

局所座標 $\left(z^1, \ldots, z^d\right)$ では、g は $d \times d$ の正定値対称行列値関数 $g_{ij}(z)$ で表される：

$$g_p(u, v) = g_{ij}(z)\, u^i v^j = \sum_{i=1}^{d} \sum_{j=1}^{d} g_{ij}(z)\, u^i v^j$$

ここで $u = u^i \partial_i,\ v = v^j \partial_j \in T_pM$ であり、$z = \varphi(p)$ は p の局所座標である。

正定値とは、$0 \neq u \in T_pM$ に対して $g_p(u, u) = g_{ij}(z) u^i u^j > 0$ が成り立つことをいう。行列の言葉では、$g_{ij}(z)$ のすべての固有値が正であること。

対称とは、$g_{ij}(z) = g_{ji}(z)$ が成り立つことをいう。

【高校数学からの橋渡し】 リーマン計量を理解する最も簡単な方法は、「三平方の定理の一般化」として考えることです。

高校で学んだ三平方の定理（ピタゴラスの定理）は、平面上の2点間の距離を

$$ds^2 = dx^2 + dy^2$$

で与えます。ここで ds は微小距離、dx と dy は x 方向と y 方向の微小変位です。

これは $g_{11} = 1,\ g_{22} = 1,\ g_{12} = g_{21} = 0$ の場合、すなわち

$$ds^2 = g_{11}dz^1dz^1 + g_{12}dz^1dz^2 + g_{21}dz^2dz^1 + g_{22}dz^2dz^2 = (dz^1)^2 + (dz^2)^2$$

に一致します。

リーマン計量では、g_{ij} が点ごとに変わります。たとえば $g_{11}(z) = 4$ の点では、z^1 方向の微小距離が $ds^2 = 4\left(dz^1\right)^2 + \cdots$ となり、同じ dz^1 の変位でも「距離が2倍に測られる」—つまり、z^1 方向に動くのが「2倍大変」であることを意味します。

地球の表面で考えると、赤道付近では経度1度の距離は約111キロメートルですが、北緯60度では約56キロメートルです。同じ「経度1度」の変位でも、場所によって距離が異なる—これがリーマン計量の効果です。球面上のリーマン計量は $g_{11} = R^2,\ g_{22} = R^2\sin^2\theta,\ g_{12} = 0$（$\theta$ は余緯度、ϕ は経度、R は半径）であり、$\sin^2\theta$ の項が「緯度による距離の変化」を記述しています。

曲線の長さとエネルギー汎関数

定義2.6（曲線の長さ）． 多様体 (M, g) 上の滑らかな曲線 $\gamma: [a, b] \to M$ の**長さ**（length）は

$$L[\gamma] = \int_a^b \sqrt{g_{ij}\big(\gamma(t)\big)\,\dot{\gamma}^i(t)\,\dot{\gamma}^j(t)}\ dt$$

で定義される。

定義2.7（エネルギー汎関数）． 同じ曲線 γ の**エネルギー**（energy）は

$$E[\gamma] = \frac{1}{2}\int_a^b g_{ij}\left(\gamma(t)\right)\dot{\gamma}^i(t)\,\dot{\gamma}^j(t)\,dt$$

で定義される。

定理2.1（エネルギー最小曲線としての測地線）． 両端点 $\gamma(a)=p,\ \gamma(b)=q$ を固定した変分のもとで、エネルギー汎関数 $E[\gamma]$ の臨界点（オイラー・ラグランジュ方程式の解）は**測地線**（geodesic）であり、測地線方程式

$$\ddot{\gamma}^k(t)+\Gamma_{ij}^k\big(\gamma(t)\big)\,\dot{\gamma}^i(t)\,\dot{\gamma}^j(t)=0$$

を満たす。ここで Γ_{ij}^k はレヴィ・チヴィタ接続のクリストッフェル記号（第3章で定義）である。

【高校数学からの橋渡し】 高校で学んだ曲線の長さの公式を思い出してください。パラメータ表示 $\big(x(t),y(t)\big)$ の曲線の長さは

$$L=\int_a^b\sqrt{\dot{x}(t)^2+\dot{y}(t)^2}\,dt$$

でした。これはリーマン計量が $g_{11}=g_{22}=1,\ g_{12}=0$（平面のユークリッド計量）の場合に一致します。

リーマン計量を使った公式 $L[\gamma]=\int\sqrt{g_{ij}\dot{\gamma}^i\dot{\gamma}^j}\,dt$ は、g_{ij} が点ごとに変わる一般の場合です。「空間が曲がっている」効果が g_{ij} を通じて反映されます。

測地線はこの長さ（より正確にはエネルギー）を最小化する曲線です。平面では最短経路は直線ですが、球面では**大圏**（great circle）—球の中心を通る平面で球面を切ったときの交線—になります。東京からニューヨークへの最短飛行ルートが北極圏を通るのは、球面上の測地線がそのような経路になるからです。

引き戻し計量

定義2.8（引き戻し計量）． VAEデコーダ $f_\theta\colon U\subset\mathbb{R}^d\to\mathbb{R}^n$（仮定A2を満たす）に対し、$\mathbb{R}^n$ のユークリッド計量の**引き戻し**（pullback）として

$$g_{ij}(z)=\sum_{a=1}^{n}\frac{\partial f_\theta^a}{\partial z^i}(z)\,\frac{\partial f_\theta^a}{\partial z^j}(z)=\Big(J_{f_\theta}(z)^\top J_{f_\theta}(z)\Big)_{ij}$$

を定義する。ここで $J_{f_\theta}(z)\in\mathbb{R}^{n\times d}$ はデコーダのヤコビ行列である。

命題2.1（引き戻し計量の性質）． 仮定A2の下で（$\operatorname{rank}J_{f_\theta}(z)=d,\ \forall z\in U$）：
（i）$g_{ij}(z)$ は各点で正定値対称行列。（ii）$f_\theta\in C^k$ のとき $g_{ij}\in C^{k-1}$。

証明. (i) 対称性は $\left(J^\top J\right)_{ij} = \left(J^\top J\right)_{ji}$ から明らか。正定値性：任意の $0 \neq v \in \mathbb{R}^d$ に対し、$J_{f_\theta}(z)$ がフルランク d であるから $Jv \neq 0$。したがって $v^\top J^\top J v = \| Jv \|^2 > 0$。 (ii) $f_\theta \in C^k$ のとき $J_{f_\theta} = \left(\partial f_\theta^a / \partial z^i\right) \in C^{k-1}$。$g_{ij} = \sum_a \left(\partial f^a / \partial z^i\right)\left(\partial f^a / \partial z^j\right)$ は C^{k-1} 関数の積の和であるから $g_{ij} \in C^{k-1}$。 ▫

引き戻し計量の幾何学的意味。 潜在空間上の点 z から微小変位 δz を行うと、データ空間での変位は1次近似で $\delta f = J_{f_\theta}(z) \cdot \delta z$ である。データ空間でのユークリッド距離の2乗は

$$\| \delta f \|^2 = \delta z^\top J^\top J \delta z = g_{ij}(z)\, \delta z^i\, \delta z^j$$

したがって、引き戻し計量 $g_{ij}(z)$ は「潜在空間上の微小変位が、データ空間でどれだけの距離変化を引き起こすか」を測る指標である。

直感的には：デコーダの出力が z の変化に敏感な方向（$\| J \cdot \delta z \|$ が大きい方向）では「リーマン距離が大きい」（その方向に動くのは「大変」）。鈍感な方向では「距離が小さい」（その方向に動くのは「容易」）。

▶ AIモデルの基礎から理解したい方へ

引き戻し計量のPython実装。

```
import torch
import torch.nn as nn

class PullbackMetric(nn.Module):
    """
    VAEデコーダから引き戻し計量 g_{ij}(z) = (J^T J)_{ij} を計算する。

    引き戻し計量は、潜在空間上の各点で「データ空間での距離の測り方」を定義する。
    直感的には、「潜在変数をこの方向に少し動かすと、データ空間の出力が
    どれだけ変わるか」を、方向ごとに測る指標。
    """
    def __init__(self, decoder):
        super().__init__()
        self.decoder = decoder  # 訓練済みVAEデコーダ

    def forward(self, z):
        """
        Args:
            z: 潜在空間上の点 (shape: [d])
```

```
        Returns:
            g: 引き戻し計量テンソル (shape: [d, d])
               正定値対称行列 (フルランク条件の下)
        """
        z = z.requires_grad_(True)
        f = self.decoder(z)          # デコーダ出力 (shape: [n])
        n_out = f.shape[-1]          # データ空間の次元 n
        d_in = z.shape[-1]           # 潜在空間の次元 d

        # ヤコビ行列 J ∈ R^{n×d} の計算
        # J[a, i] = ∂f^a/∂z^i
        J = torch.zeros(n_out, d_in)
        for a in range(n_out):
            grad = torch.autograd.grad(
                f[a], z,
                retain_graph=True,     # 他の成分の微分にも計算グラフを再利用
                create_graph=True      # 第3章でg_{ij}の微分 (Γ計算) に必要
            )[0]
            J[a] = grad

        # 引き戻し計量 g = J^T J ∈ R^{d×d}
        g = J.T @ J
        return g

# 使用例：潜在空間の格子点上で計量を計算し、可視化用データを準備
def compute_metric_field(decoder, grid_points, d=2):
    """
    潜在空間の格子点上で引き戻し計量を一括計算する。

    Args:
        decoder: 訓練済みVAEデコーダ
        grid_points: 格子点の座標 (shape: [N, d])
        d: 潜在空間の次元
    Returns:
        metrics: 各格子点の計量テンソル (shape: [N, d, d])
        det_g: 各格子点の計量行列式 (shape: [N])
        cond_g: 各格子点の条件数 (shape: [N])
    """
    metric_fn = PullbackMetric(decoder)
    metrics = []
    det_g = []
    cond_g = []
```

```
    for z in grid_points:
        z_t = torch.tensor(z, dtype=torch.float32)
        g = metric_fn(z_t)
        metrics.append(g.detach().numpy())
        det_g.append(torch.det(g).item())
        cond_g.append(torch.linalg.cond(g).item())

    import numpy as np
    return np.array(metrics), np.array(det_g), np.array(cond_g)
```

引き戻し計量が「なぜ」重要かの再確認。

通常のVAEでは、潜在空間はユークリッド空間（$g_{ij} = \delta_{ij}$：単位行列）として扱われます。つまり、潜在空間のどの方向も「等しい重み」で扱われます。しかし、デコーダが非線形関数であるため、潜在空間の異なる方向への微小移動がデータ空間では異なる大きさの変化を引き起こします。

引き戻し計量 $g_{ij}(z) = \left(J^{\top}J\right)_{ij}$ は、この「方向ごとの感度の違い」を正確に反映した計量です。計量を導入することで、潜在空間上の距離が「データ空間での実質的な変化の大きさ」を反映するようになり、VAEの潜在空間で計算された測地線や曲率が、データ空間における意味のある構造を表すようになります。

Arvanitidis et al. [5] は、この引き戻し計量を用いた測地線がユークリッド直線と大きく異なることを実証し、VAEの潜在空間が「見た目は平坦だが実は曲がっている」ことを示しました（Latent Space Oddity, ICLR 2018）。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—「経営状態間の距離」が初めて定義される

X社のデータサイエンスチームは、潜在空間上の格子点で引き戻し計量を計算し、「経営状態間の距離」を数値化しました。

発見1：ユークリッド距離とリーマン距離は大きく異なる。

経営状態ペア	ユークリッド距離	リーマン距離	比率
国内市場 ↔ 東南アジア市場	2.3	5.8	2.5倍
国内市場 ↔ 欧州市場	3.1	2.1	0.7倍
国内市場 ↔ 北米市場	2.8	3.4	1.2倍

東南アジア市場は、ユークリッド距離（直線距離）では近く見えますが、リーマン距離（データ空間の構造を反映した距離）では遠い。これは、両市場の間に「データ空間上で大きな変化を伴う領域」（たとえば、為替リスクや規制環境の大きな差異）が存在するためです。逆に、欧州市場はユークリッド距離では遠いですが、リーマン距離では近い。経営環境の構造が類似しているためです。

発見2：距離の「方向依存性」。 計量テンソルの固有値分析により、「売上を増やす方向に動くと、利益率が低下する方向にも連動して動く」ことが判明。計量の非対角成分 g_{ij}（$i \neq j$）がこの連動の強さを定量的に表しています。

経営者への示唆： ユークリッド距離だけで市場の「近さ」を判断すると、重大な誤りにつながる。東南アジア市場は「見た目は近いが、移行には大きな構造的コストがかかる」市場であり、欧州市場は「見た目は遠いが、構造的には移行しやすい」市場である。進出先の優先順位は、リーマン距離に基づいて再評価すべきです。

ケーススタディB：安全保障環境の「距離」

安全保障チームは、国際情勢多様体上で以下の結果を得ました。

「日本の安全保障環境とオーストラリアの安全保障環境のリーマン距離は1.4。日本と韓国のリーマン距離は2.7。」

これは地理的な直感（韓国の方が日本に近い）とは異なる結果です。安全保障環境のデータ構造上は、オーストラリアの方が日本と「近い」—すなわち、安全保障政策の構造（同盟関係の型、軍事力の構成、経済安全保障の優先事項）が類似しているのです。

政策立案者への示唆： 同盟政策の優先順位を「地理的近接性」ではなく「安全保障環境のリーマン距離」に基づいて再評価することで、より構造的に整合的な同盟ネットワークの設計が可能になります。

```
import numpy as np
import matplotlib.pyplot as plt

# 引き戻し計量による「等距離線」の可視化
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

theta = np.linspace(0, 2*np.pi, 100)

# 左：ユークリッド計量（平坦）- 等距離線は同心円
for r in [0.5, 1.0, 1.5, 2.0]:
```

```
    axes[0].plot(r*np.cos(theta), r*np.sin(theta), 'b-', alpha=0.5)
axes[0].set_title('Euclidean metric (flat)¥nEqual-distance = concentric circles
')
axes[0].set_aspect('equal')
axes[0].set_xlabel('z¹'); axes[0].set_ylabel('z²')
axes[0].plot(0, 0, 'ko', markersize=8)  # 原点

# 右: 引き戻し計量（曲がった距離）- 等距離線は歪んだ楕円
for r in [0.5, 1.0, 1.5, 2.0]:
    axes[1].plot(r*np.cos(theta)*1.8, r*np.sin(theta)*0.6, 'r-', alpha=0.5)
axes[1].set_title('Pullback metric (curved)¥nEqual-distance = deformed ellipses
')
axes[1].set_aspect('equal')
axes[1].set_xlabel('z¹'); axes[1].set_ylabel('z²')
axes[1].plot(0, 0, 'ko', markersize=8)

plt.suptitle('Equal-distance contours: Euclidean vs Riemannian', fontsize=14)


> **【図：図2-2：ユークリッド計量vs引き戻し計量】**
*図2-2：等距離線の比較。ユークリッド計量（左：同心円）と引き戻し計量（右：歪んだ楕円）。引き戻し計量は「方向ごとの感度の違い」を反映*
plt.tight_layout()
plt.show()
```

▶ AIモデルの基礎から理解したい方へ：コードの解説

上のコードは、ユークリッド計量と引き戻し計量での「等距離線」（中心からの距離が等しい点の集合）の違いを可視化しています。

左図（ユークリッド計量）： 等距離線は同心円です。中心からどの方向に動いても、同じ距離だけ動けば同じ「遠さ」です。これは「空間が平坦で、方向による違いがない」場合の距離です。

右図（引き戻し計量）： 等距離線は歪んだ楕円です。z^1 方向に動くと「遠い」（楕円が z^1 方向に長い）ですが、z^2 方向に動いても「近い」（楕円が z^2 方向に短い）。これは「z^1 方向への微小変位がデータ空間で大きな変化を引き起こし、z^2 方向への微小変位はデータ空間であまり変化を引き起こさない」ことを反映しています。

引き戻し計量は、このような「方向ごとの感度の違い」を自動的に捉えます。

本章の品質チェックポイント—先に進むべきか、戻るべきか？

▶ 数学の基礎から理解したい方へ

命題2.1により、引き戻し計量の正定値性はフルランク条件（仮定A2(b)）に依存する。$g_{ij} \in C^{k-1}$（$k \geq 3$ のとき $g_{ij} \in C^2$）が第3章のクリストッフェル記号の well-definedness に直結する。

判断基準： 計量が正定値であることを検証できなければ、第3章以降の接続・曲率の計算は数学的に正当化されない。計量の条件数 $\kappa(g(z)) = \sigma_{\max}(g)/\sigma_{\min}(g)$ が 10^4 を超える領域は、数値的に不安定であり、その領域での結果は慎重に扱うべきである。

▶ AIモデルの基礎から理解したい方へ

☐ チェックリスト（第2章終了時点）

1. **引き戻し計量を計算できたか？** 潜在空間の代表点（最低50点）で $g_{ij}(z)$ を計算。
   - Yes → 次へ
   - NaNやInfが出た → **活性化関数を確認（第1章のチェック1）。create_graph=True を忘れていないか確認。**
2. **すべての代表点で g は正定値か？** 全固有値が正であることを torch.linalg.eigvalsh(g) で確認。
   - Yes → 次へ
   - 一部でゼロまたは負の固有値 → **ヤコビ行列のランクが落ちている。第1章のチェック2に戻る。**
3. **計量の条件数は 10^4 以下か？** torch.linalg.cond(g) で確認。
   - Yes → 次へ
   - 10^4 を超える領域あり → **その領域での計算は数値的に不安定。正則化 $g + \delta I$（$\delta \sim 10^{-6}$）を適用するか、その領域を分析対象から除外。**
4. **計量が「意味のある構造」を反映しているか？** 等距離線のパターン、計量行列式 $\det(g)$ の空間的分布がドメイン知識と整合するか。
   - 整合する → 次へ

- 非現実的なパターン（たとえば、経営危機の状態と好況の状態がリーマン距離で「近い」） → **データの前処理（第9章）またはVAEの設計を見直してください。**

▶ **ビジネス・政策の意思決定に活用したい方へ**

チームに確認してください：

「リーマン距離で測った市場間の距離は、業界の常識と整合しているか？」

たとえば、リーマン距離が「国内市場と新興国市場は近い」「国内市場と欧州市場は遠い」と示した場合、それはチームの実務的感覚と整合するか。もし計量が「明らかに非現実的な距離」を示す場合、データの前処理やVAEの設計に問題がある可能性があります。

非現実的な結果の例： 経済危機中の経営状態と好況時の経営状態がリーマン距離で「非常に近い」と計算される場合、デコーダがこれらの状態を区別できていない（フルランク条件が限界的、またはVAEの訓練が不十分）可能性がある。**第9章（データ収集とVAE構成）に戻ることを検討してください。**

次章予告： 第3章では、計量から「クリストッフェル記号」（座標の曲がりの補正量）と「曲率テンソル」（空間の曲がり具合の数値）を計算します。

ケーススタディA予告： X社の経営環境多様体のスカラー曲率マップが作成され、「安定領域」と「不安定領域」が色分けされた地形図が完成します。

ケーススタディB予告： 国際情勢多様体の曲率分析により、「現在の国際秩序が最も脆弱な構造的ポイント」が特定されます。

第3章：接続・共変微分・曲率テンソル

この章で得られるもの—コストに見合うリターン

第2章で多様体に「距離」を定義しました。この章ではさらに2つの重要な構造を導入します。第一は**共変微分**—曲がった空間で「ベクトルを微分する」方法。第二は**曲率テンソル**—空間が「どれだけ曲がっているか」を数値化する量。曲率テンソルから導かれるスカラー曲率 $\mathrm{Scal}(z)$ は、経営環境や国際情勢の「安定性/不安定性マップ」を作成するための核心的な道具です。

▶ 数学の基礎から理解したい方へ

レヴィ・チヴィタ接続 ∇ は、リーマン計量 g から一意に定まる、捩れなし（$\Gamma_{ij}^{k} = \Gamma_{ji}^{k}$）かつ計量整合（$\nabla_k g_{ij} = 0$）なアフィン接続である。クリストッフェル記号は $\Gamma_{ij}^{k} = \frac{1}{2} g^{kl}\left(\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij}\right)$ で与えられる。

リーマン曲率テンソル $R^{l}{}_{ijk} = \partial_j \Gamma_{ik}^{l} - \partial_i \Gamma_{jk}^{l} + \Gamma_{jm}^{l}\Gamma_{ik}^{m} - \Gamma_{im}^{l}\Gamma_{jk}^{m}$ は、平行移動の経路依存性を定量化する。リッチテンソル $\mathrm{Ric}_{ij} = R^{l}{}_{ilj}$、スカラー曲率 $\mathrm{Scal} = g^{ij}\mathrm{Ric}_{ij}$ はその縮約。

コスト： テンソル添字計算の習熟と自動微分によるフルチェーン計算の実装。
リターン： スカラー曲率マップはPCA・UMAP・回帰分析のいずれでも原理的に得られない情報であり、本書の最も独自なリターンの1つ。$\mathrm{Scal}(z) > 0$（測地線収束、安定構造）と $\mathrm{Scal}(z) < 0$（測地線発散、不安定構造）の区別は、リスク評価において決定的。

▶ AIモデルの基礎から理解したい方へ

クリストッフェル記号と曲率テンソルは、すべてPyTorchの自動微分で計算されます。手計算は不要です。

なぜ自動微分が必要か。 クリストッフェル記号 Γ_{ij}^{k} は計量 g_{ij} の1階偏微分から構成され、曲率テンソル $R^{l}{}_{ijk}$ はクリストッフェル記号の1階偏微分（すなわち計量の2階偏微分）から構成されます。これらを有限差分法で近似計算することも可能ですが、有限差分は（i）微小幅 h の選択に結果が依存し、（ii）数値誤差が蓄積する、（iii）高次微分ほど精度が劣化する、という問題があります。PyTorchの自動微分は連鎖律を厳密に適用するため、これらの問題を回避し、**理論的に厳密な微分値**を計算します。

コスト： 自動微分コードの実装と計算時間（格子点数 × テンソル成分数の計算）。 **リターン：** 従来手法では得られない「データの非線形構造の定量マップ」。

▶ ビジネス・政策の意思決定に活用したい方へ

この章のリターンは極めて大きい。 曲率マップは「経営環境の地形図」に「等高線と危険地帯の色分け」を加えるものです。

曲率	地形図でのたとえ	経営環境での意味	経営判断への示唆
Scal > 0（正）	なだらかな丘陵	安定的な市場構造	通常のオペレーションで対処可能
Scal ≈ 0	平坦な平野	線形近似が有効な領域	従来の回帰分析でも十分機能
Scal < 0（負）	断崖、急斜面	構造的に不安定な領域	**リスク管理を強化。段階的投資、撤退条件の事前設定が必要**

3.1 接続とクリストッフェル記号

▶ 数学の基礎から理解したい方へ

定義3.1（アフィン接続）． 多様体 M 上の**アフィン接続**（affine connection）∇ は、ベクトル場の対 $(V,W) \in \mathfrak{X}(M) \times \mathfrak{X}(M)$ にベクトル場 $\nabla_V W \in \mathfrak{X}(M)$ を対応させる操作で、以下を満たすもの：

（C1）$\nabla_{fV+gW} U = f\nabla_V U + g\nabla_W U$（第1変数の $C^\infty(M)$-線形性）（C2）$\nabla_V(W+U) = \nabla_V W + \nabla_V U$（第2変数の加法性）（C3）$\nabla_V(fW) = (Vf)W + f\nabla_V W$（第2変数のライプニッツ則）

ここで $f,g \in C^\infty(M)$，$U,V,W \in \mathfrak{X}(M)$ である。

【高校数学からの橋渡し】 なぜ曲がった空間では「普通の微分」ではダメなのか。$\mathbb{R}^n$ ではベクトル場 W の各成分 W^k を座標 z^i で偏微分すれば、W の変化率が求まります。しかし曲面上では、**異なる点の接空間が異なるベクトル空間**です。球面の北極での「東」ベクトルと赤道での「東」ベクトルは、異なる接空間に属しているため、単純に引き算（差分を取る操作）ができません。

接続 ∇ は、「異なる点のベクトルを比較可能にする」ための道具です。「平行移動」（ある点のベクトルを別の点に持っていく操作）を定義し、2点のベクトルの「差」を意味のある形で計算できるようにします。

定義3.2（クリストッフェル記号）． 座標基底 $\{\partial_i = \partial/\partial z^i\}$ に対し、$\nabla_{\partial_i}\partial_j = \Gamma_{ij}^k \partial_k$ で定まる d^3 個の関数 Γ_{ij}^k を**クリストッフェル記号**（Christoffel symbols of the second kind）と呼ぶ。

定理3.1（レヴィ・チヴィタ接続の一意性）． リーマン多様体 (M, g) 上に、以下の2条件を同時に満たすアフィン接続がただ一つ存在する：（LC1）**捩れなし（torsion-free）**：$\Gamma_{ij}^k = \Gamma_{ji}^k$。（LC2）**計量整合（metric-compatible）**：$\nabla_k g_{ij} = 0$。

この一意な接続を**レヴィ・チヴィタ接続**と呼び、そのクリストッフェル記号は

$$\Gamma_{ij}^k(z) = \frac{1}{2} g^{kl}(z) \left(\frac{\partial g_{jl}}{\partial z^i}(z) + \frac{\partial g_{il}}{\partial z^j}(z) - \frac{\partial g_{ij}}{\partial z^l}(z) \right)$$

で一意に決まる。ここで $g^{kl} = \left(g^{-1}\right)_{kl}$ は計量テンソルの逆行列の成分。

共変微分の定義。 ベクトル場 $W = W^k \partial_k$ の ∂_i 方向の共変微分は

$$\nabla_{\partial_i} W = \left(\frac{\partial W^k}{\partial z^i} + \Gamma_{ij}^k W^j \right) \partial_k$$

第1項 $\partial W^k / \partial z^i$ は通常の偏微分（成分の変化率）。第2項 $\Gamma_{ij}^k W^j$ は「座標系の曲がりに起因する見かけの変化」の補正項。共変微分はこの2つの効果を合わせた「本当の変化率」を与える。

▶ **AIモデルの基礎から理解したい方へ**

クリストッフェル記号の自動微分による計算。

```
import torch

def christoffel_symbols(metric_fn, z, d=2):
    """
    計量関数からクリストッフェル記号を自動微分で計算する。

    数学: Γ^k_{ij} = (1/2) g^{kl} (∂g_{jl}/∂z^i + ∂g_{il}/∂z^j - ∂g_{ij}/
∂z^l)

    Args:
        metric_fn: 点zを受け取り、d×d計量テンソルを返す関数
        z: 潜在空間上の点 (shape: [d])
        d: 潜在空間の次元
    Returns:
        Gamma: クリストッフェル記号 (shape: [d, d, d])
               Gamma[k, i, j] = Γ^k_{ij}
    """
    z = z.clone().requires_grad_(True)
```

```
    g = metric_fn(z)
    g_inv = torch.inverse(g)

    # Step 1: 計量の偏微分 ∂g_{jl}/∂z^i を自動微分で計算
    dg = torch.zeros(d, d, d)  # dg[i, j, l] = ∂g_{jl}/∂z^i
    for j in range(d):
        for l in range(d):
            grad = torch.autograd.grad(
                g[j, l], z,
                retain_graph=True,
                create_graph=True  # 曲率計算で∂Γ/∂zが必要
            )[0]
            dg[:, j, l] = grad

    # Step 2: クリストッフェル記号の組み立て
    Gamma = torch.zeros(d, d, d)
    for k in range(d):
        for i in range(d):
            for j in range(d):
                s = 0.0
                for l in range(d):
                    s += g_inv[k, l] * (
                        dg[i, j, l] + dg[j, i, l] - dg[l, i, j]
                    )
                Gamma[k, i, j] = 0.5 * s
    return Gamma
```

create_graph=True がなぜ必要か。 曲率テンソルの計算には $\partial\Gamma^l_{ik}/\partial z^j$ が必要です。Γ は g の1階微分から作られ、g はデコーダの1階微分から作られるので、曲率は**計量の2階微分**（＝デコーダの3階微分）を含みます。create_graph=True は、1階微分の計算結果をさらに微分可能にするオプションです。

▶ ビジネス・政策の意思決定に活用したい方へ

クリストッフェル記号自体は直接的な経営的意味を持ちませんが、次のセクション（曲率テンソル）の計算に不可欠な中間量です。

3.2 リーマン曲率テンソルとスカラー曲率

▶ 数学の基礎から理解したい方へ

定義3.3（リーマン曲率テンソル）． $\Gamma_{ij}^{k} \in C^{1}$（$g_{ij} \in C^{2}$，$f_{\theta} \in C^{3}$ が必要）のとき、

$$R^{l}{}_{ijk}(z) = \frac{\partial \Gamma_{ik}^{l}}{\partial z^{j}} - \frac{\partial \Gamma_{jk}^{l}}{\partial z^{i}} + \Gamma_{jm}^{l}\Gamma_{ik}^{m} - \Gamma_{im}^{l}\Gamma_{jk}^{m}$$

曲率テンソルの幾何学的意味は**平行移動の経路依存性**である。ベクトル v を ∂_i 方向に ε 移動してから ∂_j 方向に ε 移動する場合と、逆順で移動する場合の差は $\varepsilon^{2}R(\partial_i, \partial_j)v + O\left(\varepsilon^{3}\right)$ である。平坦空間（$\mathbb{R}^{n}$）では $R \equiv 0$ であり、曲率テンソルがゼロでないことは空間が「曲がっている」ことの正確な定量化である。

定義3.4（リッチテンソルとスカラー曲率）．

$$\mathrm{Ric}_{ij} = R^{l}{}_{ilj} \quad \text{（第1・第3添字で縮約）}$$

$$\mathrm{Scal}(z) = g^{ij}(z)\,\mathrm{Ric}_{ij}(z) \quad \text{（さらに計量で縮約）}$$

スカラー曲率の幾何学的解釈。 $\mathrm{Scal}(z) > 0$：点 z の近傍で測地球の体積がユークリッド空間より小さい（測地線が収束）。$\mathrm{Scal}(z) < 0$：測地球の体積が大きい（測地線が発散）。

▶ AIモデルの基礎から理解したい方へ

スカラー曲率の計算コード。 クリストッフェル記号をさらにautogradで微分します。

```
def scalar_curvature_2d(metric_fn, z):
    """2次元の場合のスカラー曲率（ガウス曲率の2倍）"""
    z = z.clone().requires_grad_(True)
    Gamma = christoffel_symbols(metric_fn, z, d=2)
    g = metric_fn(z)
    g_inv = torch.inverse(g)

    # 2Dではスカラー曲率 = 2K（Kはガウス曲率）
    # 簡略化のため、det(g)の変化率から近似計算
    det_g = torch.det(g)
    print(f"det(g) = {det_g.item():.6f}")
    print(f"Γ^0_{{01}} = {Gamma[0,0,1].item():.6f}")
```

```
    # 完全な実装は第7章参照（dΓ/dzのautograd計算が必要）
    return Gamma, g, g_inv
```

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—経営環境の「安定領域」と「不安定領域」

X社のチームは、経営環境多様体のスカラー曲率マップを計算しました。結果を色分けした地形図として、経営会議に提示しました。

> **【図：図3-1：経営環境多様体のスカラー曲率マップ】** *図3-1：経営環境多様体のスカラー曲率マップ。赤＝安定領域（正の曲率）、青＝不安定領域（負の曲率）。緑線＝測地線（不安定領域を回避する最適経路）*

発見： 国内市場の中心領域はスカラー曲率が正（赤色、安定構造）。国内市場と新興国市場の境界領域で曲率が強い負の値（-2.3、青色、不安定構造）。

検証： 過去にリーマンショック（2008年）で業績が急落した時期の経営状態が、この曲率負の領域に位置することを確認。COVID-19（2020年）の時期も同様。**曲率分析は過去の危機と整合しており、分析の信頼性に裏付けが得られました。**

経営者への示唆： 新興国市場への進出経路がこの不安定領域を通過する場合、リスクバッファ（予備資金、段階的投資、撤退条件の事前設定）が不可欠です。

ケーススタディB：国際情勢の「構造的脆弱点」

安全保障チームの曲率分析結果：特定の指標の組み合わせ（軍事費急増＋同盟弱体化＋貿易依存度の非対称化）が同時に進行する領域で、スカラー曲率が強い負の値。

政策立案者への示唆： この領域は「国際秩序の構造的脆弱点」であり、小さな外交ショックが大きな秩序変動を引き起こしうる。**現在の国際状態がこの脆弱領域からどれだけ離れているか**が、安全保障リスク評価の核心的指標です。

本章の品質チェックポイント

▶ AIモデルの基礎から理解したい方へ

☐ チェックリスト

1. スカラー曲率を格子点で計算できたか？ NaN/Infが出た場合 → **計量の条件数を確認（第2章チェック3）**
2. 曲率マップに意味のあるパターンがあるか？ ランダムノイズに見える場合 → **データ不足（仮定A3）か、VAE訓練不足の可能性。第9章に戻る。**
3. ランダムシード変更で曲率パターンは安定か？ 不安定な場合 → **曲率に基づく意思決定は控える。**

▶ **ビジネス・政策の意思決定に活用したい方へ**

チームに確認： 「曲率マップの『不安定領域』は、過去の既知の危機と整合するか？」整合しない場合、**第9章（データ収集）からやり直すことを真剣に検討してください。**

第4章：Lie微分とKilling場

この章で得られるもの—コストに見合うリターン

本書で最も独自かつ強力な道具—Lie微分—を導入します。Lie微分は「政策や戦略が空間の構造そのものをどう変えるか」を数値で測定します。**これは従来のどのデータ分析手法でも原理的に不可能だったことです。**

▶ **数学の基礎から理解したい方へ**

Lie微分 $\mathcal{L}_V$ は計量に依存しない（任意の可微分多様体上で定義可能な）微分作用素であり、テンソル場がベクトル場 V のフロー ϕ_t に沿ってどう変化するかを測る。計量のLie微分 $(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$ はKilling方程式の左辺であり、$\mathcal{L}_V g = 0$ は V が等長変換を生成することと同値。Lie括弧 $[V, W]$ は2つのフローの非可換性を測定する。

コスト： フロー、Lie括弧、一般テンソルのLie微分の概念。 **リターン：** (i) 計量のLie微分 → 政策の構造保存/変形の判定。(ii) Lie括弧 → 複数政策の干渉判定。**これらは従来の統計的政策評価手法（DID等）では原理的に得られない情報。**

▶ AIモデルの基礎から理解したい方へ

Lie微分の計算は、ベクトル場（ニューラルネットで表現）のヤコビ行列をautogradで計算することで実装されます。

コスト： 政策ベクトル場の学習（時系列差分からNNで推定）＋Lie微分の計算。
リターン： 「この政策は空間の構造を変えるか」「2つの政策は干渉するか」— **構造的影響の定量評価。**

▶ ビジネス・政策の意思決定に活用したい方へ

この章のリターンは経営者・政策立案者にとって最大です。 従来の政策評価は「結果がどう変わったか」を測定します。本章の手法は「環境の構造そのものがどう変わるか」を**打つ前に**シミュレーションします。

4.1 フローとLie括弧

▶ 数学の基礎から理解したい方へ

定義4.1（フロー）． ベクトル場 $V \in \mathfrak{X}(M)$ の**フロー** $\phi_t: M \to M$ は $\frac{d}{dt}\phi_t(p) = V(\phi_t(p))$，$\phi_0(p) = p$ を満たす1パラメータ変換群。

定義4.2（Lie括弧）． $V, W \in \mathfrak{X}(M)$ の**Lie括弧**：

$$[V, W]^k = V^j \frac{\partial W^k}{\partial z^j} - W^j \frac{\partial V^k}{\partial z^j}$$

幾何学的意味： V で ε 流し、W で ε 流し、V の逆で ε 戻し、W の逆で ε 戻す。出発点からのずれが $\varepsilon^2[V, W] + O(\varepsilon^3)$。$[V, W] = 0 \Leftrightarrow$ 2つのフローは可換。

▶ AIモデルの基礎から理解したい方へ

```
import torch

def lie_bracket(V_net, W_net, z, d=2):
    """2つのベクトル場のLie括弧 [V, W] を自動微分で計算"""
    z = z.requires_grad_(True)
    V = V_net(z)
    W = W_net(z)

    JV = torch.zeros(d, d)  # ∂V^k/∂z^j
    JW = torch.zeros(d, d)  # ∂W^k/∂z^j
```

```
    for k in range(d):
        JV[k] = torch.autograd.grad(V[k], z, retain_graph=True, create_graph=Tru
e)[0]
        JW[k] = torch.autograd.grad(W[k], z, retain_graph=True, create_graph=Tru
e)[0]

    # [V, W]^k = V^j ∂W^k/∂z^j - W^j ∂V^k/∂z^j = JW·V - JV·W
    bracket = JW @ V - JV @ W
    return bracket
```

Lie括弧がゼロ ⇔ 2つの政策は干渉しない（実行順序に依存しない）。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—コスト削減と市場開拓は干渉するか？

X社は「コスト削減」と「新興国市場開拓」を同時に進めることを検討。Lie括弧を計算した結果：

- 国内市場の安定領域：$\| [V_{\text{cost}}, V_{\text{market}}] \| \approx 0.05$（ほぼゼロ→独立に機能）
- 移行途上の領域：$\| [V_{\text{cost}}, V_{\text{market}}] \| \approx 2.1$（大きい→強い干渉）

経営者への示唆： 国内市場では両施策を同時進行して問題ないが、移行フェーズでは**順序を分ける**方が効果的。

ケーススタディB：制裁と同盟強化の干渉

$\| [V_{\text{sanction}}, V_{\text{alliance}}] \|$ が特定の緊張状態で大きくなる → **段階的実施（先に同盟強化、次に制裁）が最適。**

4.2 計量のLie微分とKilling方程式

▶ 数学の基礎から理解したい方へ

定義4.3（計量のLie微分）.

$$(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$$

ここで $V_j = g_{jk} V^k$（添字の下げ）。$\mathcal{L}_V g = 0$ を**Killing方程式**と呼び、これを満たす V を**Killing場**と呼ぶ。Killing場は等長変換（計量を保存する変換）を生成する。

命題4.1. $\mathcal{L}_V g = 0 \Leftrightarrow V$ のフロー ϕ_t は計量の等長変換群を生成する（$\phi_t^* g = g$, $\forall t$）。

▶ AIモデルの基礎から理解したい方へ

```
def lie_derivative_metric(V_net, metric_fn, z, Gamma, d=2):
    """計量のLie微分 (L_V g)_{ij} = ∇_i V_j + ∇_j V_i"""
    z = z.requires_grad_(True)
    V = V_net(z)
    g = metric_fn(z)
    V_lower = g @ V  # V_j = g_{jk} V^k

    dV = torch.zeros(d, d)  # ∂V_j/∂z^i
    for j in range(d):
        dV[:, j] = torch.autograd.grad(V_lower[j], z, retain_graph=True, create_
graph=True)[0]

    nabla_V = torch.zeros(d, d)
    for i in range(d):
        for j in range(d):
            nabla_V[i, j] = dV[i, j]
            for k in range(d):
                nabla_V[i, j] -= Gamma[k, i, j] * V_lower[k]

    L_V_g = nabla_V + nabla_V.T
    return L_V_g
```

結果の解釈： $\| L_V g \| \approx 0$ → 構造保存型。$\| L_V g \|$ が大きい → 構造変形型。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—3つの施策の「構造的インパクト」比較

施策	$\| \mathcal{L}_V g \|$	分類	意味
既存顧客向けサービス強化	0.12	構造保存型	市場構造を壊さず、状態を少し改善
新興国市場への段階的進出	1.8	構造変形型（中）	市場構造に中程度の変化
事業ポートフォリオ抜本再編	4.5	構造変形型（強）	経営環境の地形が大きく変わる

経営者の判断： $\|\mathcal{L}_V g\|$ は「施策の構造的インパクト」の定量指標。リスクとリターンのバランスに応じて施策を選択。

> **【図：図4-1：Lie微分分析—3つの政策の構造的インパクト比較】** *図4-1：Lie微分分析。左：Killing場（構造保存型、||L_Vg||≈0）。中：中程度の変形（段階的進出）。右：強い変形（抜本再編）。矢印＝政策ベクトル場*

本章の品質チェックポイント

▶ AIモデルの基礎から理解したい方へ

☐ チェックリスト

1. 政策ベクトル場 V は学習できたか？ 時系列データが不足 → **第9章に戻ってデータ追加。**
2. Lie括弧の結果は安定か？ 不安定 → **ベクトル場の学習エポック増加。**
3. $\mathcal{L}_V g$ の大きさはドメイン知識と整合するか？ 「穏やかな施策」なのにLie微分が巨大 → **計量（第2章）に遡って検証。**

▶ ビジネス・政策の意思決定に活用したい方へ

チームに確認： 「施策のLie微分ノルムの順序は直感と整合するか？」「穏やかなサービス改善」のLie微分が「抜本再編」より大きいなら、何かがおかしい。

第I部完了。 多様体（第1章）、計量（第2章）、接続・曲率（第3章）、Lie微分（第4章）の4つの基礎概念が揃いました。

第II部予告： 第5章では、VAE（変分オートエンコーダ）を詳しく学びます。数学者でAIを知らない方は特に重要な章です。

第II部：幾何学的AIの道具

第I部では、微分幾何学の4つの基礎概念—多様体、計量、接続・曲率、Lie微分—を学びました。第II部では、これらの数学的概念を社会データに適用するための**AIモデル**を学びます。VAE（第5章）、Vector Diffusion Mapsと計量学習（第6章）、Neural ODEと自動微分テンソル計算（第7章）、Geometric Data Scienceの全体像（第8章）の4つです。

数学に強くAIに弱い読者（数学者を含む）は、この第II部が最も重要です。 逆に、AIに強く数学に弱い読者は、第I部に戻って数学的基礎を確認してください。

第5章：VAEと滑らかな潜在多様体の構成

この章で得られるもの—コストに見合うリターン

VAE（変分オートエンコーダ）は、本書のアプローチ全体を支える最も重要なAIモデルです。高次元のビジネスデータや政策データを、低次元の「滑らかな空間」に圧縮し、その空間が数学的に正当な多様体になることを保証します。この章は、特に「微分幾何学は知っているがVAEを知らない数学者」にとって必読です。

▶ 数学の基礎から理解したい方へ

VAEは確率的生成モデルであり、観測データ $x \in \mathbb{R}^n$ と潜在変数 $z \in \mathbb{R}^d$（$d \ll n$）の同時分布 $p_\theta(x,z) = p_\theta(x|z)p(z)$ をモデル化する。エンコーダ $q_\phi(z|x)$ は変分事後分布、デコーダ $p_\theta(x|z)$ は尤度関数であり、ELBO（Evidence Lower Bound）

$$\mathcal{L}(\theta,\phi;x) = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] + \mathrm{KL}\left(q_\phi(z|x) \parallel p(z)\right)$$

の最小化により θ, ϕ を同時に学習する。

本書の文脈でVAEが決定的に重要な理由は、**デコーダ $f_\theta: \mathbb{R}^d \to \mathbb{R}^n$ が可微分多様体の構成写像を提供する**ことにある。第1章の命題1.1で示した通り、$f_\theta \in C^k$（$k \geq 3$）かつ $\mathrm{rank} J_{f_\theta}(z) = d$ かつ $f_\theta|_U$ が単射であれば、$f_\theta(U)$ は C^k 級埋め込み部分多様体となる。

数学者へのメッセージ： VAEは「確率論と最適化の組み合わせで、高次元データの低次元構造を学習する」道具です。以下では、確率的定式化の各要素を、微分幾何学の観点から何が重要かに焦点を当てて解説します。微分幾何学の観点から最も重要なのは**デコーダの滑らかさ（C^k 級性）**と**ヤコビ行列のフルランク性**であり、VAEの確率的側面（KLダイバージェンス、ELBO）は多様体構造の保証には直接関与しないが、潜在空間の「穴のなさ」（データの潜在表現が潜在空間全体に分布すること）を促進し、間接的にフルランク条件の成立を助ける。

コスト： 確率的生成モデルの枠組みの理解。変分推論、KLダイバージェンス、再パラメータ化トリックといった確率論・最適化の概念が必要。 **リターン：** 社会データから C^k 級可微分多様体を構成するための、現時点で最も実用的かつ数学的に正当化可能な方法。

▶ AIモデルの基礎から理解したい方へ

VAEを「ゼロから」理解する。

VAEは2014年にKingmaとWellingによって提案されたニューラルネットワークモデルです。名前の「変分（Variational）」は「変分推論」という統計学の手法から、「オートエンコーダ（Autoencoder）」は「データを圧縮して復元するニューラルネット」から来ています。

VAEを理解するために、まず**通常のオートエンコーダ**から始めましょう。

通常のオートエンコーダ（AE）とは。 入力データ x（たとえば30個の経営指標からなるベクトル）を、**エンコーダ**と呼ばれるニューラルネットワークで低次元の表現 z（たとえば6個の数値）に圧縮し、**デコーダ**と呼ばれるニューラルネットワークで元のデータ $\hat{x}$ に復元する。目標は $\hat{x} \approx x$（復元がなるべく元に近い）にすること。

$$x \xrightarrow{\text{エンコーダ}} z \xrightarrow{\text{デコーダ}} \hat{x} \approx x$$

エンコーダ：$\mathbb{R}^n \to \mathbb{R}^d$ （圧縮）。デコーダ：$\mathbb{R}^d \to \mathbb{R}^n$ （復元）。$d \ll n$ とすることで次元圧縮が行われます。

損失関数は再構成誤差 $\| x - \hat{x} \|^2$ の平均であり、これを最小化するようにエンコーダとデコーダのパラメータを同時に学習します。

通常のAEの問題点。 通常のAEでは、エンコーダは各入力 x に対して**1つの確定的な点** z を出力します。問題は、訓練データが潜在空間の「まばらな点」としてしか配置されないことです。訓練データの点と点の間の潜在空間には「何もない」——デコーダはその領域で学習していないため、そこに点を入力すると、意味のないデータが生成されます。

これは多様体構成にとって致命的です。多様体は**連続的で滑らかな空間**であり、すべての点（訓練データに対応する点だけでなく、点と点の間の点も）が意味を持つ必要があります。「穴だらけの潜在空間」では、多様体の連続性が保証されません。

VAEのアイデア：確率的にする。 VAEでは、エンコーダが確定的な点 z ではなく、**確率分布**（正規分布）を出力します。

具体的には、エンコーダは入力 x に対して以下を出力します。

- 平均ベクトル $\mu_\phi(x) \in \mathbb{R}^d$
- 分散ベクトル $\sigma_\phi^2(x) \in \mathbb{R}^d_{>0}$（各成分が正）

潜在表現 z は、この正規分布からサンプリングされます：

$$q_\phi(z|x) = \mathcal{N}\left(z; \mu_\phi(x), \mathrm{diag}\left(\sigma_\phi^2(x)\right)\right)$$

つまり、z の各成分 z^i は独立な正規分布 $\mathcal{N}\left(\mu_\phi^i(x), \sigma_\phi^{i2}(x)\right)$ に従います。

「確率的にする」ことの効果。 各訓練データ点は、潜在空間上の「1点」ではなく「広がり」（正規分布の裾野）として配置されます。これにより、訓練データの点と点の間の空間にもデータの潜在表現が（低い確率ながら）存在するようになり、デコーダはその領域でも学習が行われ、「穴のない」滑らかな潜在空間が実現します。

VAEの損失関数（ELBO）。

$$\mathcal{L}(\theta, \phi; x) = \underbrace{-\mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x|z)\right]}_{\text{再構成誤差}} + \underbrace{\mathrm{KL}\left(q_\phi(z|x) \parallel p(z)\right)}_{\text{KL正則化項}}$$

第1項（再構成誤差）： デコーダがデータをどれだけ正確に復元できるか。$-\log p_\theta(x|z)$ はデコーダが z から x を生成する「尤もらしさ」の負の対数。ガウスデコーダ（$p_\theta(x|z) = \mathcal{N}\left(x; f_\theta(z), \sigma^2 I\right)$）の場合、再構成誤差は $\parallel x - f_\theta(z) \parallel^2 / \left(2\sigma^2\right) + \mathrm{const}$ となり、通常のAEの再構成誤差と本質的に同じ。

第2項（KLダイバージェンス）： エンコーダが出力する事後分布 $q_\phi(z|x)$ が、事前分布 $p(z) = \mathcal{N}\left(0, I_d\right)$（標準正規分布）にどれだけ近いかを測る。KLダイバージェンスは

$$\mathrm{KL}\left(q_\phi(z|x) \parallel p(z)\right) = \frac{1}{2}\sum_{i=1}^{d}\left(\sigma_\phi^{i2}(x) + \mu_\phi^{i2}(x) - 1 - \log\sigma_\phi^{i2}(x)\right)$$

と解析的に計算でき、この項を最小化すると $\mu \to 0,\ \sigma^2 \to 1$ となり、潜在表現が原点周辺の標準正規分布に近づく。これが潜在空間の「穴のなさ」を促進する。

再パラメータ化トリック。 z は確率変数であるため、そのままでは逆伝播（勾配計算）ができない。再パラメータ化トリックは、$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$（$\varepsilon \sim \mathcal{N}(0, I)$, $\odot$ は要素ごとの積）と書き直すことで、ランダム性を ε に分離し、μ と σ に関する勾配を計算可能にする。

▶ ビジネス・政策の意思決定に活用したい方へ

VAEが行うことを「地形図作成」のたとえで説明します。

> **【図：図5-1：VAEアーキテクチャ】** *図5-1：VAEのアーキテクチャ。入力データx（30次元）→エンコーダ→潜在空間z（6次元）→デコーダf_θ（★C∞活性化関数必須）→復元データx^*

あなたの企業には、30個の経営指標（売上、利益率、市場シェア、為替レート…）の10年分の月次データがあります。各月のデータは30次元の空間の中の1点です。10年×12ヶ月＝120点が、30次元空間に散らばっています。

VAEは、この30次元のデータを6次元の「経営環境地形図」に圧縮します。圧縮の方法は以下の通りです。

Step 1（圧縮）： 30個の指標を6個の「本質的な因子」に要約する。これがエンコーダの仕事。 **Step 2（復元検証）：** 6個の因子から元の30個の指標を復元できるか確認する。復元精度が高ければ、圧縮時に重要な情報が失われていない。 **Step 3（地形図の完成）：** 復元に使う関数（デコーダ）が「地形図の描き方」を定義する。この関数が滑らかであれば、地形図も滑らかになり、曲率やLie微分が計算可能。

コスト： VAEの訓練に必要な計算資源と時間（GPUで数時間〜数日）。 **リターン：** 30次元の「見えない」データ構造が、6次元の「計算可能な地形図」に変換される。

5.1 VAEの数学的構造

▶ 数学の基礎から理解したい方へ

エンコーダ（変分事後分布）

定義5.1（エンコーダ）． エンコーダ $q_\phi(z|x)$ は、パラメータ ϕ を持つニューラルネットワークにより

$$q_\phi(z|x) = \mathcal{N}\left(z;\, \mu_\phi(x),\, \mathrm{diag}\left(\sigma_\phi^2(x)\right)\right)$$

と定義される。ここで $\mu_\phi: \mathbb{R}^n \to \mathbb{R}^d$ と $\sigma_\phi: \mathbb{R}^n \to \mathbb{R}^d_{>0}$ はニューラルネットワーク。

数学的には、エンコーダは $\mathbb{R}^n$ から $\mathbb{R}^d$ 上の正規分布の族への写像であり、データ空間の各点 x を潜在空間上の確率測度 $q_\phi(\cdot\,|x)$ に対応させる。この写像の像は、$\mathbb{R}^d$ 上の（対角共分散を持つ）正規分布の $(2d)$-パラメータ族の部分族をなす。

デコーダ（生成モデル）

定義5.2（デコーダ）． デコーダ $p_\theta(x|z)$ は、決定論的関数 $f_\theta: \mathbb{R}^d \to \mathbb{R}^n$（デコーダネットワーク）を用いて

$$p_\theta(x|z) = \mathcal{N}\left(x;\, f_\theta(z),\, \sigma_{\mathrm{dec}}^2 I_n\right)$$

と定義される（σ_{dec}^2 は固定のデコーダノイズ分散）。

本書における最重要条件。 デコーダネットワーク f_θ に対して以下を要求する：

(A) $f_\theta \in C^\infty$．すべての層で C^∞ 活性化関数（tanh, GELU, Softplus）を使用。ReLU（$C^0 \setminus C^1$）は不可。

(B) $\mathrm{rank}\, J_{f_\theta}(z) = d$ $(\forall z \in U)$．関心領域 U のすべての点でヤコビ行列がフルランク。

条件（A）は活性化関数の選択で自動的に保証される。条件（B）は訓練後に数値的に検証する必要がある（第1章のヤコビ行列SVD検証コード参照）。

命題5.1（KL正則化の限界）． KLダイバージェンス $\mathrm{KL}\left(q_\phi(z|x) \,\|\, p(z)\right)$ は、集約事後分布 $q_\phi(z) = \mathbb{E}_{p_{\mathrm{data}}(x)}\left[q_\phi(z|x)\right]$ が事前分布 $p(z) = \mathcal{N}(0, I_d)$ に近づくことを

促進する。これは潜在空間の分布的な正則性（「穴のなさ」）を保証するが、**デコーダの多様体構造（C^k 級性、フルランク性、単射性）を直接保証するものではない**。多様体構造は条件（A），（B）および単射性によって保証される。

事前分布

定義5.3（事前分布）． $p(z) = \mathcal{N}(0, I_d)$（$d$ 次元標準正規分布）。

事前分布の選択は「潜在空間の基準分布」を定めるものであり、多様体の幾何構造には直接影響しない。幾何構造はデコーダ f_θ のヤコビ行列から導かれる引き戻し計量によって決定される。

▶ AIモデルの基礎から理解したい方へ

VAEのPython実装。

```
import torch
import torch.nn as nn

class VAE(nn.Module):
    """
    変分オートエンコーダ（VAE）の完全な実装。

    重要: デコーダに nn.Tanh() を使用（C∞活性化関数）。
    nn.ReLU() は使用不可（C^0、曲率計算不能になるため）。
    """
    def __init__(self, input_dim, latent_dim, hidden_dim=128):
        super().__init__()

        # === エンコーダ ===
        # 入力データ x（input_dim次元）→ 隠れ層 → μ と log(σ²)
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Tanh(),                          # C∞ 活性化関数
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.Tanh(),
        )
        # 平均 μ を出力する層
        self.mu_layer = nn.Linear(hidden_dim // 2, latent_dim)
        # log(σ²) を出力する層（σ²ではなくlogを出力する理由:
        #   σ²は正でなければならないが、NNの出力は任意の実数。
        #   logを取ることで、exp(log σ²) = σ² > 0 を自動的に保証）
        self.logvar_layer = nn.Linear(hidden_dim // 2, latent_dim)
```

```python
        # === デコーダ ===
        # 潜在変数 z (latent_dim次元) → 隠れ層 → 復元データ x^
        # ★★★ 全層で nn.Tanh() を使用 (C∞保証) ★★★
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim // 2),
            nn.Tanh(),                          # ← ReLUではなくTanh
            nn.Linear(hidden_dim // 2, hidden_dim),
            nn.Tanh(),                          # ← ReLUではなくTanh
            nn.Linear(hidden_dim, input_dim),
            # 最終層は活性化なし (データのスケールに制約を設けない)
        )

    def encode(self, x):
        """エンコーダ: x → (μ, log σ²)"""
        h = self.encoder(x)
        mu = self.mu_layer(h)
        logvar = self.logvar_layer(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        """
        再パラメータ化トリック: z = μ + σ * ε (ε ~ N(0, I))

        なぜ必要か: z ~ N(μ, σ²I) からのサンプリングは
        「ランダムな操作」であり、μやσに関する勾配が計算できない。
        z = μ + σ*ε と書き直すと、ランダム性がεに分離され、
        zのμとσに関する勾配が計算可能になる。
        """
        std = torch.exp(0.5 * logvar)  # σ = exp(log σ²/2)
        eps = torch.randn_like(std)     # ε ~ N(0, I)
        return mu + std * eps           # z = μ + σ * ε

    def decode(self, z):
        """デコーダ: z → x^"""
        return self.decoder(z)

    def forward(self, x):
        """
        完全なフォワードパス: x → (μ, logσ²) → z → x^
```

```
        Returns:
            x_recon: 復元されたデータ
            mu: 潜在表現の平均
            logvar: 潜在表現の log(分散)
        """
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        x_recon = self.decode(z)
        return x_recon, mu, logvar


def vae_loss(x_recon, x, mu, logvar):
    """
    VAEの損失関数 (ELBO の負)

    = 再構成誤差 + KLダイバージェンス

    再構成誤差: ||x - x^||²  (データの復元精度)
    KL項: -0.5 * Σ(1 + log σ² - μ² - σ²)  (潜在分布の正則化)
    """
    # 再構成誤差 (平均二乗誤差)
    recon_loss = nn.functional.mse_loss(x_recon, x, reduction='sum')

    # KLダイバージェンス KL(N(μ, σ²I) || N(0, I))
    # = -0.5 * Σ_i (1 + log σ²_i - μ²_i - σ²_i)
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())

    return recon_loss + kl_loss
```

コードの各部分の意味：

self.encoder：30次元のデータを中間表現に圧縮するニューラルネット。nn.Tanh() は $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ を計算する活性化関数。出力は $(-1,1)$ の範囲。

self.mu_layer, self.logvar_layer：中間表現から、潜在空間の平均 μ と分散の対数 $\log\sigma^2$ を出力。$\log\sigma^2$ を出力する理由は、$\sigma^2 > 0$ を保証するため。$\exp(\log \sigma^2) = \sigma^2$ は常に正。

self.decoder：6次元の潜在表現から30次元のデータを復元。**全層で nn.Tanh() を使用**—これが本書の最重要要件。

reparameterize：再パラメータ化トリック。$z = \mu + \sigma \cdot \varepsilon$（$\varepsilon \sim \mathcal{N}(0, I)$）。

vae_loss：ELBO = 再構成誤差 + KLダイバージェンス。再構成誤差が小さいほどデータの復元が正確。KL項が小さいほど潜在分布が標準正規に近い。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—VAEで経営環境多様体を構成する

X社のデータサイエンスチームは、以下の手順でVAEを訓練しました。

入力データ： 50店舗×月次×10年分のデータから、30指標を標準化（平均0、標準偏差1に変換）。計6,000データ点×30次元。

VAEの設計： 入力次元 $n = 30$、潜在次元 $d = 6$（第1章のPCAで固有次元が約6と推定されたため）。隠れ層は128次元と64次元の2層。活性化関数は全層 tanh。

訓練： 500エポック、バッチサイズ64、学習率0.001（Adam optimizer）。

結果： - 再構成誤差：元データの分散の4.2%（基準の10%以下を満たす → ✓）- KLダイバージェンス：3.8（潜在表現が標準正規分布に近い → ✓） - ヤコビ行列の最小特異値：8.3×10^{-3}（$> 10^{-4}$ → フルランク条件 ✓）

経営者への報告： 「30個の経営指標から、6つの『本質的因子』を抽出し、6次元の経営環境地形図を構成しました。地形図の精度は96%（再構成誤差4.2%）であり、元のデータの構造をほぼ忠実に反映しています。」

ケーススタディB：国際情勢多様体の構成

安全保障分析チームは、193カ国×40指標×30年のデータ（月次追加データ含め約70,000点）からVAEを訓練。潜在次元 $d = 8$。

結果： 再構成誤差 6.1%（基準内）。ただし、データが疎な一部の国家周辺でヤコビ行列の特異値が低下。チームは「信頼度マップ」を作成し、分析結果に付記。

5.2 活性化関数と滑らかさ—なぜReLUは使えないのか

▶ 数学の基礎から理解したい方へ

デコーダ f_θ の滑らかさのクラスは、活性化関数の滑らかさで決定される。本書で使用可能/不可能な活性化関数を以下にまとめる。

活性化関数	定義	滑らかさ	使用可否	理由
ReLU	$\max(0,x)$	C^0（$x=0$で微分不連続）	✗ 不可	$g_{ij} \in C^{k-1}$, $k \geq 1$ すら非保証。曲率テンソル定義不能
Leaky ReLU	$\max(\alpha x, x)$	C^0（$x=0$で微分不連続）	✗ 不可	同上
tanh	$(\square^{\square} - \square^{-\square})/(e^x + e^{-x})$	C^∞	✓ 推奨	$k=\infty$ で全条件を満たす
GELU	$x\Phi(x)$（Φ: 標準正規CDF）	C^∞	✓ 推奨	同上。Transformerで広く使用
Softplus	$\log(1+e^x)$	C^∞	✓ 推奨	ReLUの滑らかな近似
SiLU/Swish	$x\sigma(x)$（σ: シグモイド）	C^∞	✓ 使用可	GELU に類似
ELU	$\begin{cases} x & x>0 \\ \alpha(e^x-1) & x \leq 0 \end{cases}$	C^1	△ 条件付き	C^1 だが C^2 でない。曲率に不十分

注意5.1. ReLUを使った場合のデコーダ f_θ は、入力空間 $\mathbb{R}^d$ を多面体的な領域（ポリトープ）に分割し、各領域内ではアフィン写像 $z \mapsto W_{\text{eff}} z + b_{\text{eff}}$ を行う区分的線形関数となる。領域の境界面（ReLUが 0 から正に切り替わる超平面）上では、ヤコビ行列が不連続に変化し、$g_{ij} = \left(J^\top J\right)_{ij}$ の偏微分（クリストッフェル記号の計算に必要）が定義されない。したがって、曲率テンソルは**領域の内部でのみ**定義可能であり、境界面上では定義できない。

▶ AIモデルの基礎から理解したい方へ

ReLUとtanhの違いを、コードで確認する。

【図：図5-2：活性化関数の滑らかさ比較】 *図5-2：ReLU（左上：x=0で微分不連続、曲率計算不能）とtanh（右上：どこでも滑らか、すべての高階微分が定義可能）の比較*

```
import torch
import matplotlib.pyplot as plt
import numpy as np
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

x = torch.linspace(-3, 3, 1000, requires_grad=True)

# ReLU とその微分
relu = torch.relu(x)
relu_grad = torch.autograd.grad(relu.sum(), x, create_graph=True)[0]
axes[0,0].plot(x.detach(), relu.detach(), 'b-', label='ReLU(x)')
axes[0,0].plot(x.detach(), relu_grad.detach(), 'r--', label="ReLU'(x)")
axes[0,0].set_title('ReLU: C⁰ (not differentiable at x=0)')
axes[0,0].legend(); axes[0,0].grid(True)

# tanh とその微分
tanh = torch.tanh(x)
tanh_grad = torch.autograd.grad(tanh.sum(), x, create_graph=True)[0]
axes[0,1].plot(x.detach(), tanh.detach(), 'b-', label='tanh(x)')
axes[0,1].plot(x.detach(), tanh_grad.detach(), 'r--', label="tanh'(x)")
axes[0,1].set_title('tanh: C∞ (infinitely differentiable)')
axes[0,1].legend(); axes[0,1].grid(True)

# ReLUの2階微分（定義不能を示す）
try:
    relu_grad2 = torch.autograd.grad(relu_grad.sum(), x, create_graph=True)[0]
    axes[1,0].plot(x.detach(), relu_grad2.detach(), 'g-')
except:
    axes[1,0].text(0.5, 0.5, 'Cannot compute¥n2nd derivative',
                   transform=axes[1,0].transAxes, ha='center', fontsize=14)
axes[1,0].set_title("ReLU''(x): UNDEFINED at x=0")

# tanhの2階微分（問題なく計算可能）
tanh_grad2 = torch.autograd.grad(tanh_grad.sum(), x, create_graph=True)[0]
axes[1,1].plot(x.detach(), tanh_grad2.detach(), 'g-', label="tanh''(x)")
axes[1,1].set_title("tanh''(x): well-defined everywhere")
axes[1,1].legend(); axes[1,1].grid(True)

plt.suptitle('Why tanh is required (not ReLU) for curvature computation', fontsi
ze=14)
plt.tight_layout()
plt.show()
```

このコードは、ReLUとtanhの1階微分と2階微分を比較します。ReLUの1階微分は $x = 0$ でステップ関数（0から1へのジャンプ）になり、2階微分はその点でデルタ関数（定義不能）です。tanh の微分はすべて滑らかです。

本章の品質チェックポイント

▶ AIモデルの基礎から理解したい方へ

☐ チェックリスト（第5章終了時点）

1. **活性化関数：** デコーダの全層で tanh/GELU/Softplus を使用しているか？ → ReLU使用中なら**変更して再訓練**。
2. **再構成誤差：** 元データの分散の10%以下か？ → 超えている場合、**隠れ層のサイズ増加、潜在次元の増加、訓練エポック数の増加**を検討。
3. **ヤコビ行列フルランク：** 最小特異値 $> 10^{-4}$（代表50点以上で検証）。→ 不合格なら**その領域を除外するかdを下げる**。
4. **潜在次元の適切さ：** d を1, 2, …, と増やして再構成誤差の「肘」を確認。PCAの固有次元推定と一致するか。
5. **訓練の安定性：** 異なるランダムシードで3回以上訓練し、結果（潜在表現の構造、再構成誤差）が概ね一致するか。

▶ ビジネス・政策の意思決定に活用したい方へ

チームに確認： 「VAEの再構成誤差は何%か？デコーダにtanhを使っているか？ヤコビ行列の検証は済んでいるか？」

これらに明確に回答できないチームの分析結果は信頼すべきではありません。

次章予告： 第6章ではVDM（Vector Diffusion Maps）と計量学習を学びます。VAEが「多様体を構成する」のに対し、VDMは「多様体の幾何構造を抽出する」補完的なツールです。

第6章：Vector Diffusion Mapsと計量学習

この章で得られるもの—コストに見合うリターン

VAE（第5章）が「多様体を構成する」AIモデルであるのに対し、VDM（Vector Diffusion Maps）は「離散データ点から多様体の幾何構造を抽出する」手法です。さらに、引き戻し計量とCholeskyパラメトリゼーションによる計量学習を解説します。

▶ 数学の基礎から理解したい方へ

VDMの数学的内容。 Singer & Wu（2012）のVector Diffusion Mapsは、離散データ $\{z_i\}_{i=1}^N \subset \mathbb{R}^d$ からグラフ接続ラプラシアン S_ε を構成する。ガウスカーネル $w_{ij} = \exp\left(-\| z_i - z_j \|^2/\varepsilon\right)$、局所接空間間の最適回転行列 $O_{ij} \in O(d)$ を用いて

$$(S_\varepsilon v)_i = \frac{1}{d_i}\sum_j w_{ij}\left(v_i - O_{ij}v_j\right), \quad d_i = \sum_j w_{ij}$$

連続極限定理（Singer-Wu, 2012）． 仮定A1の下で、$N \to \infty$, $\varepsilon \to 0$, $N\varepsilon^{d/2+1} \to \infty$ のとき、S_ε は M_0 の接束上の接続ラプラシアンにスペクトルの意味で収束する。

計量学習：引き戻し計量とCholeskyパラメトリゼーション。 引き戻し計量は第2章で導入済み。代替手法として、計量を直接ニューラルネットでパラメトリゼーションするCholesky法がある：$g_{ij}(z) = \left(L_\psi(z)L_\psi(z)^\top\right)_{ij}$。$L_\psi$ は下三角行列で対角に exp 適用（正値保証）。

▶ AIモデルの基礎から理解したい方へ

VDMの内部動作を段階的に説明。

(1) 各データ点の k 個の最近傍を見つけ、ガウスカーネルで重み付け。
(2) 各点の近傍データ点から、局所的な接空間（「方向」の空間）をPCAで推定。
(3) 隣接する点の接空間間の「向きのずれ」を回転行列 O_{ij} で表す。
(4) 接続ラプラシアンの固有値分解で多様体の大域的幾何構造を抽出。

```
import numpy as np
from sklearn.neighbors import NearestNeighbors

def build_vdm_laplacian(data, epsilon, k_neighbors=10):
    """VDM接続ラプラシアンの簡易実装"""
    N, d = data.shape
```

```
    nn = NearestNeighbors(n_neighbors=k_neighbors).fit(data)
    distances, indices = nn.kneighbors(data)

    # ガウスカーネル重み
    W = np.zeros((N, N))
    for i in range(N):
        for j_idx, j in enumerate(indices[i]):
            W[i, j] = np.exp(-distances[i, j_idx]**2 / epsilon)
            W[j, i] = W[i, j]  # 対称化

    D = np.diag(W.sum(axis=1))
    L = D - W  # グラフラプラシアン（スカラー版の簡略実装）

    # 固有値分解
    eigenvalues, eigenvectors = np.linalg.eigh(L)
    return eigenvalues, eigenvectors, L
```

引き戻し計量との関係。 VDMは「データ駆動型」、引き戻し計量は「モデル駆動型」の幾何構造抽出です。両者を比較し整合性を確認することで、分析の信頼性が高まります。

▶ ビジネス・政策の意思決定に活用したい方へ

VDMは「別の角度から地形図の信頼性を検証する」ツールです。VAEで作った地形図とVDMで抽出した構造が整合していれば、地形図の信頼性が高い。整合しなければ、どちらかに問題がある。

本章の品質チェックポイント

1. VDMのスペクトルギャップ $\lambda_2 - \lambda_1$ は ε を変えても安定か？
2. 引き戻し計量とVDMの幾何構造は整合するか？

第7章：Neural ODEと自動微分テンソル計算

この章で得られるもの—コストに見合うリターン

Neural ODEはデータの「変化の法則」を学習するAIモデル、自動微分は計量→曲率→Lie微分のフルチェーン計算を自動化する技術です。この2

つが揃うと、「政策シミュレーション」と「テンソル計算」がPythonで実行可能になります。

▶ 数学の基礎から理解したい方へ

Neural ODE. 常微分方程式 $dz/dt = f_\theta(z,t)$ の右辺 f_θ をニューラルネットでパラメトリゼーション。ODE solver（Runge-Kutta等）で数値解を計算。勾配は随伴法で計算：随伴状態 $a(t) = \partial L/\partial z(t)$ は $da/dt = -a^\top(\partial f_\theta/\partial z)$ を時間逆向きに解くことで得られ、これはポントリャーギンの最大原理の離散アナログ。

自動微分テンソル計算。 PyTorchの autograd（create_graph=True）により、計量 g_{ij} → クリストッフェル記号 Γ^k_{ij} → 曲率テンソル $R^l{}_{ijk}$ → Lie微分 $\mathcal{L}_V g$ のフルチェーン計算が実行可能。

▶ AIモデルの基礎から理解したい方へ

Neural ODEを「ゼロから」理解する。

通常のニューラルネット（ResNetなど）では、データは離散的な「層」を順番に通過します：$h_{t+1} = h_t + f_\theta(h_t, t)$。Neural ODEは、この層の厚みを無限に薄くした極限として、連続的な微分方程式 $dz/dt = f_\theta(z,t)$ を考えます。

```
import torch
import torch.nn as nn
from torchdiffeq import odeint

class NeuralODEFunc(nn.Module):
    """Neural ODEの速度場（右辺）をニューラルネットで定義"""
    def __init__(self, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(dim, 64), nn.Tanh(),   # C∞活性化
            nn.Linear(64, 64), nn.Tanh(),
            nn.Linear(64, dim))

    def forward(self, t, z):
        return self.net(z)

# 使用例
func = NeuralODEFunc(dim=6)  # 6次元（X社の潜在空間次元）
z0 = torch.randn(1, 6)       # 初期状態
t_span = torch.linspace(0, 5, 100)  # 時間区間
```

```
trajectory = odeint(func, z0, t_span)  # ODE解を計算
print(f"Trajectory shape: {trajectory.shape}")  # [100, 1, 6]
```

自動微分によるフルチェーン計算のまとめ。

```
import torch

def full_geometry_chain(metric_fn, z, d=2):
    """計量からスカラー曲率まで一気に計算するフルチェーン"""
    z = z.clone().requires_grad_(True)

    # Step 1: 計量テンソル g_{ij}
    g = metric_fn(z)
    g_inv = torch.inverse(g)

    # Step 2: 計量の偏微分 ∂g_{jl}/∂z^i
    dg = torch.zeros(d, d, d)
    for j in range(d):
        for l in range(d):
            grad = torch.autograd.grad(g[j,l], z, retain_graph=True, create_grap
h=True)[0]
            dg[:, j, l] = grad

    # Step 3: クリストッフェル記号 Γ^k_{ij}
    Gamma = torch.zeros(d, d, d)
    for k in range(d):
        for i in range(d):
            for j in range(d):
                s = 0.0
                for l in range(d):
                    s += g_inv[k,l] * (dg[i,j,l] + dg[j,i,l] - dg[l,i,j])
                Gamma[k,i,j] = 0.5 * s

    print(f"det(g) = {torch.det(g).item():.4f}")
    print(f"Γ^0_{{01}} = {Gamma[0,0,1].item():.4f}")
    return g, Gamma
```

▶ ビジネス・政策の意思決定に活用したい方へ

Neural ODEは「経営環境の変化法則をデータから自動的に発見する」ツールです。「売上が増えると利益率はどう動くか」「為替が変動すると原材料コストは

どう反応するか」—これらの**変化の法則**を、過去のデータから自動的に学習します。

自動微分テンソル計算は、これまで数学者が何週間もかけて手計算していた曲率やLie微分を、コンピュータが秒単位で正確に計算する技術です。

本章の品質チェックポイント

1. Neural ODEの軌道は物理的に妥当か（発散しない、値域が合理的）？
2. 自動微分でNaN/Infが出ていないか？
3. 有限差分との比較で自動微分の結果が整合するか？

第8章：Geometric Data Scienceの全体像

この章で得られるもの—コストに見合うリターン

本書のアプローチ（多様体微分解析）は、**Geometric Data Science（幾何学的データサイエンス）**と呼ばれる広大な分野のほんの一部です。本章では、この分野の全体像を俯瞰し、本書のアプローチがどこに位置するのかを明らかにします。自分が今いる場所と、今後探索できる地図の全体を把握してください。

▶ 数学の基礎から理解したい方へ

Geometric Data Scienceは、多様体（manifold）、グラフ（graph）、群・対称性（symmetry）、トポロジー（topology）、層（sheaf）、圏（category）といった幾何学的・代数的概念を活用し、データの本質的な構造を反映した分析を行う分野の総称である。本章では、8つの主要アプローチを体系的に解説し、各アプローチの数学的基盤と本書との関係を明らかにする。

▶ AIモデルの基礎から理解したい方へ

Geometric Data Scienceには多様な手法が存在します。本章はそれらの「地図」を提供します。GNN、TDA、Equivariant Networks、Poincaré Embeddings、Sheaf Neural Networks—これらの名前を聞いたことがあっても、全体像の中でどこに位置するか把握していない方は多いはずです。

▶ ビジネス・政策の意思決定に活用したい方へ

本書の手法は万能ではありません。データの性質や分析目的に応じて、最適なGeometric Data Scienceの手法は異なります。本章を読むことで、「自分の問題に最適な手法はどれか」を判断するための基盤が得られます。

Geometric Data Scienceの全体像

Geometric Data Science（幾何学的データサイエンス）は、データの形状・構造・空間的関係を数学的に捉え、それをAIや統計学に組み込むアプローチの総称です。

通常の統計学や機械学習は、データがユークリッド空間 $\mathbb{R}^n$ 上に存在することを暗黙に仮定しています。しかし、現実のデータの多くは—画像、分子構造、ソーシャルネットワーク、時系列、確率分布—ユークリッド空間には収まりません。

以下の図は、Geometric Data Scienceの主要な8つのアプローチを俯瞰したものです。本書のアプローチ（★印）はこの中の「1. 多様体・リーマン幾何」に位置します。

図8-1：Geometric Data Scienceの8つの主要アプローチ。本書のアプローチ（★本書Step 4-9）は「1. 多様体・リーマン幾何」に位置する。

---

1. 多様体・リーマン幾何学ベースのモデル ★本書のメインアプローチ

▶ 数学の基礎から理解したい方へ

データが非線形空間に分布している場合、その空間のリーマン幾何学的構造を考慮して次元削減やクラスタリングを行う手法群。

Riemannian PCA（Principal Geodesic Analysis, PGA）： 通常のPCAは、データの分散を最大化する方向（直線）を見つける。PGAは多様体上でこれを一般化し、分散を最大化する**測地線の方向**を見つける。フレシェ平均と測地的主成分を用いて、多様体上で意味のある統計量を計算する。

測地距離埋め込み： データ点間のユークリッド距離の代わりに、多様体上の測地距離を用いて低次元埋め込みを行う。Isomapが代表例。

リーマン勾配降下法： パラメータ空間が多様体（$SO(3)$、確率単体等）上にある場合の最適化。ユークリッド勾配 ∇f の代わりにリーマン勾配 $\mathrm{grad} f$ を計算し、指数写像 $z_{t+1} = \mathrm{Exp}_{z_t}\left(-\eta \cdot \mathrm{grad} f(z_t)\right)$ で多様体に沿って更新する。

▶ **本書との関係：** 本書のStep 4〜9は、まさにこのカテゴリに属する。引き戻し計量の学習（Step 4）、クリストッフェル記号の計算（Step 5）、測地線の算出（Step 9）はすべてリーマン幾何学ベースの手法。さらに本書は、これをデータ分析だけでなく**制御シミュレーション**にまで拡張している点が特徴的。

---

2. グラフ・ネットワークベースのモデル

▶ 数学の基礎から理解したい方へ

GCN（Graph Convolutional Network）： 画像におけるCNNのグラフ版。各ノードの特徴量を、隣接ノードの特徴量で畳み込み（集約）して更新する。

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \frac{1}{\sqrt{d_i d_j}} W^{(l)} h_j^{(l)}\right)$$

ここで $h_i^{(l)}$ はノード i の第 l 層の特徴量、$\mathcal{N}(i)$ は隣接ノード集合、d_i は次数、$W^{(l)}$ は学習パラメータ、σ は活性化関数。

▶ AIモデルの基礎から理解したい方へ

GCNの数式を「日常言語」で翻訳すると：「友達の情報を集めて、自分の情報を更新する」操作です。σ は活性化関数（非線形性の導入）、$W^{(l)}$ はどの情報を重視するかの重み（学習される）、$1/\sqrt{d_i d_j}$ は正規化項（友達が多いノードの影響を適度に抑制）。

GAT（Graph Attention Network）： 隣接ノードへの重み付けを、注意機構（attention mechanism）で動的に決定。「重要な隣人」に大きな重みを与える。

GraphSAGE： 大規模グラフ（数百万ノード）に対応。全隣接ノードではなくサンプリングした隣接ノードの情報を集約。

応用例： 分子の構造から物性予測、ソーシャルネットワークのコミュニティ検出、サプライチェーンのボトルネック予測、推薦システム。

▶ **本書との関係：** 本書のStep 3（VDM）はデータ点間のグラフを構成し、接続ラプラシアンを計算する。GNNがグラフ上での学習を目的とするのに対し、本書はグラフを**連続多様体への橋渡し**として用いる。

---

3. トポロジカルデータ解析（TDA）

▶ 数学の基礎から理解したい方へ

パーシステントホモロジー（Persistent Homology）： スケールパラメータ ε を動かしながらデータの位相構造（穴、ループ、空洞）を追跡する。結果はバーコード図（各位相的特徴の「誕生」と「消滅」のスケール）として可視化。

Mapperアルゴリズム： 高次元データを低次元に射影し、被覆（cover）とクラスタリングを組み合わせてデータの「骨格」を構成する。

▶ **本書との関係：** TDAは本書の多様体微分解析と**相補的**。TDAがデータの位相構造（穴の数）を定性的に捉えるのに対し、本書は微分構造（曲率、ベクトル場）を定量的に扱う。**制御シミュレーションは本書の手法でのみ可能。**

---

4. 群・対称性を考慮したモデル

▶ 数学の基礎から理解したい方へ

群 G がデータに作用するとき、ネットワーク f が以下を満たすように設計：

- **等変性（equivariance）：** $f(g \cdot x) = g \cdot f(x)$
- **不変性（invariance）：** $f(g \cdot x) = f(x)$

SE(3)-Equivariant Networks： 3次元の回転・平行移動に対して等変。分子の3D構造から物性を予測する際に、分子をどの向きで入力しても同じ結果を出力。AlphaFoldの基盤技術の一つ。

▶ **本書との関係：** Lie群はLie微分（Step 7）と深く関連。Equivariant Networksの「対称性の保存」は、本書のKilling方程式（$\mathcal{L}_V g = 0$）と同じ数学的精神。

---

5. 非ユークリッド空間の埋め込みモデル

▶ **数学の基礎から理解したい方へ**

Poincaré Embeddings (Nickel & Kiela, 2017) ： ポアンカレ円板モデル（双曲空間の一つのモデル）上に、単語や概念の階層関係を埋め込む。双曲空間の体積は指数関数的に増えるため、木構造との相性が抜群。WordNetのような概念階層を、わずか2次元の双曲空間で効率的に表現。ユークリッド空間では数百次元が必要な表現が、双曲空間では5〜10次元で十分。

球面・トーラス埋め込み： 周期的なデータを球面 S^n やトーラス T^n 上に埋め込む。

▶ **本書との関係：** 本書ではVAEの潜在空間を $\mathbb{R}^d$ としているが、潜在空間を双曲空間やSPD多様体に設定するVAEの研究も進んでいる。

---

6. Geometric Deep Learning（統一枠組み）

▶ **数学の基礎から理解したい方へ**

Bronstein et al. (2021) が提唱した統一枠組み。上記のすべてを「データが存在するドメイン（グラフ、多様体、群、etc.）の構造を活用した深層学習」として統一的に捉える。PointNet（点群学習）、MeshCNN（メッシュ学習）、Spectral Graph CNN（スペクトルグラフ畳み込み）、Riemannian VAE等を包含。

▶ **本書との関係：** 本書のStep 2（VAE）とStep 4（計量学習）は、Bronsteinの枠組みでは「多様体ドメイン上の信号処理」として位置づけられる。

---

7. 情報幾何学

▶ **数学の基礎から理解したい方へ**

情報幾何学（Amari, 1985）は、確率分布の族をパラメータ空間上のリーマン多様体として捉える。**フィッシャー情報行列**

$$g_{ij}(\theta) = \mathbb{E}\left[\frac{\partial \log p(x;\theta)}{\partial \theta^i}\frac{\partial \log p(x;\theta)}{\partial \theta^j}\right]$$

をリーマン計量として導入。これは「パラメータを少し変えたとき、確率分布がどれだけ変わるか」を測る指標であり、本書の引き戻し計量 $g_{ij}(z) = \left(J^\top J\right)_{ij}$ と**数学的に同じ構造**を持つ。

▶ AIモデルの基礎から理解したい方へ

フィッシャー情報行列と引き戻し計量の対応関係：

情報幾何学	本書
パラメータ θ を少し変える	潜在変数 z を少し変える
確率分布 $p(x;\theta)$ がどれだけ変わるか	デコーダ出力 $f_\theta(z)$ がどれだけ変わるか
フィッシャー情報行列 $g_{ij}(\theta)$	引き戻し計量 $g_{ij}(z)$

どちらも**変化の感度を計量として定義する**という同じ数学的構造。

自然勾配法（Natural Gradient Descent）： ユークリッド勾配 ∇L の代わりに、フィッシャー情報行列 G で補正した勾配 $G^{-1}\nabla L$ で更新。パラメータ空間の「歪み」を考慮するため、通常の勾配降下法よりも効率的に収束。TRPO（Trust Region Policy Optimization）はこの応用。

$$\theta_{t+1} = \theta_t - \eta \cdot G(\theta_t)^{-1}\nabla L(\theta_t)$$

---

8. その他の幾何学的モデル

▶ 数学の基礎から理解したい方へ

モース理論： 多様体上の滑らかな関数の臨界点（極大・極小・鞍点）から多様体の位相構造を読み取る。データの密度関数をモース関数と見なし、臨界点からクラスタや異常値を検出。

Ollivier-リッチ曲率： グラフ上で離散的にリッチ曲率を定義。コミュニティ構造の検出やネットワークのボトルネック特定に用いる。

Sheaf Neural Networks（Hansen & Gebhart, 2020; Bodnar et al., 2022）： 代数幾何学の**層（Sheaf）理論**をGNNに応用。各ノードに固有の「ローカル情報空間」を持たせ、エッジごとに**制限写像（翻訳行列）**を学習。Sheaf Laplacian

$$(L_{\mathcal{F}}\mathbf{x})_v = \sum_{e=(u,v)\in E} \mathcal{F}_{v\leftarrow e}^\top \left(\mathcal{F}_{v\leftarrow e}x_v - \mathcal{F}_{u\leftarrow e}x_u\right)$$

により、通常のGNNの過剰平滑化問題とヘテロフィリーへの弱さを劇的に改善（精度50-60% → 80-90%の事例あり）。

層理論とデータサイエンスの将来。 Sheaf Neural Networkは、**抽象代数幾何学の概念が実用的な機械学習の性能を劇的に改善する**という、数学とAIの驚くべき接点。さらに圏論的機械学習への発展も進んでいる。

```
import torch
import torch.nn as nn

class SheafDiffusion(nn.Module):
    """Sheaf Neural Networkの簡略化された実装"""
    def __init__(self, n_features, n_edge_features, n_edges):
        super().__init__()
        self.restriction_maps = nn.ParameterList([
            nn.Parameter(torch.randn(n_edge_features, n_features))
            for _ in range(n_edges * 2)
        ])

    def sheaf_laplacian(self, x, edge_index):
        """Sheaf Laplacian を計算"""
        out = torch.zeros_like(x)
        for idx, (u, v) in enumerate(edge_index.T):
            F_v = self.restriction_maps[2 * idx]
            F_u = self.restriction_maps[2 * idx + 1]
            diff = F_v @ x[v] - F_u @ x[u]
            out[v] += F_v.T @ diff
        return out

    def forward(self, x, edge_index):
        Lx = self.sheaf_laplacian(x, edge_index)
        return x - 0.1 * Lx
```

▶ ビジネス・政策の意思決定に活用したい方へ

どの手法を使うべきか—判断の指針

あなたのデータの性質	最適な手法	本書との関係
連続的な指標が多数（経営指標、経済指標）	**本書の手法**（多様体微分解析）	まさに本書

あなたのデータの性質	最適な手法	本書との関係
ネットワーク構造が重要（サプライチェーン、同盟関係）	GNN（GCN, GAT）	本書のVDMと補完的
「穴」や「ループ」の構造を調べたい	TDA（パーシステントホモロジー）	本書と相補的
階層構造のデータ（組織図、分類体系）	Poincaré Embeddings	本書の拡張可能
物理的対称性が重要（分子、3D形状）	Equivariant NN	本書のKilling場と同精神

本章の品質チェックポイント

この章には品質チェックポイントはありません。 本章は概観章であり、計算を伴わないためです。ただし、本章の内容を踏まえて、「自分の問題に本書の手法が適切かどうか」を改めて判断してください。もしデータの主要な構造がグラフ的（ネットワーク的）であり、連続的な多様体構造よりもノード間の関係性が重要であるなら、GNNベースのアプローチの方が適切かもしれません。

第II部完了。 幾何学的AIの4つの道具と、Geometric Data Scienceの全体像が揃いました。

第III部予告： 第9章から、いよいよ実践に入ります。

第III部：社会データからの多様体構築と分析

図9-1：Geometric Intelligenceの10ステップパイプライン。各ステップに品質チェックポイントがあり、不合格の場合はStep 1に戻る

第I部で数学の基礎を、第II部で幾何学的AIの道具を学びました。第III部ではいよいよ実践に入ります。ケーススタディA（X社の経営環境多様体）とケーススタディB（国際情勢多様体）を、具体的なPythonコードとともに実行します。

第9章：データ収集とVAEによる多様体構成（Step 1-2）

この章で得られるもの—コストに見合うリターン

この章は、実践の出発点です。生のビジネスデータや政策データを、「計算可能な多様体」に変換する最初のステップ—データ収集・前処理（Step 1）とVAEによる潜在多様体の構成（Step 2）—を実行します。**この2ステップの品質が、以降のすべての分析の信頼性を決定します。**

▶ 数学の基礎から理解したい方へ

Step 1 は、観測データ $X = \{x_i\}_{i=1}^{N} \subset \mathbb{R}^n$ を数値行列として整備する前処理段階。欠損値処理（多重代入法、k-NN補完）、カテゴリ変数の数値化（one-hot, entity embedding）、標準化 $\tilde{x}^{(j)} = \left(x^{(j)} - \mu_j\right)/\sigma_j$、時系列平滑化（カーネル平滑化）を行う。

Step 2 は、仮定A1（多様体仮説）とA2（デコーダ正則性）の下で、VAEのデコーダ $f_\theta: U \subset \mathbb{R}^d \to \mathbb{R}^n$ が C^k 級（$k \geq 3$）埋め込み部分多様体 $f_\theta(U) \subset \mathbb{R}^n$ を構成することを保証する段階。命題1.1の3条件（C^k 級、フルランク、単射）の検証を含む。

コスト： データ収集・整備の実務的労力、VAEの設計・訓練・検証の計算時間。
リターン： 以降のStep 3〜10のすべての計算の「舞台」となる可微分多様体の構成。この舞台が不適切であれば、曲率もLie微分も最適制御も、すべて意味を失う。

▶ AIモデルの基礎から理解したい方へ

Step 1の前処理は通常のデータサイエンスの手順です。Step 2のVAE構成は第5章で詳しく学んだ内容の実践です。本章では、第5章の知識を使って、実際のデータに対してVAEを訓練し、命題1.1の条件を検証する完全なパイプラインを示します。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社が集めるデータセット

X社のデータサイエンスチームは以下のデータを収集しました。

自社データ（50店舗×月次×10年）： 売上高、客単価、来店数、在庫回転率、従業員数、顧客満足度、リピート率、客層比率、商品カテゴリ別売上構成比、店舗面積、営業時間、広告宣伝費（計15指標）。

マクロ経済データ（月次×10年）： GDP成長率、インフレ率（CPI）、失業率、消費者信頼感指数、為替レート（USD/JPY）、金利（10年国債利回り）、原油価格、鉱工業生産指数（計8指標）。

業界データ（月次×10年）： 市場規模推定値、主要競合3社の売上推定値、原材料価格指数、サプライチェーン遅延指数、特許出願数、業界平均利益率（計7指標）。

合計30指標×6,000データ点。

ケーススタディB：安全保障チームが集めるデータセット

軍事指標（193カ国×年次×30年）： 軍事費/GDP比、核弾頭数、現役兵力数、主要装備数、サイバー能力指数、宇宙能力指数（計6指標）。

外交指標： 同盟関係ネットワーク（二国間同盟の有無と強度）、国連総会投票一致度行列、二国間条約数、外交官交換数、首脳会談頻度（計5指標）。

経済安全保障指標： 二国間貿易額/GDP比、エネルギー輸入依存度、レアアース調達先集中度、半導体サプライチェーン依存度、金融制裁脆弱性指標（ドル建て取引依存度）（計5指標）。

社会・政治指標： 民主主義指数、報道の自由度、政権安定性指数、国内世論の対外政策支持率（計4指標）。

合計40指標。年次データ5,790点＋月次貿易金融データ追加で約70,000点。

Step 1の実装

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.impute import KNNImputer

def preprocess_social_data(df):
    """
    Step 1: 社会データの前処理パイプライン
```

```
    1. KNN補完で欠損値を処理
    2. 標準化（平均0、標準偏差1）
    """
    # 1. 欠損値をKNN補完
    imputer = KNNImputer(n_neighbors=5)
    df_numeric = df.select_dtypes(include=[np.number])
    df_imputed = pd.DataFrame(
        imputer.fit_transform(df_numeric),
        columns=df_numeric.columns
    )

    # 2. 標準化
    scaler = StandardScaler()
    X = scaler.fit_transform(df_imputed)

    return X, scaler

# 使用例
df = pd.DataFrame(np.random.randn(6000, 30))  # X社のデータ（模擬）
df.iloc[5, 3] = np.nan  # 欠損値の例
X, scaler = preprocess_social_data(df)
print(f"Preprocessed: shape={X.shape}, mean={X.mean():.4f}, std={X.std():.4f}")
```

Step 2の実装：VAE訓練とヤコビ行列検証

```
import torch
import torch.nn as nn

# VAE定義（第5章のコードを再掲）
class VAE(nn.Module):
    def __init__(self, input_dim=30, latent_dim=6, hidden_dim=128):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim//2), nn.Tanh())
        self.mu_layer = nn.Linear(hidden_dim//2, latent_dim)
        self.logvar_layer = nn.Linear(hidden_dim//2, latent_dim)
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim//2), nn.Tanh(),
            nn.Linear(hidden_dim//2, hidden_dim), nn.Tanh(),
            nn.Linear(hidden_dim, input_dim))

    def encode(self, x):
```

```
        h = self.encoder(x)
        return self.mu_layer(h), self.logvar_layer(h)

    def reparameterize(self, mu, logvar):
        return mu + torch.exp(0.5*logvar) * torch.randn_like(mu)

    def decode(self, z): return self.decoder(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

def verify_jacobian_rank(decoder, z_samples, latent_dim):
    """Step 2の核心：ヤコビ行列のフルランク検証"""
    min_sv = float('inf')
    for z in z_samples:
        z = z.requires_grad_(True)
        f = decoder(z)
        J = torch.zeros(f.shape[0], z.shape[0])
        for i in range(f.shape[0]):
            grad = torch.autograd.grad(f[i], z, retain_graph=True)[0]
            J[i] = grad
        svs = torch.linalg.svdvals(J)
        min_sv = min(min_sv, svs[-1].item())

    print(f"Minimum singular value: {min_sv:.6f}")
    if min_sv > 1e-4:
        print("✓ Full rank condition satisfied (Assumption A2(b))")
    else:
        print("✗ WARNING: Jacobian may be rank-deficient")
    return min_sv
```

本章の品質チェックポイント—先に進むべきか、戻るべきか？

▶ 全読者共通

これは本書全体で最も重要なチェックポイントです。 ここでの合否が、以降6章分の分析の信頼性を決定します。

1. **PCAで固有次元を確認したか？** → d の選択根拠。
2. **データ量は** $N > 100 \times 2^d$ **を満たすか？** → 不足なら追加データ収集。
3. **VAEの再構成誤差は分散の10%以下か？** → 超えるならアーキテクチャ変更。
4. **ヤコビ行列の最小特異値** $> 10^{-4}$ **か？** → 不合格なら d を下げるか領域除外。
5. **活性化関数は** tanh/GELU/Softplus **か？** → ReLU使用中なら変更必須。

1つでも不合格の場合、先に進まないでください。 このステップをやり直してください。

第10章：幾何構造抽出とリーマン計量の学習（Step 3-4）

この章で得られるもの—コストに見合うリターン

Step 3（VDMによる幾何構造抽出）とStep 4（引き戻し計量の計算）を実行します。多様体に「距離」が定義され、経営状態間・国際情勢間の定量的な距離比較が可能になります。

▶ 数学の基礎から理解したい方へ

Step 3では VDM（第6章）を VAE潜在空間に適用し、接続ラプラシアンのスペクトルから多様体の大域的幾何構造を抽出する。Step 4では引き戻し計量 $g_{ij}(z) = \left(J_{f_\theta}^{\top} J_{f_\theta}\right)_{ij}$ を潜在空間の格子点上で計算する。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—「距離」の定義

「新興国市場と国内市場のリーマン距離は5.8。欧州市場との距離は2.1。」この数値により、「どの市場が構造的に最も移行しやすいか」が定量的に判断可能に。

```
import torch
import numpy as np

def compute_metric_field(decoder, grid_points, d=2):
```

```
    """Step 4: 格子点上で引き戻し計量を計算"""
    metrics = []
    for z in grid_points:
        z_t = torch.tensor(z, dtype=torch.float32).requires_grad_(True)
        f = decoder(z_t)
        J = torch.zeros(f.shape[0], d)
        for a in range(f.shape[0]):
            grad = torch.autograd.grad(f[a], z_t, retain_graph=True, create_grap
h=True)[0]
            J[a] = grad
        g = (J.T @ J).detach().numpy()
        metrics.append(g)
    return np.array(metrics)

print("Metric field computation ready")
```

本章の品質チェックポイント

1. **計量は全格子点で正定値か？** → 全固有値が正であることを確認。
2. **条件数は** 10^4 **以下か？** → 超える領域は信頼性低。
3. **リーマン距離は業界の直感と整合するか？** → 非現実的なら第9章に戻る。

第11章：曲率テンソルの計算と経営環境・国際情勢の構造分析（Step 5-6）

この章で得られるもの—コストに見合うリターン

計量からクリストッフェル記号（Step 5）と曲率テンソル（Step 6）を計算します。スカラー曲率マップにより、「安定領域」と「不安定領域」が可視化されます。**これは本書のアプローチが提供する最も独自なリターンの1つです。**

▶ 数学の基礎から理解したい方へ

Step 5. クリストッフェル記号 $\Gamma_{ij}^{k}(z) = \frac{1}{2} g^{kl}\left(\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij}\right)$ を自動微分で計算。

Step 6. リーマン曲率テンソル $R^l{}_{ijk} = \partial_j \Gamma^l_{ik} - \partial_i \Gamma^l_{jk} + \Gamma^l_{jm}\Gamma^m_{ik} - \Gamma^l_{im}\Gamma^m_{jk}$。リッチテンソル $\mathrm{Ric}_{ij} = R^l{}_{ilj}$。スカラー曲率 $\mathrm{Scal} = g^{ij}\mathrm{Ric}_{ij}$。

:::message alert **滑らかさの連鎖：** 曲率テンソル計算には $g_{ij} \in C^2 \to f_\theta \in C^3$ が必要。ReLUは使用不可。 :::

```
import torch

def riemann_curvature(Gamma_fn, z, d):
    """Step 6: リーマン曲率テンソルを自動微分で計算"""
    # Γの微分が必要なため、create_graph=Trueで計算
    #  (完全な実装は第7章のコードを拡張)
    print("Curvature tensor computation (requires C³ decoder)")
    return None  # Full implementation: see Chapter 7 extended
```

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—スカラー曲率マップ

チームは潜在空間の格子点上でスカラー曲率を計算し、カラーマップとして経営会議に提示。

発見： 国内市場中心部は正の曲率（安定）。国内市場と新興国市場の境界にスカラー曲率 -2.3 の不安定領域。過去のリーマンショック・COVID-19時の経営状態がこの領域に位置することを確認。

経営者への示唆： 新興国進出経路がこの不安定領域を通過する場合、リスクバッファ（予備資金、段階的投資、撤退条件）が不可欠。

ケーススタディB：国際情勢の構造的脆弱点

「軍事費急増＋同盟弱体化＋貿易依存度非対称化」が同時に進行する領域で強い負の曲率。**この領域は国際秩序の構造的脆弱点であり、現在の国際状態からの距離がリスク評価の核心指標。**

```
import plotly.graph_objects as go
import numpy as np

def visualize_curvature_map(z_grid, scalar_curvature, title="Scalar Curvature Map"):
    """Step 6の結果を可視化: スカラー曲率マップ"""
    fig = go.Figure(data=go.Heatmap(
        z=scalar_curvature,
```

```
        colorscale='RdBu_r',
        zmid=0))
    fig.update_layout(title=title, xaxis_title='z¹', yaxis_title='z²')
    fig.show()

print("Visualization tools ready")
```

本章の品質チェックポイント

1. **曲率計算でNaN/Infが出ていないか？** → 出た場合は計量の条件数を確認。
2. **曲率パターンはランダムシード変更で安定か？** → 不安定なら曲率に基づく判断は控える。
3. **不安定領域は過去の既知危機と整合するか？** → 整合しなければ第9章に戻ることを検討。

第12章：Lie微分による政策・戦略の影響分析（Step 7）

この章で得られるもの—コストに見合うリターン

この章は、経営者・政策立案者にとってのクライマックスです。 Lie微分の計算により、「この施策は経営環境の構造を保存するか、変形するか」「2つの施策は干渉するか」が数値で示されます。

▶ 数学の基礎から理解したい方へ

Step 7. 政策ベクトル場 $V \in \mathfrak{X}(M)$ の計量に対するLie微分

$$(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$$

を計算する。$\mathcal{L}_V g = 0$（Killing方程式）なら構造保存型、$\mathcal{L}_V g \neq 0$ なら構造変形型。2つのベクトル場のLie括弧 $[V_A, V_B]$ で干渉を判定。

▶ AIモデルの基礎から理解したい方へ

政策ベクトル場 V は時系列データの差分 $V(z) \approx \Delta z/\Delta t$ からニューラルネットで学習・推定。Lie微分はautogradでヤコビ行列を計算し、第4章のコードを適用。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—3つの施策の構造的インパクト

施策	$\lVert \mathcal{L}_V g \rVert$	分類	経営判断への示唆
既存顧客サービス強化	0.12	構造保存型	低リスク・低リターン。安定期に適切。
新興国市場段階的進出	1.8	構造変形型（中）	中リスク・中リターン。段階的管理が可能。
事業ポートフォリオ抜本再編	4.5	構造変形型（強）	高リスク・高リターン。十分な準備と覚悟が必要。

Lie括弧分析： $[V_{\text{cost}}, V_{\text{market}}]$ が移行途上の領域で大きい → 「コスト削減と市場開拓を同時進行するな。順序を分けよ。」

ケーススタディB：安全保障政策の構造的影響

同盟強化政策 V_{alliance} のLie微分ノルムは1.2（中程度の構造変形）。経済制裁政策 V_{sanction} は2.8（強い構造変形）。

$[V_{\text{sanction}}, V_{\text{alliance}}]$ が特定の緊張状態で大きい → **「まず同盟を強化し、その後に制裁を発動する」順序が最適。**

```python
# Step 7: ベクトル場のLie微分と計量のLie微分の計算
#  (第4章のコードを実データに適用)

import torch

def compute_lie_derivative_norm(V_net, metric_fn, z_grid, Gamma_fn, d):
    """格子点上でLie微分のノルムを計算し、マップとして返す"""
    norms = []
    for z in z_grid:
        z_t = torch.tensor(z, dtype=torch.float32)
        Gamma = Gamma_fn(z_t)
        L_V_g = lie_derivative_metric(V_net, metric_fn, z_t, Gamma, d)
        norm = torch.norm(L_V_g).item()
        norms.append(norm)
    return norms

print("Lie derivative computation ready")
```

本章の品質チェックポイント

1. **ベクトル場の学習は安定か？** → 複数回訓練して一致確認。
2. **Lie微分のノルムの順序はドメイン知識と整合するか？** → 不整合なら計量（第10章）に遡る。
3. **Lie括弧の干渉パターンは安定か？** → 不安定なら干渉に基づく判断は控える。

第13章：制御シミュレーションと最適パスの算出（Step 8-9）

この章で得られるもの—コストに見合うリターン

多様体上の制御系を定式化し（Step 8）、最適パスを計算します（Step 9）。「現在の経営状態から目標状態への、最小リスクの移行経路」が算出されます。

▶ 数学の基礎から理解したい方へ

Step 8. アフィン制御系：

$$\dot{z}^k(t) = F^k\big(z(t)\big) + \sum_{\alpha=1}^{m} u_\alpha(t) B_\alpha^k\big(z(t)\big)$$

F：ドリフト（自然な変化）。B_α：制御ベクトル場（施策 α の効果）。u_α：制御入力（施策の強度）。F, B_α はNeural ODEでパラメトリゼーション。

Step 9. 測地線方程式：

$$\ddot{\gamma}^k + \Gamma_{ij}^k \dot{\gamma}^i \dot{\gamma}^j = 0$$

ポントリャーギンの最大原理：ハミルトニアン $H(z,p,u) = p_k(F^k + u_\alpha B_\alpha^k) - \ell(z,u)$、正準方程式 $\dot{z}^k = \partial H / \partial p_k$, $\dot{p}_k = -\partial H / \partial z^k$ から最適制御 $u^*(t)$ と最適経路 $z^*(t)$ を計算。

▶ AIモデルの基礎から理解したい方へ

Neural ODEで F と B_α を学習し、`torchdiffeq.odeint` で数値解を計算。

```python
import torch
import torch.nn as nn
from torchdiffeq import odeint

class ManifoldControlODE(nn.Module):
    """Step 8: 多様体上のアフィン制御系"""
    def __init__(self, dim, n_controls):
        super().__init__()
        self.drift = nn.Sequential(
            nn.Linear(dim, 32), nn.Tanh(), nn.Linear(32, dim))
        self.control_fields = nn.ModuleList([
            nn.Sequential(nn.Linear(dim, 32), nn.Tanh(), nn.Linear(32, dim))
            for _ in range(n_controls)])

    def forward(self, t, z, u=None):
        dz = self.drift(z)
        if u is not None:
            for alpha, B in enumerate(self.control_fields):
                dz = dz + u[alpha] * B(z)
        return dz

def geodesic_ode(Gamma_fn):
    """Step 9: 測地線方程式をODEとして定式化"""
    def func(t, state):
        d = state.shape[0] // 2
        z, v = state[:d], state[d:]
        Gamma = Gamma_fn(z)
        dz = v
        dv = torch.zeros(d)
        for k in range(d):
            for i in range(d):
                for j in range(d):
                    dv[k] -= Gamma[k, i, j] * v[i] * v[j]
        return torch.cat([dz, dv])
    return func

print("Manifold control ODE and geodesic formulation ready")
```

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—新興国市場への最適移行経路

チームは3つの経路を計算しました。

> **【図：図13-1：3つの移行経路の比較】** *図13-1：現在の経営状態から目標状態への3つの経路。黒破線＝直線的計画（高リスク）、緑＝測地線（エネルギー最小）、マゼンタ＝制御付き最適経路（推奨）*

経路1（直線的計画）： ユークリッド距離で最短。しかし、曲率負の不安定領域を通過。リーマンショック級の危機が発生した場合、経路上で経営が破綻するリスク。

経路2（測地線）： リーマン距離で最短。不安定領域を迂回するため、ユークリッド的には遠回りだが、多様体上では最もエネルギーが低い経路。

経路3（制御付き最適経路）： 利用可能な3施策（サービス強化、段階的進出、合弁事業）の制約下で、移行コストを最小化する経路。ポントリャーギンの最大原理により、「最初の6ヶ月はサービス強化を最大強度、次の12ヶ月は段階的進出を中強度、最後の6ヶ月は合弁事業を高強度」という最適スケジュールが算出。

経営者への示唆： 経路1（直線的計画）は最短だが最もリスクが高い。経路3（制御付き最適経路）は最もリスクが低く、具体的な施策スケジュールが付いている。**経路3を推奨。**

ケーススタディB：安全保障環境の最適政策経路

「現在の安全保障環境」から「目標とする国際秩序」への最小リスク経路を計算。「まず同盟ネットワークの強化（12ヶ月）→ 次に経済安全保障の多角化（18ヶ月）→ その後に緊張緩和のための外交イニシアチブ（12ヶ月）」という段階的政策が最適と算出。

本章の品質チェックポイント

1. **制御系の軌道は発散しないか？** → 物理的に妥当な範囲に収まるか確認。
2. **測地線は不安定領域を適切に迂回しているか？** → 曲率マップ（第11章）と照合。
3. **最適経路はパラメータ変更で安定か？** → 初期条件やコスト関数を変えて安定性確認。

第14章：可視化・解釈・意思決定（Step 10）

この章で得られるもの—コストに見合うリターン

第9〜13章で計算したすべての結果を統合し、可視化し、意思決定に結びつけます。**本書のゴールはここです。**

▶ 数学の基礎から理解したい方へ

Step 10は数学的には新たな概念を導入しない。潜在空間の可視化（$d \leq 3$ は直接、$d > 3$ はt-SNE/UMAP投影）、スカラー曲率マップ、測地線と最適パスの描画、Lie微分場の可視化を行う。

:::message alert **因果推論との区別：** 本手順の幾何構造は**非線形相関構造**であり、因果関係ではない。因果推論には介入データ、構造因果モデル、自然実験が別途必要。 :::

誤差の累積： Step 2のデコーダ誤差 → Step 3の離散化誤差 → Step 4の計量誤差は下流に伝播する。特に曲率は2階微分を含み、誤差に敏感。ロバスト性チェックが不可欠。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—経営会議への報告

X社のデータサイエンスチームは、経営会議に以下の資料を提示しました。

資料1：経営環境の地形図（スカラー曲率マップ）。 安定領域（赤）と不安定領域（青）が色分けされた6次元多様体の2次元投影。過去のリーマンショック・COVID-19の位置をプロット。

資料2：施策のLie微分分析。 3施策の構造的インパクト（$\| \mathcal{L}_V g \|$）の比較表。コスト削減と市場開拓の干渉分析（Lie括弧）。

資料3：最適移行経路。 国内市場→新興国市場への3経路の比較（直線、測地線、制御付き最適経路）。推奨経路と施策スケジュール。

資料4：信頼度マップ。 データが密な領域（信頼度高）とデータが疎な領域（信頼度低）の色分け。「この分析は全領域で均一に信頼できるわけではない」ことを明示。

経営者の判断： 「経路3（制御付き最適経路）を採用する。ただし、不安定領域の通過時にはリスクバッファとして予備資金3億円を確保する。Lie括弧分析に基づき、コスト削減と市場開拓は同時並行ではなく、段階的に実施する。」

ケーススタディB：安全保障会議への報告書

安全保障分析チームは、以下の構成で報告書を作成しました。

セクション1：国際情勢の安定性評価。 曲率分析に基づく「構造的脆弱点」の特定。現在の国際状態と脆弱領域との距離。

セクション2：政策オプションの構造的分析。 同盟強化、経済制裁、外交イニシアチブの各政策のLie微分分析。政策間の干渉分析（Lie括弧）。

セクション3：最適政策経路。 現在の安全保障環境から目標とする国際秩序への最小リスク経路。段階的政策スケジュール。

セクション4：信頼度と限界。 データが疎な領域のリスト。分析の限界（非線形相関であり因果関係ではないこと）。ロバスト性チェックの結果。

```
import plotly.graph_objects as go
import numpy as np

def visualize_curvature_map(z_grid, scalar_curvature, title="Scalar Curvature Ma
p"):
    """Step 10: 対話的な曲率マップの描画"""
    fig = go.Figure(data=go.Heatmap(
        z=scalar_curvature,
        colorscale='RdBu_r',
        zmid=0))
    fig.update_layout(title=title, xaxis_title='z¹', yaxis_title='z²')
    fig.show()

print("Visualization tools ready")
```

本章の品質チェックポイント—最終チェック

このチェックポイントは本書全体の最終確認です。

1. 曲率マップは過去の既知イベントと整合するか？
2. Lie微分の順序はドメイン知識と整合するか？

3. 最適経路は物理的に妥当か（発散しない、値域が合理的）？
4. ランダムシード変更、データ10%除去で結果は安定か？
5. 結果は「探索的洞察」として提示されているか（確定的結論ではなく）？
6. 因果関係と相関の区別が明記されているか？

すべてYesなら、分析結果を意思決定の参考情報として活用できます。1つでもNoなら、該当する章に戻ってやり直してください。

付録A：仮定の一覧と数学的正当性

- **A1. 多様体仮説：** データが低次元 C^∞ 部分多様体の近傍から標本されている。
- **A2. デコーダの正則性：** (a) $f_\theta \in C^k$ ($k \geq 3$)。(b) $\mathrm{rank}\, J_{f_\theta}(z) = d$ ($\forall z \in U$)。
- **A3. 十分なデータ量：** $N\varepsilon^{d/2+1} \to \infty$ のレート条件。
- **A4. 数値計算の安定性：** $\kappa(g(z))$ が有界。

仮定	破れた場合の影響
A1	多様体自体が存在しない。全手順が無意味
A2(a)	曲率テンソルが定義できない（ReLU使用時）
A2(b)	計量が退化し、逆行列が存在しない
A3	VDMの幾何構造抽出が不正確
A4	テンソル計算が数値的に不安定

付録B：Python環境セットアップ

```
# 基本環境
pip install numpy pandas scikit-learn matplotlib plotly

# 深層学習
pip install torch torchvision torchdiffeq

# TDA（位相的データ解析、第8章で使用）
pip install ripser persim giotto-tda
```

```
# 固有次元推定（チェック1で使用）
pip install scikit-dimension

# 可視化
pip install plotly jupyter
```

付録C：参考文献

1. J. M. Lee, *Introduction to Smooth Manifolds*, 2nd ed., Springer, 2012.
2. A. Singer and H.-T. Wu, "Vector Diffusion Maps and the Connection Laplacian," *Comm. Pure Appl. Math.*, 65(8), 2012.
3. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," *Proc. ICLR*, 2014.
4. R. T. Q. Chen et al., "Neural Ordinary Differential Equations," *Proc. NeurIPS*, 2018.
5. G. Arvanitidis et al., "Latent Space Oddity," *Proc. ICLR*, 2018.
6. E. Facco et al., "Estimating the intrinsic dimension of datasets," *Scientific Reports*, 7, 2017.
7. E. Levina and P. J. Bickel, "Maximum likelihood estimation of intrinsic dimension," *Proc. NeurIPS*, 2004.
8. L. S. Pontryagin et al., *The Mathematical Theory of Optimal Processes*, 1962.
9. M. M. Bronstein et al., "Geometric Deep Learning," *arXiv:2104.13478*, 2021.
10. M. Gidea and Y. Katz, "Topological Data Analysis of Financial Time Series," *Frontiers in Applied Mathematics and Statistics*, 2018.
11. J. Hansen and T. Gebhart, "Sheaf Neural Networks," *NeurIPS Workshop*, 2020.
12. C. Bodnar et al., "Neural Sheaf Diffusion," *Proc. ICLR*, 2022.
13. S. Amari, *Information Geometry and Its Applications*, Springer, 2016.

第II部：幾何学的AIの道具

第I部では、微分幾何学の4つの基礎概念—多様体、計量、接続・曲率、Lie微分—を学びました。第II部では、これらの数学的概念を社会データに適用するための**AIモデル**を学びます。VAE（第5章）、Vector Diffusion Mapsと計量学習（第6章）、Neural ODEと自動微分テンソル計算（第7章）、Geometric Data Scienceの全体像（第8章）の4つです。

数学に強くAIに弱い読者（数学者を含む）は、この第II部が最も重要です。 逆に、AIに強く数学に弱い読者は、第I部に戻って数学的基礎を確認してください。

第5章：VAEと滑らかな潜在多様体の構成

この章で得られるもの—コストに見合うリターン

VAE（変分オートエンコーダ）は、本書のアプローチ全体を支える最も重要なAIモデルです。高次元のビジネスデータや政策データを、低次元の「滑らかな空間」に圧縮し、その空間が数学的に正当な多様体になることを保証します。この章は、特に「微分幾何学は知っているがVAEを知らない数学者」にとって必読です。

▶ 数学の基礎から理解したい方へ

VAEは確率的生成モデルであり、観測データ $x \in \mathbb{R}^n$ と潜在変数 $z \in \mathbb{R}^d$（$d \ll n$）の同時分布 $p_\theta(x,z) = p_\theta(x|z)p(z)$ をモデル化する。エンコーダ $q_\phi(z|x)$ は変分事後分布、デコーダ $p_\theta(x|z)$ は尤度関数であり、ELBO（Evidence Lower Bound）

$$\mathcal{L}(\theta,\phi;x) = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] + \mathrm{KL}\left(q_\phi(z|x) \parallel p(z)\right)$$

の最小化により θ, ϕ を同時に学習する。

本書の文脈でVAEが決定的に重要な理由は、**デコーダ $f_\theta: \mathbb{R}^d \to \mathbb{R}^n$ が可微分多様体の構成写像を提供する**ことにある。第1章の命題1.1で示した通り、$f_\theta \in C^k$（$k \geq 3$）かつ $\mathrm{rank} J_{f_\theta}(z) = d$ かつ $f_\theta|_U$ が単射であれば、$f_\theta(U)$ は C^k 級埋め込み部分多様体となる。

数学者へのメッセージ： VAEは「確率論と最適化の組み合わせで、高次元データの低次元構造を学習する」道具です。以下では、確率的定式化の各要素を、微分

幾何学の観点から何が重要かに焦点を当てて解説します。微分幾何学の観点から最も重要なのは**デコーダの滑らかさ（C^k 級性）**と**ヤコビ行列のフルランク性**であり、VAEの確率的側面（KLダイバージェンス、ELBO）は多様体構造の保証には直接関与しないが、潜在空間の「穴のなさ」（データの潜在表現が潜在空間全体に分布すること）を促進し、間接的にフルランク条件の成立を助ける。

コスト： 確率的生成モデルの枠組みの理解。変分推論、KLダイバージェンス、再パラメータ化トリックといった確率論・最適化の概念が必要。 **リターン：** 社会データから C^k 級可微分多様体を構成するための、現時点で最も実用的かつ数学的に正当化可能な方法。

▶ AIモデルの基礎から理解したい方へ

VAEを「ゼロから」理解する。

VAEは2014年にKingmaとWellingによって提案されたニューラルネットワークモデルです。名前の「変分（Variational）」は「変分推論」という統計学の手法から、「オートエンコーダ（Autoencoder）」は「データを圧縮して復元するニューラルネット」から来ています。

VAEを理解するために、まず**通常のオートエンコーダ**から始めましょう。

通常のオートエンコーダ（AE）とは。 入力データ x（たとえば30個の経営指標からなるベクトル）を、**エンコーダ**と呼ばれるニューラルネットワークで低次元の表現 z（たとえば6個の数値）に圧縮し、**デコーダ**と呼ばれるニューラルネットワークで元のデータ $\hat{x}$ に復元する。目標は $\hat{x} \approx x$（復元がなるべく元に近い）にすること。

$$x \xrightarrow{\text{エンコーダ}} z \xrightarrow{\text{デコーダ}} \hat{x} \approx x$$

エンコーダ：$\mathbb{R}^n \to \mathbb{R}^d$ （圧縮）。デコーダ：$\mathbb{R}^d \to \mathbb{R}^n$ （復元）。$d \ll n$ とすることで次元圧縮が行われます。

損失関数は再構成誤差 $\| x - \hat{x} \|^2$ の平均であり、これを最小化するようにエンコーダとデコーダのパラメータを同時に学習します。

通常のAEの問題点。 通常のAEでは、エンコーダは各入力 x に対して**1つの確定的な点** z を出力します。問題は、訓練データが潜在空間の「まばらな点」としてしか配置されないことです。訓練データの点と点の間の潜在空間には「何もない」——デコーダはその領域で学習していないため、そこに点を入力すると、意味のないデータが生成されます。

これは多様体構成にとって致命的です。多様体は**連続的で滑らかな空間**であり、すべての点（訓練データに対応する点だけでなく、点と点の間の点も）が意味を持つ必要があります。「穴だらけの潜在空間」では、多様体の連続性が保証されません。

VAEのアイデア：確率的にする。 VAEでは、エンコーダが確定的な点 z ではなく、**確率分布**（正規分布）を出力します。

具体的には、エンコーダは入力 x に対して以下を出力します。

- 平均ベクトル $\mu_\phi(x) \in \mathbb{R}^d$
- 分散ベクトル $\sigma_\phi^2(x) \in \mathbb{R}^d_{>0}$（各成分が正）

潜在表現 z は、この正規分布からサンプリングされます：

$$q_\phi(z|x) = \mathcal{N}\left(z; \mu_\phi(x), \mathrm{diag}\left(\sigma_\phi^2(x)\right)\right)$$

つまり、z の各成分 z^i は独立な正規分布 $\mathcal{N}\left(\mu_\phi^i(x), \sigma_\phi^{i2}(x)\right)$ に従います。

「確率的にする」ことの効果。 各訓練データ点は、潜在空間上の「1点」ではなく「広がり」（正規分布の裾野）として配置されます。これにより、訓練データの点と点の間の空間にもデータの潜在表現が（低い確率ながら）存在するようになり、デコーダはその領域でも学習が行われ、「穴のない」滑らかな潜在空間が実現します。

VAEの損失関数（ELBO）。

$$\mathcal{L}(\theta, \phi; x) = \underbrace{-\mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x|z)\right]}_{\text{再構成誤差}} + \underbrace{\mathrm{KL}\left(q_\phi(z|x) \parallel p(z)\right)}_{\text{KL正則化項}}$$

第1項（再構成誤差）： デコーダがデータをどれだけ正確に復元できるか。$-\log p_\theta(x|z)$ はデコーダが z から x を生成する「尤もらしさ」の負の対数。ガウスデコーダ（$p_\theta(x|z) = \mathcal{N}\left(x; f_\theta(z), \sigma^2 I\right)$）の場合、再構成誤差は $\parallel x - f_\theta(z) \parallel^2 / \left(2\sigma^2\right) + \mathrm{const}$ となり、通常のAEの再構成誤差と本質的に同じ。

第2項（KLダイバージェンス）： エンコーダが出力する事後分布 $q_\phi(z|x)$ が、事前分布 $p(z) = \mathcal{N}\left(0, I_d\right)$（標準正規分布）にどれだけ近いかを測る。KLダイバージェンスは

$$\mathrm{KL}\left(q_\phi(z|x) \parallel p(z)\right) = \frac{1}{2}\sum_{i=1}^{d}\left(\sigma_\phi^{i2}(x) + \mu_\phi^{i2}(x) - 1 - \log\sigma_\phi^{i2}(x)\right)$$

と解析的に計算でき、この項を最小化すると $\mu \to 0,\ \sigma^2 \to 1$ となり、潜在表現が原点周辺の標準正規分布に近づく。これが潜在空間の「穴のなさ」を促進する。

再パラメータ化トリック。 z は確率変数であるため、そのままでは逆伝播（勾配計算）ができない。再パラメータ化トリックは、$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$（$\varepsilon \sim \mathcal{N}(0, I)$, $\odot$ は要素ごとの積）と書き直すことで、ランダム性を ε に分離し、μ と σ に関する勾配を計算可能にする。

▶ ビジネス・政策の意思決定に活用したい方へ

VAEが行うことを「地形図作成」のたとえで説明します。

あなたの企業には、30個の経営指標（売上、利益率、市場シェア、為替レート…）の10年分の月次データがあります。各月のデータは30次元の空間の中の1点です。10年×12ヶ月＝120点が、30次元空間に散らばっています。

VAEは、この30次元のデータを6次元の「経営環境地形図」に圧縮します。圧縮の方法は以下の通りです。

Step 1（圧縮）： 30個の指標を6個の「本質的な因子」に要約する。これがエンコーダの仕事。 **Step 2（復元検証）：** 6個の因子から元の30個の指標を復元できるか確認する。復元精度が高ければ、圧縮時に重要な情報が失われていない。 **Step 3（地形図の完成）：** 復元に使う関数（デコーダ）が「地形図の描き方」を定義する。この関数が滑らかであれば、地形図も滑らかになり、曲率やLie微分が計算可能。

コスト： VAEの訓練に必要な計算資源と時間（GPUで数時間〜数日）。 **リターン：** 30次元の「見えない」データ構造が、6次元の「計算可能な地形図」に変換される。

5.1 VAEの数学的構造

▶ 数学の基礎から理解したい方へ

エンコーダ（変分事後分布）

定義5.1（エンコーダ）． エンコーダ $q_\phi(z|x)$ は、パラメータ ϕ を持つニューラルネットワークにより

$$q_\phi(z|x) = \mathcal{N}\left(z;\, \mu_\phi(x),\, \mathrm{diag}\left(\sigma_\phi^2(x)\right)\right)$$

と定義される。ここで $\mu_\phi: \mathbb{R}^n \to \mathbb{R}^d$ と $\sigma_\phi: \mathbb{R}^n \to \mathbb{R}^d_{>0}$ はニューラルネットワーク。

数学的には、エンコーダは $\mathbb{R}^n$ から $\mathbb{R}^d$ 上の正規分布の族への写像であり、データ空間の各点 x を潜在空間上の確率測度 $q_\phi(\cdot\,|x)$ に対応させる。この写像の像は、$\mathbb{R}^d$ 上の（対角共分散を持つ）正規分布の $(2d)$-パラメータ族の部分族をなす。

デコーダ（生成モデル）

定義5.2（デコーダ）． デコーダ $p_\theta(x|z)$ は、決定論的関数 $f_\theta: \mathbb{R}^d \to \mathbb{R}^n$（デコーダネットワーク）を用いて

$$p_\theta(x|z) = \mathcal{N}\left(x;\, f_\theta(z),\, \sigma_{\mathrm{dec}}^2 I_n\right)$$

と定義される（σ_{dec}^2 は固定のデコーダノイズ分散）。

本書における最重要条件。 デコーダネットワーク f_θ に対して以下を要求する：

(A) $f_\theta \in C^\infty$．すべての層で C^∞ 活性化関数（tanh, GELU, Softplus）を使用。ReLU（$C^0 \setminus C^1$）は不可。

(B) $\mathrm{rank}\, J_{f_\theta}(z) = d$ $(\forall z \in U)$．関心領域 U のすべての点でヤコビ行列がフルランク。

条件（A）は活性化関数の選択で自動的に保証される。条件（B）は訓練後に数値的に検証する必要がある（第1章のヤコビ行列SVD検証コード参照）。

命題5.1（KL正則化の限界）． KLダイバージェンス $\mathrm{KL}\left(q_\phi(z|x) \,\|\, p(z)\right)$ は、集約事後分布 $q_\phi(z) = \mathbb{E}_{p_{\mathrm{data}}(x)}\left[q_\phi(z|x)\right]$ が事前分布 $p(z) = \mathcal{N}(0, I_d)$ に近づくことを

促進する。これは潜在空間の分布的な正則性（「穴のなさ」）を保証するが、**デコーダの多様体構造（C^k 級性、フルランク性、単射性）を直接保証するものではない**。多様体構造は条件（A），（B）および単射性によって保証される。

事前分布

定義5.3（事前分布）． $p(z) = \mathcal{N}(0, I_d)$（$d$ 次元標準正規分布）。

事前分布の選択は「潜在空間の基準分布」を定めるものであり、多様体の幾何構造には直接影響しない。幾何構造はデコーダ f_θ のヤコビ行列から導かれる引き戻し計量によって決定される。

▶ AIモデルの基礎から理解したい方へ

VAEのPython実装。

```
import torch
import torch.nn as nn

class VAE(nn.Module):
    """
    変分オートエンコーダ（VAE）の完全な実装。

    重要: デコーダに nn.Tanh() を使用（C∞活性化関数）。
    nn.ReLU() は使用不可（C^0、曲率計算不能になるため）。
    """
    def __init__(self, input_dim, latent_dim, hidden_dim=128):
        super().__init__()

        # === エンコーダ ===
        # 入力データ x（input_dim次元）→ 隠れ層 → μ と log(σ²)
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Tanh(),                          # C∞ 活性化関数
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.Tanh(),
        )
        # 平均 μ を出力する層
        self.mu_layer = nn.Linear(hidden_dim // 2, latent_dim)
        # log(σ²) を出力する層（σ²ではなくlogを出力する理由:
        #   σ²は正でなければならないが、NNの出力は任意の実数。
        #   logを取ることで、exp(log σ²) = σ² > 0 を自動的に保証）
        self.logvar_layer = nn.Linear(hidden_dim // 2, latent_dim)
```

```python
        # === デコーダ ===
        # 潜在変数 z (latent_dim次元) → 隠れ層 → 復元データ x^
        # ★★★ 全層で nn.Tanh() を使用 (C∞保証) ★★★
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim // 2),
            nn.Tanh(),                          # ← ReLUではなくTanh
            nn.Linear(hidden_dim // 2, hidden_dim),
            nn.Tanh(),                          # ← ReLUではなくTanh
            nn.Linear(hidden_dim, input_dim),
            # 最終層は活性化なし (データのスケールに制約を設けない)
        )

    def encode(self, x):
        """エンコーダ: x → (μ, log σ²)"""
        h = self.encoder(x)
        mu = self.mu_layer(h)
        logvar = self.logvar_layer(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        """
        再パラメータ化トリック: z = μ + σ * ε (ε ~ N(0, I))

        なぜ必要か: z ~ N(μ, σ²I) からのサンプリングは
        「ランダムな操作」であり、μやσに関する勾配が計算できない。
        z = μ + σ*ε と書き直すと、ランダム性がεに分離され、
        zのμとσに関する勾配が計算可能になる。
        """
        std = torch.exp(0.5 * logvar)  # σ = exp(log σ²/2)
        eps = torch.randn_like(std)     # ε ~ N(0, I)
        return mu + std * eps           # z = μ + σ * ε

    def decode(self, z):
        """デコーダ: z → x^"""
        return self.decoder(z)

    def forward(self, x):
        """
        完全なフォワードパス: x → (μ, logσ²) → z → x^
```

```
        Returns:
            x_recon: 復元されたデータ
            mu: 潜在表現の平均
            logvar: 潜在表現の log(分散)
        """
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        x_recon = self.decode(z)
        return x_recon, mu, logvar


def vae_loss(x_recon, x, mu, logvar):
    """
    VAEの損失関数 (ELBO の負)

    = 再構成誤差 + KLダイバージェンス

    再構成誤差: ||x - x^||²  (データの復元精度)
    KL項: -0.5 * Σ(1 + log σ² - μ² - σ²)  (潜在分布の正則化)
    """
    # 再構成誤差 (平均二乗誤差)
    recon_loss = nn.functional.mse_loss(x_recon, x, reduction='sum')

    # KLダイバージェンス KL(N(μ, σ²I) || N(0, I))
    # = -0.5 * Σ_i (1 + log σ²_i - μ²_i - σ²_i)
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())

    return recon_loss + kl_loss
```

コードの各部分の意味：

self.encoder：30次元のデータを中間表現に圧縮するニューラルネット。nn.Tanh() は $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ を計算する活性化関数。出力は $(-1,1)$ の範囲。

self.mu_layer, self.logvar_layer：中間表現から、潜在空間の平均 μ と分散の対数 $\log\sigma^2$ を出力。$\log\sigma^2$ を出力する理由は、$\sigma^2 > 0$ を保証するため。$\exp(\log\sigma^2) = \sigma^2$ は常に正。

self.decoder：6次元の潜在表現から30次元のデータを復元。**全層で nn.Tanh() を使用**—これが本書の最重要要件。

reparameterize：再パラメータ化トリック。$z = \mu + \sigma \cdot \varepsilon$（$\varepsilon \sim \mathcal{N}(0, I)$）。

vae_loss：ELBO = 再構成誤差 + KLダイバージェンス。再構成誤差が小さいほどデータの復元が正確。KL項が小さいほど潜在分布が標準正規に近い。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—VAEで経営環境多様体を構成する

X社のデータサイエンスチームは、以下の手順でVAEを訓練しました。

入力データ： 50店舗×月次×10年分のデータから、30指標を標準化（平均0、標準偏差1に変換）。計6,000データ点×30次元。

VAEの設計： 入力次元 $n = 30$、潜在次元 $d = 6$（第1章のPCAで固有次元が約6と推定されたため）。隠れ層は128次元と64次元の2層。活性化関数は全層 tanh。

訓練： 500エポック、バッチサイズ64、学習率0.001（Adam optimizer）。

結果： - 再構成誤差：元データの分散の4.2%（基準の10%以下を満たす → ✓）- KLダイバージェンス：3.8（潜在表現が標準正規分布に近い → ✓） - ヤコビ行列の最小特異値：8.3×10^{-3}（$> 10^{-4}$ → フルランク条件 ✓）

経営者への報告： 「30個の経営指標から、6つの『本質的因子』を抽出し、6次元の経営環境地形図を構成しました。地形図の精度は96%（再構成誤差4.2%）であり、元のデータの構造をほぼ忠実に反映しています。」

ケーススタディB：国際情勢多様体の構成

安全保障分析チームは、193カ国×40指標×30年のデータ（月次追加データ含め約70,000点）からVAEを訓練。潜在次元 $d = 8$。

結果： 再構成誤差 6.1%（基準内）。ただし、データが疎な一部の国家周辺でヤコビ行列の特異値が低下。チームは「信頼度マップ」を作成し、分析結果に付記。

5.2 活性化関数と滑らかさ—なぜReLUは使えないのか

▶ 数学の基礎から理解したい方へ

デコーダ f_θ の滑らかさのクラスは、活性化関数の滑らかさで決定される。本書で使用可能/不可能な活性化関数を以下にまとめる。

活性化関数	定義	滑らかさ	使用可否	理由
ReLU	$\max(0,x)$	C^0（$x=0$で微分不連続）	✗ 不可	$g_{ij}\in C^{k-1}$, $k\geq 1$ すら非保証。曲率テンソル定義不能
Leaky ReLU	$\max(\alpha x,x)$	C^0（$x=0$で微分不連続）	✗ 不可	同上
tanh	$(\square^{\square}-\square^{-\square})/(e^x+e^{-x})$	C^∞	✓ 推奨	$k=\infty$ で全条件を満たす
GELU	$x\Phi(x)$（Φ: 標準正規CDF）	C^∞	✓ 推奨	同上。Transformerで広く使用
Softplus	$\log(1+e^x)$	C^∞	✓ 推奨	ReLUの滑らかな近似
SiLU/Swish	$x\sigma(x)$（σ: シグモイド）	C^∞	✓ 使用可	GELU に類似
ELU	$\begin{cases} x & x>0 \\ \alpha(e^x-1) & x\leq 0 \end{cases}$	C^1	△ 条件付き	C^1 だが C^2 でない。曲率に不十分

注意5.1. ReLUを使った場合のデコーダ f_θ は、入力空間 $\mathbb{R}^d$ を多面体的な領域（ポリトープ）に分割し、各領域内ではアフィン写像 $z\mapsto W_{\text{eff}}z+b_{\text{eff}}$ を行う区分的線形関数となる。領域の境界面（ReLUが 0 から正に切り替わる超平面）上では、ヤコビ行列が不連続に変化し、$g_{ij}=\left(J^\top J\right)_{ij}$ の偏微分（クリストッフェル記号の計算に必要）が定義されない。したがって、曲率テンソルは**領域の内部でのみ**定義可能であり、境界面上では定義できない。

▶ AIモデルの基礎から理解したい方へ

ReLUとtanhの違いを、コードで確認する。

```
import torch
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

x = torch.linspace(-3, 3, 1000, requires_grad=True)
```

```
# ReLU とその微分
relu = torch.relu(x)
relu_grad = torch.autograd.grad(relu.sum(), x, create_graph=True)[0]
axes[0,0].plot(x.detach(), relu.detach(), 'b-', label='ReLU(x)')
axes[0,0].plot(x.detach(), relu_grad.detach(), 'r--', label="ReLU'(x)")
axes[0,0].set_title('ReLU: C⁰ (not differentiable at x=0)')
axes[0,0].legend(); axes[0,0].grid(True)

# tanh とその微分
tanh = torch.tanh(x)
tanh_grad = torch.autograd.grad(tanh.sum(), x, create_graph=True)[0]
axes[0,1].plot(x.detach(), tanh.detach(), 'b-', label='tanh(x)')
axes[0,1].plot(x.detach(), tanh_grad.detach(), 'r--', label="tanh'(x)")
axes[0,1].set_title('tanh: C∞ (infinitely differentiable)')
axes[0,1].legend(); axes[0,1].grid(True)

# ReLUの2階微分（定義不能を示す）
try:
    relu_grad2 = torch.autograd.grad(relu_grad.sum(), x, create_graph=True)[0]
    axes[1,0].plot(x.detach(), relu_grad2.detach(), 'g-')
except:
    axes[1,0].text(0.5, 0.5, 'Cannot compute¥n2nd derivative',
                   transform=axes[1,0].transAxes, ha='center', fontsize=14)
axes[1,0].set_title("ReLU''(x): UNDEFINED at x=0")

# tanhの2階微分（問題なく計算可能）
tanh_grad2 = torch.autograd.grad(tanh_grad.sum(), x, create_graph=True)[0]
axes[1,1].plot(x.detach(), tanh_grad2.detach(), 'g-', label="tanh''(x)")
axes[1,1].set_title("tanh''(x): well-defined everywhere")
axes[1,1].legend(); axes[1,1].grid(True)

plt.suptitle('Why tanh is required (not ReLU) for curvature computation', fontsi
ze=14)
plt.tight_layout()
plt.show()
```

このコードは、ReLUとtanhの1階微分と2階微分を比較します。ReLUの1階微分は $x = 0$ でステップ関数（0から1へのジャンプ）になり、2階微分はその点でデルタ関数（定義不能）です。tanh の微分はすべて滑らかです。

本章の品質チェックポイント

▶ AIモデルの基礎から理解したい方へ

☐ チェックリスト（第5章終了時点）

1. **活性化関数：** デコーダの全層で tanh/GELU/Softplus を使用しているか？ → ReLU使用中なら**変更して再訓練**。
2. **再構成誤差：** 元データの分散の10%以下か？ → 超えている場合、**隠れ層のサイズ増加、潜在次元の増加、訓練エポック数の増加**を検討。
3. **ヤコビ行列フルランク：** 最小特異値 $> 10^{-4}$（代表50点以上で検証）。→ 不合格なら**その領域を除外するかdを下げる**。
4. **潜在次元の適切さ：** d を1, 2, …, と増やして再構成誤差の「肘」を確認。PCAの固有次元推定と一致するか。
5. **訓練の安定性：** 異なるランダムシードで3回以上訓練し、結果（潜在表現の構造、再構成誤差）が概ね一致するか。

▶ ビジネス・政策の意思決定に活用したい方へ

チームに確認： 「VAEの再構成誤差は何%か？デコーダにtanhを使っているか？ヤコビ行列の検証は済んでいるか？」

これらに明確に回答できないチームの分析結果は信頼すべきではありません。

次章予告： 第6章ではVDM（Vector Diffusion Maps）と計量学習を学びます。VAEが「多様体を構成する」のに対し、VDMは「多様体の幾何構造を抽出する」補完的なツールです。

第6章：Vector Diffusion Mapsと計量学習

この章で得られるもの—コストに見合うリターン

VAE（第5章）が「多様体を構成する」AIモデルであるのに対し、VDM（Vector Diffusion Maps）は「離散データ点から多様体の幾何構造を抽出する」手法です。さらに、引き戻し計量とCholeskyパラメトリゼーションによる計量学習を解説します。

▶ 数学の基礎から理解したい方へ

VDMの数学的内容。 Singer & Wu (2012) のVector Diffusion Mapsは、離散データ $\{z_i\}_{i=1}^N \subset \mathbb{R}^d$ からグラフ接続ラプラシアン S_ε を構成する。ガウスカーネル $w_{ij} = \exp\left(-\| z_i - z_j \|^2/\varepsilon\right)$、局所接空間間の最適回転行列 $O_{ij} \in O(d)$ を用いて

$$(S_\varepsilon v)_i = \frac{1}{d_i}\sum_j w_{ij}\left(v_i - O_{ij}v_j\right), \quad d_i = \sum_j w_{ij}$$

連続極限定理（Singer-Wu, 2012）. 仮定A1の下で、$N \to \infty$, $\varepsilon \to 0$, $N\varepsilon^{d/2+1} \to \infty$ のとき、S_ε は M_0 の接束上の接続ラプラシアンにスペクトルの意味で収束する。

計量学習：引き戻し計量とCholeskyパラメトリゼーション。 引き戻し計量は第2章で導入済み。代替手法として、計量を直接ニューラルネットでパラメトリゼーションするCholesky法がある：$g_{ij}(z) = \left(L_\psi(z)L_\psi(z)^\top\right)_{ij}$。$L_\psi$ は下三角行列で対角に exp 適用（正値保証）。

▶ AIモデルの基礎から理解したい方へ

VDMの内部動作を段階的に説明。

(1) 各データ点の k 個の最近傍を見つけ、ガウスカーネルで重み付け。
(2) 各点の近傍データ点から、局所的な接空間（「方向」の空間）をPCAで推定。
(3) 隣接する点の接空間間の「向きのずれ」を回転行列 O_{ij} で表す。
(4) 接続ラプラシアンの固有値分解で多様体の大域的幾何構造を抽出。

```
import numpy as np
from sklearn.neighbors import NearestNeighbors

def build_vdm_laplacian(data, epsilon, k_neighbors=10):
    """VDM接続ラプラシアンの簡易実装"""
    N, d = data.shape
    nn = NearestNeighbors(n_neighbors=k_neighbors).fit(data)
    distances, indices = nn.kneighbors(data)

    # ガウスカーネル重み
    W = np.zeros((N, N))
    for i in range(N):
```

```
        for j_idx, j in enumerate(indices[i]):
            W[i, j] = np.exp(-distances[i, j_idx]**2 / epsilon)
            W[j, i] = W[i, j]  # 対称化

    D = np.diag(W.sum(axis=1))
    L = D - W  # グラフラプラシアン（スカラー版の簡略実装）

    # 固有値分解
    eigenvalues, eigenvectors = np.linalg.eigh(L)
    return eigenvalues, eigenvectors, L
```

引き戻し計量との関係。 VDMは「データ駆動型」、引き戻し計量は「モデル駆動型」の幾何構造抽出です。両者を比較し整合性を確認することで、分析の信頼性が高まります。

▶ ビジネス・政策の意思決定に活用したい方へ

VDMは「別の角度から地形図の信頼性を検証する」ツールです。VAEで作った地形図とVDMで抽出した構造が整合していれば、地形図の信頼性が高い。整合しなければ、どちらかに問題がある。

本章の品質チェックポイント

1. VDMのスペクトルギャップ $\lambda_2 - \lambda_1$ は ε を変えても安定か？
2. 引き戻し計量とVDMの幾何構造は整合するか？

第7章：Neural ODEと自動微分テンソル計算

この章で得られるもの—コストに見合うリターン

Neural ODEはデータの「変化の法則」を学習するAIモデル、自動微分は計量→曲率→Lie微分のフルチェーン計算を自動化する技術です。この2つが揃うと、「政策シミュレーション」と「テンソル計算」がPythonで実行可能になります。

▶ 数学の基礎から理解したい方へ

Neural ODE. 常微分方程式 $dz/dt = f_\theta(z,t)$ の右辺 f_θ をニューラルネットでパラメトリゼーション。ODE solver（Runge-Kutta等）で数値解を計算。勾配は随伴法で計算：随伴状態 $a(t) = \partial L/\partial z(t)$ は $da/dt = -a^\top(\partial f_\theta/\partial z)$ を時間逆向きに解くことで得られ、これはポントリャーギンの最大原理の離散アナログ。

自動微分テンソル計算。 PyTorchの autograd（create_graph=True）により、計量 g_{ij} → クリストッフェル記号 Γ^k_{ij} → 曲率テンソル $R^l{}_{ijk}$ → Lie微分 $\mathcal{L}_V g$ のフルチェーン計算が実行可能。

▶ AIモデルの基礎から理解したい方へ

Neural ODEを「ゼロから」理解する。

通常のニューラルネット（ResNetなど）では、データは離散的な「層」を順番に通過します：$h_{t+1} = h_t + f_\theta(h_t, t)$。Neural ODEは、この層の厚みを無限に薄くした極限として、連続的な微分方程式 $dz/dt = f_\theta(z,t)$ を考えます。

```
import torch
import torch.nn as nn
from torchdiffeq import odeint

class NeuralODEFunc(nn.Module):
    """Neural ODEの速度場（右辺）をニューラルネットで定義"""
    def __init__(self, dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(dim, 64), nn.Tanh(),  # C∞活性化
            nn.Linear(64, 64), nn.Tanh(),
            nn.Linear(64, dim))

    def forward(self, t, z):
        return self.net(z)

# 使用例
func = NeuralODEFunc(dim=6)  # 6次元（X社の潜在空間次元）
z0 = torch.randn(1, 6)       # 初期状態
t_span = torch.linspace(0, 5, 100)  # 時間区間
trajectory = odeint(func, z0, t_span)  # ODE解を計算
print(f"Trajectory shape: {trajectory.shape}")  # [100, 1, 6]
```

自動微分によるフルチェーン計算のまとめ。

```
import torch

def full_geometry_chain(metric_fn, z, d=2):
    """計量からスカラー曲率まで一気に計算するフルチェーン"""
    z = z.clone().requires_grad_(True)

    # Step 1: 計量テンソル g_{ij}
    g = metric_fn(z)
    g_inv = torch.inverse(g)

    # Step 2: 計量の偏微分 ∂g_{jl}/∂z^i
    dg = torch.zeros(d, d, d)
    for j in range(d):
        for l in range(d):
            grad = torch.autograd.grad(g[j,l], z, retain_graph=True, create_grap
h=True)[0]
            dg[:, j, l] = grad

    # Step 3: クリストッフェル記号 Γ^k_{ij}
    Gamma = torch.zeros(d, d, d)
    for k in range(d):
        for i in range(d):
            for j in range(d):
                s = 0.0
                for l in range(d):
                    s += g_inv[k,l] * (dg[i,j,l] + dg[j,i,l] - dg[l,i,j])
                Gamma[k,i,j] = 0.5 * s

    print(f"det(g) = {torch.det(g).item():.4f}")
    print(f"Γ^0_{{01}} = {Gamma[0,0,1].item():.4f}")
    return g, Gamma
```

▶ ビジネス・政策の意思決定に活用したい方へ

Neural ODEは「経営環境の変化法則をデータから自動的に発見する」ツールです。「売上が増えると利益率はどう動くか」「為替が変動すると原材料コストはどう反応するか」—これらの**変化の法則**を、過去のデータから自動的に学習します。

自動微分テンソル計算は、これまで数学者が何週間もかけて手計算していた曲率やLie微分を、コンピュータが秒単位で正確に計算する技術です。

本章の品質チェックポイント

1. Neural ODEの軌道は物理的に妥当か（発散しない、値域が合理的）？
2. 自動微分でNaN/Infが出ていないか？
3. 有限差分との比較で自動微分の結果が整合するか？

第8章：Geometric Data Scienceの全体像

この章で得られるもの—コストに見合うリターン

本書のアプローチ（多様体微分解析）は、**Geometric Data Science（幾何学的データサイエンス）**と呼ばれる広大な分野のほんの一部です。本章では、この分野の全体像を俯瞰し、本書のアプローチがどこに位置するのかを明らかにします。自分が今いる場所と、今後探索できる地図の全体を把握してください。

▶ 数学の基礎から理解したい方へ

Geometric Data Scienceは、多様体（manifold）、グラフ（graph）、群・対称性（symmetry）、トポロジー（topology）、層（sheaf）、圏（category）といった幾何学的・代数的概念を活用し、データの本質的な構造を反映した分析を行う分野の総称である。本章では、8つの主要アプローチを体系的に解説し、各アプローチの数学的基盤と本書との関係を明らかにする。

▶ AIモデルの基礎から理解したい方へ

Geometric Data Scienceには多様な手法が存在します。本章はそれらの「地図」を提供します。GNN、TDA、Equivariant Networks、Poincaré Embeddings、Sheaf Neural Networks—これらの名前を聞いたことがあっても、全体像の中でどこに位置するか把握していない方は多いはずです。

▶ ビジネス・政策の意思決定に活用したい方へ

本書の手法は万能ではありません。データの性質や分析目的に応じて、最適なGeometric Data Scienceの手法は異なります。本章を読むことで、「自分の問題に最適な手法はどれか」を判断するための基盤が得られます。

Geometric Data Scienceの全体像

Geometric Data Science（幾何学的データサイエンス）は、データの形状・構造・空間的関係を数学的に捉え、それをAIや統計学に組み込むアプローチの総称です。

通常の統計学や機械学習は、データがユークリッド空間 $\mathbb{R}^n$ 上に存在することを暗黙に仮定しています。しかし、現実のデータの多くは—画像、分子構造、ソーシャルネットワーク、時系列、確率分布—ユークリッド空間には収まりません。

以下の図は、Geometric Data Scienceの主要な8つのアプローチを俯瞰したものです。本書のアプローチ（★印）はこの中の「1. 多様体・リーマン幾何」に位置します。

> **【図：図8-1：Geometric Data Scienceの全体像】** *図8-1：Geometric Data Scienceの8つの主要アプローチ。本書のアプローチ（★本書Step 4-9）は「1. 多様体・リーマン幾何」に位置する。*

---

1. 多様体・リーマン幾何学ベースのモデル ★本書のメインアプローチ

▶ 数学の基礎から理解したい方へ

データが非線形空間に分布している場合、その空間のリーマン幾何学的構造を考慮して次元削減やクラスタリングを行う手法群。

Riemannian PCA（Principal Geodesic Analysis, PGA）： 通常のPCAは、データの分散を最大化する方向（直線）を見つける。PGAは多様体上でこれを一般化し、分散を最大化する**測地線の方向**を見つける。フレシェ平均と測地的主成分を用いて、多様体上で意味のある統計量を計算する。

測地距離埋め込み： データ点間のユークリッド距離の代わりに、多様体上の測地距離を用いて低次元埋め込みを行う。Isomapが代表例。

リーマン勾配降下法： パラメータ空間が多様体（$SO(3)$、確率単体等）上にある場合の最適化。ユークリッド勾配 ∇f の代わりにリーマン勾配 $\mathrm{grad} f$ を計算し、指数写像 $z_{t+1} = \mathrm{Exp}_{z_t}\left(-\eta \cdot \mathrm{grad} f(z_t)\right)$ で多様体に沿って更新する。

▶ **本書との関係：** 本書のStep 4〜9は、まさにこのカテゴリに属する。引き戻し計量の学習（Step 4）、クリストッフェル記号の計算（Step 5）、測地線の算出（Step 9）はすべてリーマン幾何学ベースの手法。さらに本書は、これをデータ分析だけでなく**制御シミュレーション**にまで拡張している点が特徴的。

---

2. グラフ・ネットワークベースのモデル

▶ 数学の基礎から理解したい方へ

GCN（Graph Convolutional Network）： 画像におけるCNNのグラフ版。各ノードの特徴量を、隣接ノードの特徴量で畳み込み（集約）して更新する。

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \frac{1}{\sqrt{d_i d_j}} W^{(l)} h_j^{(l)}\right)$$

ここで $h_i^{(l)}$ はノード i の第 l 層の特徴量、$\mathcal{N}(i)$ は隣接ノード集合、d_i は次数、$W^{(l)}$ は学習パラメータ、σ は活性化関数。

▶ AIモデルの基礎から理解したい方へ

GCNの数式を「日常言語」で翻訳すると：「友達の情報を集めて、自分の情報を更新する」操作です。σ は活性化関数（非線形性の導入）、$W^{(l)}$ はどの情報を重視するかの重み（学習される）、$1/\sqrt{d_i d_j}$ は正規化項（友達が多いノードの影響を適度に抑制）。

GAT（Graph Attention Network）： 隣接ノードへの重み付けを、注意機構（attention mechanism）で動的に決定。「重要な隣人」に大きな重みを与える。

GraphSAGE： 大規模グラフ（数百万ノード）に対応。全隣接ノードではなくサンプリングした隣接ノードの情報を集約。

応用例： 分子の構造から物性予測、ソーシャルネットワークのコミュニティ検出、サプライチェーンのボトルネック予測、推薦システム。

▶ **本書との関係：** 本書のStep 3（VDM）はデータ点間のグラフを構成し、接続ラプラシアンを計算する。GNNがグラフ上での学習を目的とするのに対し、本書はグラフを**連続多様体への橋渡し**として用いる。

---

3. トポロジカルデータ解析（TDA）

▶ 数学の基礎から理解したい方へ

パーシステントホモロジー（Persistent Homology）： スケールパラメータ ε を動かしながらデータの位相構造（穴、ループ、空洞）を追跡する。結果はバーコード図（各位相的特徴の「誕生」と「消滅」のスケール）として可視化。

Mapperアルゴリズム： 高次元データを低次元に射影し、被覆（cover）とクラスタリングを組み合わせてデータの「骨格」を構成する。

▶ **本書との関係：** TDAは本書の多様体微分解析と**相補的**。TDAがデータの位相構造（穴の数）を定性的に捉えるのに対し、本書は微分構造（曲率、ベクトル場）を定量的に扱う。**制御シミュレーションは本書の手法でのみ可能。**

---

4. 群・対称性を考慮したモデル

▶ 数学の基礎から理解したい方へ

群 G がデータに作用するとき、ネットワーク f が以下を満たすように設計：

- **等変性（equivariance）：** $f(g \cdot x) = g \cdot f(x)$
- **不変性（invariance）：** $f(g \cdot x) = f(x)$

SE(3)-Equivariant Networks： 3次元の回転・平行移動に対して等変。分子の3D構造から物性を予測する際に、分子をどの向きで入力しても同じ結果を出力。AlphaFoldの基盤技術の一つ。

▶ **本書との関係：** Lie群はLie微分（Step 7）と深く関連。Equivariant Networksの「対称性の保存」は、本書のKilling方程式（$\mathcal{L}_V g = 0$）と同じ数学的精神。

---

5. 非ユークリッド空間の埋め込みモデル

▶ **数学の基礎から理解したい方へ**

Poincaré Embeddings (Nickel & Kiela, 2017) ： ポアンカレ円板モデル（双曲空間の一つのモデル）上に、単語や概念の階層関係を埋め込む。双曲空間の体積は指数関数的に増えるため、木構造との相性が抜群。WordNetのような概念階層を、わずか2次元の双曲空間で効率的に表現。ユークリッド空間では数百次元が必要な表現が、双曲空間では5〜10次元で十分。

球面・トーラス埋め込み： 周期的なデータを球面 S^n やトーラス T^n 上に埋め込む。

▶ **本書との関係：** 本書ではVAEの潜在空間を $\mathbb{R}^d$ としているが、潜在空間を双曲空間やSPD多様体に設定するVAEの研究も進んでいる。

---

6. Geometric Deep Learning（統一枠組み）

▶ **数学の基礎から理解したい方へ**

Bronstein et al. (2021) が提唱した統一枠組み。上記のすべてを「データが存在するドメイン（グラフ、多様体、群、etc.）の構造を活用した深層学習」として統一的に捉える。PointNet（点群学習）、MeshCNN（メッシュ学習）、Spectral Graph CNN（スペクトルグラフ畳み込み）、Riemannian VAE等を包含。

▶ **本書との関係：** 本書のStep 2（VAE）とStep 4（計量学習）は、Bronsteinの枠組みでは「多様体ドメイン上の信号処理」として位置づけられる。

---

7. 情報幾何学

▶ **数学の基礎から理解したい方へ**

情報幾何学（Amari, 1985）は、確率分布の族をパラメータ空間上のリーマン多様体として捉える。**フィッシャー情報行列**

$$g_{ij}(\theta) = \mathbb{E}\left[\frac{\partial \log p(x;\theta)}{\partial \theta^i}\frac{\partial \log p(x;\theta)}{\partial \theta^j}\right]$$

をリーマン計量として導入。これは「パラメータを少し変えたとき、確率分布がどれだけ変わるか」を測る指標であり、本書の引き戻し計量 $g_{ij}(z) = \left(J^\top J\right)_{ij}$ と**数学的に同じ構造**を持つ。

▶ AIモデルの基礎から理解したい方へ

フィッシャー情報行列と引き戻し計量の対応関係：

情報幾何学	本書
パラメータ θ を少し変える	潜在変数 z を少し変える
確率分布 $p(x;\theta)$ がどれだけ変わるか	デコーダ出力 $f_\theta(z)$ がどれだけ変わるか
フィッシャー情報行列 $g_{ij}(\theta)$	引き戻し計量 $g_{ij}(z)$

どちらも**変化の感度を計量として定義する**という同じ数学的構造。

自然勾配法（Natural Gradient Descent）： ユークリッド勾配 ∇L の代わりに、フィッシャー情報行列 G で補正した勾配 $G^{-1}\nabla L$ で更新。パラメータ空間の「歪み」を考慮するため、通常の勾配降下法よりも効率的に収束。TRPO（Trust Region Policy Optimization）はこの応用。

$$\theta_{t+1} = \theta_t - \eta \cdot G(\theta_t)^{-1}\nabla L(\theta_t)$$

---

8. その他の幾何学的モデル

▶ 数学の基礎から理解したい方へ

モース理論： 多様体上の滑らかな関数の臨界点（極大・極小・鞍点）から多様体の位相構造を読み取る。データの密度関数をモース関数と見なし、臨界点からクラスタや異常値を検出。

Ollivier-リッチ曲率： グラフ上で離散的にリッチ曲率を定義。コミュニティ構造の検出やネットワークのボトルネック特定に用いる。

Sheaf Neural Networks（Hansen & Gebhart, 2020; Bodnar et al., 2022）： 代数幾何学の**層（Sheaf）理論**をGNNに応用。各ノードに固有の「ローカル情報空間」を持たせ、エッジごとに**制限写像（翻訳行列）**を学習。Sheaf Laplacian

$$(L_{\mathcal{F}}\mathbf{x})_v = \sum_{e=(u,v)\in E} \mathcal{F}_{v\leftarrow e}^{\top}\left(\mathcal{F}_{v\leftarrow e}x_v - \mathcal{F}_{u\leftarrow e}x_u\right)$$

により、通常のGNNの過剰平滑化問題とヘテロフィリーへの弱さを劇的に改善（精度50-60% → 80-90%の事例あり）。

層理論とデータサイエンスの将来。 Sheaf Neural Networkは、**抽象代数幾何学の概念が実用的な機械学習の性能を劇的に改善する**という、数学とAIの驚くべき接点。さらに圏論的機械学習への発展も進んでいる。

```
import torch
import torch.nn as nn

class SheafDiffusion(nn.Module):
    """Sheaf Neural Networkの簡略化された実装"""
    def __init__(self, n_features, n_edge_features, n_edges):
        super().__init__()
        self.restriction_maps = nn.ParameterList([
            nn.Parameter(torch.randn(n_edge_features, n_features))
            for _ in range(n_edges * 2)
        ])

    def sheaf_laplacian(self, x, edge_index):
        """Sheaf Laplacian を計算"""
        out = torch.zeros_like(x)
        for idx, (u, v) in enumerate(edge_index.T):
            F_v = self.restriction_maps[2 * idx]
            F_u = self.restriction_maps[2 * idx + 1]
            diff = F_v @ x[v] - F_u @ x[u]
            out[v] += F_v.T @ diff
        return out

    def forward(self, x, edge_index):
        Lx = self.sheaf_laplacian(x, edge_index)
        return x - 0.1 * Lx
```

▶ ビジネス・政策の意思決定に活用したい方へ

どの手法を使うべきか—判断の指針

あなたのデータの性質	最適な手法	本書との関係
連続的な指標が多数（経営指標、経済指標）	**本書の手法**（多様体微分解析）	まさに本書

あなたのデータの性質	最適な手法	本書との関係
ネットワーク構造が重要（サプライチェーン、同盟関係）	GNN（GCN, GAT）	本書のVDMと補完的
「穴」や「ループ」の構造を調べたい	TDA（パーシステントホモロジー）	本書と相補的
階層構造のデータ（組織図、分類体系）	Poincaré Embeddings	本書の拡張可能
物理的対称性が重要（分子、3D形状）	Equivariant NN	本書のKilling場と同精神

本章の品質チェックポイント

この章には品質チェックポイントはありません。 本章は概観章であり、計算を伴わないためです。ただし、本章の内容を踏まえて、「自分の問題に本書の手法が適切かどうか」を改めて判断してください。もしデータの主要な構造がグラフ的（ネットワーク的）であり、連続的な多様体構造よりもノード間の関係性が重要であるなら、GNNベースのアプローチの方が適切かもしれません。

第II部完了。 幾何学的AIの4つの道具と、Geometric Data Scienceの全体像が揃いました。

第III部予告： 第9章から、いよいよ実践に入ります。

第III部：社会データからの多様体構築と分析

第I部で数学の基礎を、第II部で幾何学的AIの道具を学びました。第III部ではいよいよ実践に入ります。ケーススタディA（X社の経営環境多様体）とケーススタディB（国際情勢多様体）を、具体的なPythonコードとともに実行します。

第9章：データ収集とVAEによる多様体構成（Step 1-2）

この章で得られるもの—コストに見合うリターン

この章は、実践の出発点です。生のビジネスデータや政策データを、「計算可能な多様体」に変換する最初のステップ—データ収集・前処理（Step 1）とVAEによる潜在多様体の構成（Step 2）—を実行します。**この2ステップの品質が、以降のすべての分析の信頼性を決定します。**

▶ 数学の基礎から理解したい方へ

Step 1 は、観測データ $X = \{x_i\}_{i=1}^{N} \subset \mathbb{R}^n$ を数値行列として整備する前処理段階。欠損値処理（多重代入法、k-NN補完）、カテゴリ変数の数値化（one-hot, entity embedding）、標準化 $\tilde{x}^{(j)} = \left(x^{(j)} - \mu_j\right)/\sigma_j$、時系列平滑化（カーネル平滑化）を行う。

Step 2 は、仮定A1（多様体仮説）とA2（デコーダ正則性）の下で、VAEのデコーダ $f_\theta : U \subset \mathbb{R}^d \to \mathbb{R}^n$ が C^k 級（$k \geq 3$）埋め込み部分多様体 $f_\theta(U) \subset \mathbb{R}^n$ を構成することを保証する段階。命題1.1の3条件（C^k 級、フルランク、単射）の検証を含む。

コスト： データ収集・整備の実務的労力、VAEの設計・訓練・検証の計算時間。
リターン： 以降のStep 3〜10のすべての計算の「舞台」となる可微分多様体の構成。この舞台が不適切であれば、曲率もLie微分も最適制御も、すべて意味を失う。

▶ AIモデルの基礎から理解したい方へ

Step 1の前処理は通常のデータサイエンスの手順です。Step 2のVAE構成は第5章で詳しく学んだ内容の実践です。本章では、第5章の知識を使って、実際のデータに対してVAEを訓練し、命題1.1の条件を検証する完全なパイプラインを示します。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社が集めるデータセット

X社のデータサイエンスチームは以下のデータを収集しました。

自社データ（50店舗×月次×10年）： 売上高、客単価、来店数、在庫回転率、従業員数、顧客満足度、リピート率、客層比率、商品カテゴリ別売上構成比、店舗面積、営業時間、広告宣伝費（計15指標）。

マクロ経済データ（月次×10年）： GDP成長率、インフレ率（CPI）、失業率、消費者信頼感指数、為替レート（USD/JPY）、金利（10年国債利回り）、原油価格、鉱工業生産指数（計8指標）。

業界データ（月次×10年）： 市場規模推定値、主要競合3社の売上推定値、原材料価格指数、サプライチェーン遅延指数、特許出願数、業界平均利益率（計7指標）。

合計30指標×6,000データ点。

ケーススタディB：安全保障チームが集めるデータセット

軍事指標（193カ国×年次×30年）： 軍事費/GDP比、核弾頭数、現役兵力数、主要装備数、サイバー能力指数、宇宙能力指数（計6指標）。

外交指標： 同盟関係ネットワーク（二国間同盟の有無と強度）、国連総会投票一致度行列、二国間条約数、外交官交換数、首脳会談頻度（計5指標）。

経済安全保障指標： 二国間貿易額/GDP比、エネルギー輸入依存度、レアアース調達先集中度、半導体サプライチェーン依存度、金融制裁脆弱性指標（ドル建て取引依存度）（計5指標）。

社会・政治指標： 民主主義指数、報道の自由度、政権安定性指数、国内世論の対外政策支持率（計4指標）。

合計40指標。年次データ5,790点＋月次貿易金融データ追加で約70,000点。

Step 1の実装

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.impute import KNNImputer

def preprocess_social_data(df):
    """
    Step 1: 社会データの前処理パイプライン

    1. KNN補完で欠損値を処理
    2. 標準化（平均0、標準偏差1）
    """
    # 1. 欠損値をKNN補完
```

```
    imputer = KNNImputer(n_neighbors=5)
    df_numeric = df.select_dtypes(include=[np.number])
    df_imputed = pd.DataFrame(
        imputer.fit_transform(df_numeric),
        columns=df_numeric.columns
    )

    # 2. 標準化
    scaler = StandardScaler()
    X = scaler.fit_transform(df_imputed)

    return X, scaler

# 使用例
df = pd.DataFrame(np.random.randn(6000, 30))  # X社のデータ（模擬）
df.iloc[5, 3] = np.nan  # 欠損値の例
X, scaler = preprocess_social_data(df)
print(f"Preprocessed: shape={X.shape}, mean={X.mean():.4f}, std={X.std():.4f}")
```

Step 2の実装：VAE訓練とヤコビ行列検証

```
import torch
import torch.nn as nn

# VAE定義（第5章のコードを再掲）
class VAE(nn.Module):
    def __init__(self, input_dim=30, latent_dim=6, hidden_dim=128):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim//2), nn.Tanh())
        self.mu_layer = nn.Linear(hidden_dim//2, latent_dim)
        self.logvar_layer = nn.Linear(hidden_dim//2, latent_dim)
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim//2), nn.Tanh(),
            nn.Linear(hidden_dim//2, hidden_dim), nn.Tanh(),
            nn.Linear(hidden_dim, input_dim))

    def encode(self, x):
        h = self.encoder(x)
        return self.mu_layer(h), self.logvar_layer(h)
```

```
    def reparameterize(self, mu, logvar):
        return mu + torch.exp(0.5*logvar) * torch.randn_like(mu)

    def decode(self, z): return self.decoder(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

def verify_jacobian_rank(decoder, z_samples, latent_dim):
    """Step 2の核心：ヤコビ行列のフルランク検証"""
    min_sv = float('inf')
    for z in z_samples:
        z = z.requires_grad_(True)
        f = decoder(z)
        J = torch.zeros(f.shape[0], z.shape[0])
        for i in range(f.shape[0]):
            grad = torch.autograd.grad(f[i], z, retain_graph=True)[0]
            J[i] = grad
        svs = torch.linalg.svdvals(J)
        min_sv = min(min_sv, svs[-1].item())

    print(f"Minimum singular value: {min_sv:.6f}")
    if min_sv > 1e-4:
        print("✓ Full rank condition satisfied (Assumption A2(b))")
    else:
        print("✗ WARNING: Jacobian may be rank-deficient")
    return min_sv
```

本章の品質チェックポイント—先に進むべきか、戻るべきか？

▶ 全読者共通

これは本書全体で最も重要なチェックポイントです。 ここでの合否が、以降6章分の分析の信頼性を決定します。

1. **PCAで固有次元を確認したか？** → d の選択根拠。
2. **データ量は $N > 100 \times 2^d$ を満たすか？** → 不足なら追加データ収集。

3. **VAEの再構成誤差は分散の10%以下か？** → 超えるならアーキテクチャ変更。
4. **ヤコビ行列の最小特異値 $> 10^{-4}$ か？** → 不合格なら d を下げるか領域除外。
5. **活性化関数は** tanh/GELU/Softplus **か？** → ReLU使用中なら変更必須。

1つでも不合格の場合、先に進まないでください。 このステップをやり直してください。

第10章：幾何構造抽出とリーマン計量の学習（Step 3-4）

この章で得られるもの—コストに見合うリターン

Step 3（VDMによる幾何構造抽出）とStep 4（引き戻し計量の計算）を実行します。多様体に「距離」が定義され、経営状態間・国際情勢間の定量的な距離比較が可能になります。

▶ 数学の基礎から理解したい方へ

Step 3では VDM（第6章）を VAE潜在空間に適用し、接続ラプラシアンのスペクトルから多様体の大域的幾何構造を抽出する。Step 4では引き戻し計量 $g_{ij}(z) = \left(J_{f_\theta}^{\top} J_{f_\theta}\right)_{ij}$ を潜在空間の格子点上で計算する。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—「距離」の定義

「新興国市場と国内市場のリーマン距離は5.8。欧州市場との距離は2.1。」この数値により、「どの市場が構造的に最も移行しやすいか」が定量的に判断可能に。

```
import torch
import numpy as np

def compute_metric_field(decoder, grid_points, d=2):
    """Step 4: 格子点上で引き戻し計量を計算"""
    metrics = []
    for z in grid_points:
```

```
        z_t = torch.tensor(z, dtype=torch.float32).requires_grad_(True)
        f = decoder(z_t)
        J = torch.zeros(f.shape[0], d)
        for a in range(f.shape[0]):
            grad = torch.autograd.grad(f[a], z_t, retain_graph=True, create_grap
h=True)[0]
            J[a] = grad
        g = (J.T @ J).detach().numpy()
        metrics.append(g)
    return np.array(metrics)

print("Metric field computation ready")
```

本章の品質チェックポイント

1. **計量は全格子点で正定値か？** → 全固有値が正であることを確認。
2. **条件数は** 10^4 **以下か？** → 超える領域は信頼性低。
3. **リーマン距離は業界の直感と整合するか？** → 非現実的なら第9章に戻る。

第11章：曲率テンソルの計算と経営環境・国際情勢の構造分析（Step 5-6）

この章で得られるもの—コストに見合うリターン

計量からクリストッフェル記号（Step 5）と曲率テンソル（Step 6）を計算します。スカラー曲率マップにより、「安定領域」と「不安定領域」が可視化されます。**これは本書のアプローチが提供する最も独自なリターンの1つです。**

▶ 数学の基礎から理解したい方へ

Step 5. クリストッフェル記号 $\Gamma^k_{ij}(z) = \frac{1}{2}g^{kl}\left(\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij}\right)$ を自動微分で計算。

Step 6. リーマン曲率テンソル $R^l{}_{ijk} = \partial_j \Gamma^l_{ik} - \partial_i \Gamma^l_{jk} + \Gamma^l_{jm}\Gamma^m_{ik} - \Gamma^l_{im}\Gamma^m_{jk}$。リッチテンソル $\mathrm{Ric}_{ij} = R^l{}_{ilj}$。スカラー曲率 $\mathrm{Scal} = g^{ij}\mathrm{Ric}_{ij}$。

:::message alert **滑らかさの連鎖：** 曲率テンソル計算には $g_{ij} \in C^2 \rightarrow f_\theta \in C^3$ が必要。ReLUは使用不可。 :::

```
import torch

def riemann_curvature(Gamma_fn, z, d):
    """Step 6: リーマン曲率テンソルを自動微分で計算"""
    # Γの微分が必要なため、create_graph=Trueで計算
    #  (完全な実装は第7章のコードを拡張)
    print("Curvature tensor computation (requires C³ decoder)")
    return None  # Full implementation: see Chapter 7 extended
```

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—スカラー曲率マップ

チームは潜在空間の格子点上でスカラー曲率を計算し、カラーマップとして経営会議に提示。

発見： 国内市場中心部は正の曲率（安定）。国内市場と新興国市場の境界にスカラー曲率 -2.3 の不安定領域。過去のリーマンショック・COVID-19時の経営状態がこの領域に位置することを確認。

経営者への示唆： 新興国進出経路がこの不安定領域を通過する場合、リスクバッファ（予備資金、段階的投資、撤退条件）が不可欠。

ケーススタディB：国際情勢の構造的脆弱点

「軍事費急増＋同盟弱体化＋貿易依存度非対称化」が同時に進行する領域で強い負の曲率。**この領域は国際秩序の構造的脆弱点であり、現在の国際状態からの距離がリスク評価の核心指標。**

```
import plotly.graph_objects as go
import numpy as np

def visualize_curvature_map(z_grid, scalar_curvature, title="Scalar Curvature Ma
p"):
    """Step 6の結果を可視化: スカラー曲率マップ"""
    fig = go.Figure(data=go.Heatmap(
        z=scalar_curvature,
        colorscale='RdBu_r',
        zmid=0))
    fig.update_layout(title=title, xaxis_title='z¹', yaxis_title='z²')
```

```
    fig.show()

print("Visualization tools ready")
```

本章の品質チェックポイント

1. **曲率計算でNaN/Infが出ていないか？** → 出た場合は計量の条件数を確認。
2. **曲率パターンはランダムシード変更で安定か？** → 不安定なら曲率に基づく判断は控える。
3. **不安定領域は過去の既知危機と整合するか？** → 整合しなければ第9章に戻ることを検討。

第12章：Lie微分による政策・戦略の影響分析（Step 7）

この章で得られるもの—コストに見合うリターン

この章は、経営者・政策立案者にとってのクライマックスです。 Lie微分の計算により、「この施策は経営環境の構造を保存するか、変形するか」「2つの施策は干渉するか」が数値で示されます。

▶ 数学の基礎から理解したい方へ

Step 7. 政策ベクトル場 $V \in \mathfrak{X}(M)$ の計量に対するLie微分

$$(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$$

を計算する。$\mathcal{L}_V g = 0$（Killing方程式）なら構造保存型、$\mathcal{L}_V g \neq 0$ なら構造変形型。2つのベクトル場のLie括弧 $[V_A, V_B]$ で干渉を判定。

▶ AIモデルの基礎から理解したい方へ

政策ベクトル場 V は時系列データの差分 $V(z) \approx \Delta z / \Delta t$ からニューラルネットで学習・推定。Lie微分はautogradでヤコビ行列を計算し、第4章のコードを適用。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—3つの施策の構造的インパクト

施策	$\lVert \mathcal{L}_V g \rVert$	分類	経営判断への示唆
既存顧客サービス強化	0.12	構造保存型	低リスク・低リターン。安定期に適切。
新興国市場段階的進出	1.8	構造変形型（中）	中リスク・中リターン。段階的管理が可能。
事業ポートフォリオ抜本再編	4.5	構造変形型（強）	高リスク・高リターン。十分な準備と覚悟が必要。

Lie括弧分析： $[V_{\text{cost}}, V_{\text{market}}]$ が移行途上の領域で大きい → 「コスト削減と市場開拓を同時進行するな。順序を分けよ。」

ケーススタディB：安全保障政策の構造的影響

同盟強化政策 V_{alliance} のLie微分ノルムは1.2（中程度の構造変形）。経済制裁政策 V_{sanction} は2.8（強い構造変形）。

$[V_{\text{sanction}}, V_{\text{alliance}}]$ が特定の緊張状態で大きい → **「まず同盟を強化し、その後に制裁を発動する」順序が最適。**

```python
# Step 7: ベクトル場のLie微分と計量のLie微分の計算
#  (第4章のコードを実データに適用)

import torch

def compute_lie_derivative_norm(V_net, metric_fn, z_grid, Gamma_fn, d):
    """格子点上でLie微分のノルムを計算し、マップとして返す"""
    norms = []
    for z in z_grid:
        z_t = torch.tensor(z, dtype=torch.float32)
        Gamma = Gamma_fn(z_t)
        L_V_g = lie_derivative_metric(V_net, metric_fn, z_t, Gamma, d)
        norm = torch.norm(L_V_g).item()
        norms.append(norm)
    return norms

print("Lie derivative computation ready")
```

本章の品質チェックポイント

1. **ベクトル場の学習は安定か？** → 複数回訓練して一致確認。
2. **Lie微分のノルムの順序はドメイン知識と整合するか？** → 不整合なら計量（第10章）に遡る。
3. **Lie括弧の干渉パターンは安定か？** → 不安定なら干渉に基づく判断は控える。

第13章：制御シミュレーションと最適パスの算出（Step 8-9）

この章で得られるもの—コストに見合うリターン

多様体上の制御系を定式化し（Step 8）、最適パスを計算します（Step 9）。「現在の経営状態から目標状態への、最小リスクの移行経路」が算出されます。

▶ 数学の基礎から理解したい方へ

Step 8. アフィン制御系：

$$\dot{z}^k(t) = F^k\big(z(t)\big) + \sum_{\alpha=1}^{m} u_\alpha(t) B_\alpha^k\big(z(t)\big)$$

F：ドリフト（自然な変化）。B_α：制御ベクトル場（施策 α の効果）。u_α：制御入力（施策の強度）。F, B_α はNeural ODEでパラメトリゼーション。

Step 9. 測地線方程式：

$$\ddot{\gamma}^k + \Gamma_{ij}^k \dot{\gamma}^i \dot{\gamma}^j = 0$$

ポントリャーギンの最大原理：ハミルトニアン $H(z,p,u) = p_k(F^k + u_\alpha B_\alpha^k) - \ell(z,u)$、正準方程式 $\dot{z}^k = \partial H / \partial p_k$, $\dot{p}_k = -\partial H / \partial z^k$ から最適制御 $u^*(t)$ と最適経路 $z^*(t)$ を計算。

▶ AIモデルの基礎から理解したい方へ

Neural ODEで F と B_α を学習し、`torchdiffeq.odeint` で数値解を計算。

```python
import torch
import torch.nn as nn
from torchdiffeq import odeint

class ManifoldControlODE(nn.Module):
    """Step 8: 多様体上のアフィン制御系"""
    def __init__(self, dim, n_controls):
        super().__init__()
        self.drift = nn.Sequential(
            nn.Linear(dim, 32), nn.Tanh(), nn.Linear(32, dim))
        self.control_fields = nn.ModuleList([
            nn.Sequential(nn.Linear(dim, 32), nn.Tanh(), nn.Linear(32, dim))
            for _ in range(n_controls)])

    def forward(self, t, z, u=None):
        dz = self.drift(z)
        if u is not None:
            for alpha, B in enumerate(self.control_fields):
                dz = dz + u[alpha] * B(z)
        return dz

def geodesic_ode(Gamma_fn):
    """Step 9: 測地線方程式をODEとして定式化"""
    def func(t, state):
        d = state.shape[0] // 2
        z, v = state[:d], state[d:]
        Gamma = Gamma_fn(z)
        dz = v
        dv = torch.zeros(d)
        for k in range(d):
            for i in range(d):
                for j in range(d):
                    dv[k] -= Gamma[k, i, j] * v[i] * v[j]
        return torch.cat([dz, dv])
    return func

print("Manifold control ODE and geodesic formulation ready")
```

▶ **ビジネス・政策の意思決定に活用したい方へ**

ケーススタディA：X社—新興国市場への最適移行経路

チームは3つの経路を計算しました。

経路1（直線的計画）： ユークリッド距離で最短。しかし、曲率負の不安定領域を通過。リーマンショック級の危機が発生した場合、経路上で経営が破綻するリスク。

経路2（測地線）： リーマン距離で最短。不安定領域を迂回するため、ユークリッド的には遠回りだが、多様体上では最もエネルギーが低い経路。

経路3（制御付き最適経路）： 利用可能な3施策（サービス強化、段階的進出、合弁事業）の制約下で、移行コストを最小化する経路。ポントリャーギンの最大原理により、「最初の6ヶ月はサービス強化を最大強度、次の12ヶ月は段階的進出を中強度、最後の6ヶ月は合弁事業を高強度」という最適スケジュールが算出。

経営者への示唆： 経路1（直線的計画）は最短だが最もリスクが高い。経路3（制御付き最適経路）は最もリスクが低く、具体的な施策スケジュールが付いている。**経路3を推奨。**

ケーススタディB：安全保障環境の最適政策経路

「現在の安全保障環境」から「目標とする国際秩序」への最小リスク経路を計算。「まず同盟ネットワークの強化（12ヶ月）→ 次に経済安全保障の多角化（18ヶ月）→ その後に緊張緩和のための外交イニシアチブ（12ヶ月）」という段階的政策が最適と算出。

本章の品質チェックポイント

1. **制御系の軌道は発散しないか？** → 物理的に妥当な範囲に収まるか確認。
2. **測地線は不安定領域を適切に迂回しているか？** → 曲率マップ（第11章）と照合。
3. **最適経路はパラメータ変更で安定か？** → 初期条件やコスト関数を変えて安定性確認。

第14章：可視化・解釈・意思決定（Step 10）

この章で得られるもの—コストに見合うリターン

第9〜13章で計算したすべての結果を統合し、可視化し、意思決定に結びつけます。**本書のゴールはここです。**

▶ 数学の基礎から理解したい方へ

Step 10は数学的には新たな概念を導入しない。潜在空間の可視化（$d \leq 3$ は直接、$d > 3$ はt-SNE/UMAP投影）、スカラー曲率マップ、測地線と最適パスの描画、Lie微分場の可視化を行う。

:::message alert **因果推論との区別：** 本手順の幾何構造は**非線形相関構造**であり、因果関係ではない。因果推論には介入データ、構造因果モデル、自然実験が別途必要。 :::

誤差の累積： Step 2のデコーダ誤差 → Step 3の離散化誤差 → Step 4の計量誤差は下流に伝播する。特に曲率は2階微分を含み、誤差に敏感。ロバスト性チェックが不可欠。

▶ ビジネス・政策の意思決定に活用したい方へ

ケーススタディA：X社—経営会議への報告

X社のデータサイエンスチームは、経営会議に以下の資料を提示しました。

資料1：経営環境の地形図（スカラー曲率マップ）。 安定領域（赤）と不安定領域（青）が色分けされた6次元多様体の2次元投影。過去のリーマンショック・COVID-19の位置をプロット。

資料2：施策のLie微分分析。 3施策の構造的インパクト（$\| \mathcal{L}_V g \|$）の比較表。コスト削減と市場開拓の干渉分析（Lie括弧）。

資料3：最適移行経路。 国内市場→新興国市場への3経路の比較（直線、測地線、制御付き最適経路）。推奨経路と施策スケジュール。

資料4：信頼度マップ。 データが密な領域（信頼度高）とデータが疎な領域（信頼度低）の色分け。「この分析は全領域で均一に信頼できるわけではない」ことを明示。

経営者の判断： 「経路3（制御付き最適経路）を採用する。ただし、不安定領域の通過時にはリスクバッファとして予備資金3億円を確保する。Lie括弧分析に基づき、コスト削減と市場開拓は同時並行ではなく、段階的に実施する。」

ケーススタディB：安全保障会議への報告書

安全保障分析チームは、以下の構成で報告書を作成しました。

セクション1：国際情勢の安定性評価。 曲率分析に基づく「構造的脆弱点」の特定。現在の国際状態と脆弱領域との距離。

セクション2：政策オプションの構造的分析。 同盟強化、経済制裁、外交イニシアチブの各政策のLie微分分析。政策間の干渉分析（Lie括弧）。

セクション3：最適政策経路。 現在の安全保障環境から目標とする国際秩序への最小リスク経路。段階的政策スケジュール。

セクション4：信頼度と限界。 データが疎な領域のリスト。分析の限界（非線形相関であり因果関係ではないこと）。ロバスト性チェックの結果。

```
import plotly.graph_objects as go
import numpy as np

def visualize_curvature_map(z_grid, scalar_curvature, title="Scalar Curvature Ma
p"):
    """Step 10: 対話的な曲率マップの描画"""
    fig = go.Figure(data=go.Heatmap(
        z=scalar_curvature,
        colorscale='RdBu_r',
        zmid=0))
    fig.update_layout(title=title, xaxis_title='z¹', yaxis_title='z²')
    fig.show()

print("Visualization tools ready")
```

本章の品質チェックポイント—最終チェック

このチェックポイントは本書全体の最終確認です。

1. 曲率マップは過去の既知イベントと整合するか？
2. Lie微分の順序はドメイン知識と整合するか？
3. 最適経路は物理的に妥当か（発散しない、値域が合理的）？
4. ランダムシード変更、データ10%除去で結果は安定か？
5. 結果は「探索的洞察」として提示されているか（確定的結論ではなく）？
6. 因果関係と相関の区別が明記されているか？

すべてYesなら、分析結果を意思決定の参考情報として活用できます。1つでもNoなら、該当する章に戻ってやり直してください。

付録A：仮定の一覧と数学的正当性

- **A1. 多様体仮説：** データが低次元 C^∞ 部分多様体の近傍から標本されている。
- **A2. デコーダの正則性：** (a) $f_\theta \in C^k$ ($k \geq 3$)。(b) $\mathrm{rank}\, J_{f_\theta}(z) = d$ ($\forall z \in U$)。
- **A3. 十分なデータ量：** $N\varepsilon^{d/2+1} \to \infty$ のレート条件。
- **A4. 数値計算の安定性：** $\kappa(g(z))$ が有界。

仮定	破れた場合の影響
A1	多様体自体が存在しない。全手順が無意味
A2(a)	曲率テンソルが定義できない（ReLU使用時）
A2(b)	計量が退化し、逆行列が存在しない
A3	VDMの幾何構造抽出が不正確
A4	テンソル計算が数値的に不安定

付録B：Python環境セットアップ

```
# 基本環境
pip install numpy pandas scikit-learn matplotlib plotly

# 深層学習
pip install torch torchvision torchdiffeq

# TDA（位相的データ解析、第8章で使用）
pip install ripser persim giotto-tda

# 固有次元推定（チェック1で使用）
pip install scikit-dimension

# 可視化
pip install plotly jupyter
```

付録C：参考文献

1. J. M. Lee, *Introduction to Smooth Manifolds*, 2nd ed., Springer, 2012.
2. A. Singer and H.-T. Wu, "Vector Diffusion Maps and the Connection Laplacian," *Comm. Pure Appl. Math.*, 65(8), 2012.
3. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," *Proc. ICLR*, 2014.
4. R. T. Q. Chen et al., "Neural Ordinary Differential Equations," *Proc. NeurIPS*, 2018.
5. G. Arvanitidis et al., "Latent Space Oddity," *Proc. ICLR*, 2018.
6. E. Facco et al., "Estimating the intrinsic dimension of datasets," *Scientific Reports*, 7, 2017.
7. E. Levina and P. J. Bickel, "Maximum likelihood estimation of intrinsic dimension," *Proc. NeurIPS*, 2004.
8. L. S. Pontryagin et al., *The Mathematical Theory of Optimal Processes*, 1962.
9. M. M. Bronstein et al., "Geometric Deep Learning," *arXiv:2104.13478*, 2021.
10. M. Gidea and Y. Katz, "Topological Data Analysis of Financial Time Series," *Frontiers in Applied Mathematics and Statistics*, 2018.
11. J. Hansen and T. Gebhart, "Sheaf Neural Networks," *NeurIPS Workshop*, 2020.
12. C. Bodnar et al., "Neural Sheaf Diffusion," *Proc. ICLR*, 2022.
13. S. Amari, *Information Geometry and Its Applications*, Springer, 2016.